UEMF | École d'Ingénierie Digitale et d'Intelligence Artificielle

EUROMED UNIVERSITY OF FES

Euromed School of Digital Engineering and Artificial Intelligence

Academic year 2025–2026

State engineering cycle in big data analytics

End of Studies Project

Design and Development of an Intelligent Platform for Clinical Risk Analysis and Prediction in Nephrology Using Big Data and Machine Learning Techniques.

Presented by :

BEN-ALI Nisrine

Supervised by:

Prof. Sara Bakkali

Company supervisor:

Mr Zakaria Alami Merrouni

Defended in .....

Hosting Company:

HAIM-FMPDF

Jury Members:

.....

Dedication

To my parents, whose unconditional love and sacrifices
have been the foundation of every step I take.

To my family and friends,
for their endless support and encouragement.

To all patients suffering from kidney disease—
may this work contribute, however modestly,
to improving their care and quality of life.

Acknowledgment

First and foremost, I would like to express my sincere gratitude to all those who contributed, directly or indirectly, to the completion of my academic journey over the past three years at the **Euromed University of Fez**. This period has been a formative experience both intellectually and personally.

I would like to thank the entire academic staff of **EIDIA (École d'Ingénierie Digitale et d'Intelligence Artificielle)** for their dedication to providing high-quality education and for fostering an environment that encourages learning, innovation, and personal growth.

Special thanks go to **Mrs. Loubna Ourabah**, Head of the Big Data Analytics program at EIDIA, for her continuous guidance and valuable support throughout my studies.

I am also sincerely grateful to my academic supervisor, **Mrs. Sara Bakkali**, for her encouragement, insightful feedback, and constant support during the development of this final year project.

I would like to extend my gratitude to the **Sidi Mohamed Ben Abdallah University — Health Artificial Intelligence and Modeling Center (HAIM)**, Faculty of Medicine, Pharmacy and Dentistry of Fès (FMPDF), for providing an enriching research environment and access to clinical data from CHU Hassan II of Fès.

My deep appreciation goes to the nephrology team at **CHU Hassan II of Fès**, and in particular to the physicians and clinical staff who made the HD-478 cohort data available. Without their meticulous data collection spanning 2020–2024, no predictive model would have been possible.

Finally, I express my deepest appreciation to **Mr. Zakaria Alami Merrouni** for his technical guidance, support, and mentorship throughout the completion of this project.

I also thank the jury members for their time and thoughtful evaluation of this work, and my fellow students and friends for the intellectual exchanges and mutual support throughout this journey.

Abstract

Keywords: Hemodialysis, Mortality Prediction, Machine Learning, Support Vector Machine, Clinical Decision Support System, Feature Engineering, LLM Preprocessing, RAG, Django, React.

Chronic kidney disease (CKD) affects over 850 million people worldwide [3], with a significant fraction requiring renal replacement therapy through hemodialysis. One-year mortality rates in this population remain alarmingly high, driven by comorbidity burden, cardiovascular events, and nutritional decline. Clinicians currently lack integrated, data-driven tools capable of stratifying individual mortality risk at the point of care.

This report presents **AI NéphroCare**, a full-stack clinical decision support platform developed at the **HAIM Center** (Health Artificial Intelligence & Modeling Center, Faculty of Medicine, Pharmacy and Dentistry of Fes – Sidi Mohamed Ben Abdellah University). The platform integrates three main components: (1) a structured patient data management module supporting over 160 clinical fields across 11 medical sections; (2) an AI-powered preprocessing pipeline combining a Large Language Model (LLM, Ollama qwen2.5:14b) with Retrieval-Augmented Generation (RAG, ChromaDB) for intelligent data cleaning and validation; and (3) a machine learning prediction engine trained on the HD-478 cohort (478 hemodialysis patients, 95 deaths within one year, CHU Hassan II of Fes, 2020–2024).

Nine machine learning algorithms were systematically evaluated using stratified 10-fold cross-validation. A **linear SVM** emerged as the best-performing model (AUC-ROC = 0.8262, PR-AUC = 0.4887, F1 = 0.5667, Brier Score = 0.1265), combining strong discriminative ability with clinical interpretability. Raw SVM probabilities are calibrated via Platt scaling (`CalibratedClassifierCV`, 5-fold sigmoid) and mapped to three evidence-based clinical risk zones, with thresholds derived by ROC curve analysis (T1 = 10%, Sensitivity $\geq$ 90%) and bootstrapped Youden index (T2 = 29%, $n = 1000$ iterations, median = 28.9%): **Low Risk** ($\hat{p} < 10\%$, observed mortality 3.8%), **Moderate Risk** ($10\% \leq \hat{p} < 29\%$, 15.6%), and **High Risk** ($\hat{p} \geq 29\%$, 46.5%).

The platform is implemented using Django REST Framework, React 18, PostgreSQL 16, Redis, Celery, and Docker Compose, with role-based access control (RBAC) across four user profiles.

Résumé

Mots-clés : Hémodialyse, Prédiction de mortalité, Apprentissage automatique, SVM, Aide à la décision clinique, Ingénierie des features, LLM, RAG, Django, React.

La maladie rénale chronique (MRC) touche plus de 850 millions de personnes dans le monde, dont une proportion significative dépend de l'hémodialyse comme thérapie de remplacement rénal. La mortalité à un an dans cette population reste élevée, principalement due à la charge en comorbidités, aux événements cardiovasculaires et à la dénutrition. Les cliniciens manquent d'outils intégrés et fondés sur les données pour stratifier le risque de mortalité individuel au lit du patient.

Ce rapport présente **AI NéphroCare**, une plateforme d'aide à la décision clinique développée au sein du **Centre HAIM** (Health Artificial Intelligence & Modeling Center, FMPDF – Université Sidi Mohamed Ben Abdellah), en collaboration avec le Service de Néphrologie du CHU Hassan II de Fès. Elle intègre : (1) un module de gestion structurée des dossiers patients couvrant plus de 160 champs cliniques en 11 sections ; (2) un pipeline de prétraitement IA combinant un modèle de langage (LLM, Ollama qwen2.5 :14b) et une génération augmentée par récupération (RAG, ChromaDB) ; et (3) un moteur de prédiction ML entraîné sur la cohorte HD-478 (478 patients, 95 décès à 1 an, CHU Hassan II de Fès, 2020–2024).

Neuf algorithmes d'apprentissage automatique ont été comparés. Un **SVM linéaire** s'est imposé comme le meilleur modèle (AUC-ROC = 0,8262, PR-AUC = 0,4887, F1 = 0,5667, Brier = 0,1265). Les probabilités brutes sont calibrées par Platt scaling (`CalibratedClassifierCV`, sigmoïde 5-fold) puis classifiées en trois zones cliniques (Faible $< 10\%$, Modérée 10–29%, Élevée $\geq 29\%$) dont les seuils ont été dérivés par analyse de la courbe ROC (T1 = 10%, Sensibilité $\geq 90\%$) et indice de Youden bootstrappé (T2 = 29%, $n = 1000$, médiane = 28,9%), validés cliniquement par l'équipe de néphrologie du CHU Hassan II de Fès. La plateforme repose sur Django REST Framework, React 18, PostgreSQL 16 et Docker Compose.

Contents

List of Figures

List of Tables

List of Abbreviations

CDSS	Clinical Decision Support System
CKD	Chronic Kidney Disease
CRF	Chronic Renal Failure
HD	Hemodialysis
PD	Peritoneal Dialysis
CHU	University Hospital Center
HAIM	Health Artificial Intelligence & Modeling Center
FMPDF	Faculty of Medicine, Pharmacy and Dentistry of Fes
USMBA	Sidi Mohamed Ben Abdellah University
ML	Machine Learning
SVM	Support Vector Machine
RF	Random Forest
GB	Gradient Boosting
LR	Logistic Regression
AUC	Area Under the ROC Curve
ROC	Receiver Operating Characteristic
PR	Precision-Recall
CV	Cross-Validation
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
DRF	Django REST Framework
JWT	JSON Web Token
RBAC	Role-Based Access Control
API	Application Programming Interface
REST	Representational State Transfer
PTH	Parathyroid Hormone
AVF	Arteriovenous Fistula
KDOQI	Kidney Disease Outcomes Quality Initiative

KNN K-Nearest Neighbors

OOF Out-Of-Fold

CHAPTER 1

Overview of the Host Organization

Context, medical setting, and project objectives.

CONTENTS

CHAPTER 1

Overview of the Host Organization

1.1 Background

1.1.1 Digital Transformation in Healthcare Systems

The global healthcare sector is undergoing a profound digital transformation, driven by the exponential growth of medical data, the democratization of cloud computing, and the maturation of artificial intelligence (AI) technologies. Electronic Health Records (EHR) have become the standard in most developed countries, generating terabytes of structured and unstructured clinical data daily. According to the WHO Global Strategy on Digital Health 2020–2025, health systems must accelerate digital transformation to improve health outcomes and service delivery [1].

In low- and middle-income countries (LMICs), including Morocco, this transition is accelerating. The Moroccan Ministry of Health's national digital health strategy (2020–2025) prioritizes the deployment of integrated hospital information systems, telemedicine platforms, and AI-based decision support tools in university hospitals. CHU Hassan II of Fès, as one of the country's largest tertiary care centers, serves as a pilot site for several of these initiatives.

Despite this momentum, a significant gap remains between data availability and clinical utility. Patient records are often fragmented across departments, stored in heterogeneous formats (paper, Excel, proprietary HIS systems), and rarely exploited for predictive analytics. Bridging this gap — from raw clinical data to actionable risk predictions — is precisely the challenge addressed by this project.

1.1.2 Importance of Data-Driven Clinical Decision Support

Clinical decision-making is inherently complex. Physicians must integrate information from dozens of variables — biological markers, comorbidities, imaging findings, treatment history — under time pressure, and with incomplete information. Cognitive

overload, variable experience, and the sheer volume of data contribute to suboptimal decisions, particularly in high-acuity settings such as nephrology and intensive care.

Clinical Decision Support System (CDSS)

A CDSS is a health information technology system designed to assist clinicians in making clinical decisions by integrating patient-specific information with a curated medical knowledge base. CDSS can be rule-based (expert systems) or data-driven (machine learning models).

Data-driven CDSS offer several advantages over traditional rule-based systems:

— **Pattern discovery**: ML models can detect non-linear, high-dimensional relationships invisible to human experts.

— **Personalization**: Predictions are tailored to the individual patient profile rather than population-average guidelines.

— **Continuous learning**: Models can be retrained as new data accumulates, adapting to evolving patient populations.

— **Explainability**: Modern interpretability tools (SHAP, feature importance) make model reasoning accessible to clinicians.

The integration of CDSS in nephrology has shown particular promise. Early identification of high-risk dialysis patients enables timely interventions — nutritional support, cardiovascular optimization, transplant listing — that can significantly improve clinical outcomes [2, 6].

1.2 Host Organization

1.2.1 Sidi Mohamed Ben Abdellah University

Founded in 1975, Sidi Mohamed Ben Abdellah University (USMBA) is one of Morocco's largest public universities, headquartered in Fès. With over 100,000 students across 13 faculties and schools, USMBA covers a broad spectrum of disciplines from sciences and technology to humanities and medicine. The university has signed cooperation agreements with more than 200 international institutions and is actively engaged in research partnerships focused on health, sustainable development, and digital innovation.

1.2.2 Faculty of Medicine, Pharmacy and Dentistry (FMPDF)

The Faculty of Medicine, Pharmacy and Dentistry of Fès (FMPDF-Fès) is the academic arm responsible for training physicians, pharmacists, dentists, and health informaticians in the region. In partnership with CHU Hassan II, the faculty provides clinical training and conducts translational research bridging laboratory science and bedside medicine. Its research units work on topics ranging from infectious diseases and oncology to health systems and biomedical informatics.

Figure 1.1 – Logo of the Faculty of Medicine, Pharmacy and Dentistry of Fès (FMPDF)

1.2.3 Health Artificial Intelligence & Modeling Center (HAIM)

The **Health Artificial Intelligence & Modeling Center (HAIM)** is a research and innovation unit operating within FMPDF under the direction of **Mr. Zakaria Alami Merrouni**. The HAIM Center focuses on the application of artificial intelligence, data science, and computational modeling to challenges in healthcare, clinical research, and medical education. Its work spans from predictive modeling in clinical settings to the development of AI-powered tools, making it a natural incubator for a project like AI NéphroCare.

HAIM's core activities include:

— Building predictive models for high-mortality conditions (chronic kidney disease, oncology, sepsis)

— Designing health data platforms that comply with clinical and ethical standards

— Training the next generation of health informaticians and clinical data scientists

— Providing decision support tools deployable in resource-limited clinical environments

As the host institution for this internship, the HAIM Center provided the problem context, the domain expertise needed to design a medically relevant platform, access to real clinical workflows, and the validation environment for testing AI NéphroCare in conditions close to production use.

Figure 1.2 – Logo of the Health Artificial Intelligence & Modeling Center (HAIM)

This project was developed within HAIM, in direct collaboration with the Nephrology Department of CHU Hassan II of Fès, ensuring clinical relevance and real-world applicability.

1.3 Medical Context

1.3.1 Kidney Disease and Dialysis

▶ Anatomy and Function of the Kidney

The kidneys are paired retroperitoneal organs responsible for maintaining homeostasis through filtration of approximately 180 liters of blood per day [38]. Their principal functions include:

— Excretion of metabolic waste products (urea, creatinine, uric acid)

— Regulation of fluid and electrolyte balance (sodium, potassium, calcium, phosphate)

— Acid-base homeostasis (bicarbonate reabsorption)

— Endocrine functions: renin secretion (blood pressure), erythropoietin (red blood cell production), vitamin D activation [4]

▶ Chronic Kidney Disease (CKD)

CKD is defined as structural or functional kidney abnormalities persisting for more than three months, with implications for health [4]. The **KDIGO 2012 classification** stages CKD into five categories based on glomerular filtration rate (GFR) [4]:

Table 1.1 – CKD staging by KDIGO 2012

Stage	Description	GFR (mL/min/1.73m^2)	Prevalence
G1	Normal/high	≥ 90	∼3.5%
G2	Mildly decreased	60–89	∼3.9%
G3a	Mildly to moderately decreased	45–59	∼7.6%
G3b	Moderately to severely decreased	30–44	∼3.4%
G4	Severely decreased	15–29	∼0.4%
G5	Kidney failure	< 15	∼0.1%

At stage G5 (end-stage renal disease, ESRD), renal replacement therapy (RRT) becomes necessary. The three modalities of RRT are: hemodialysis (HD), peritoneal dialysis (PD), and kidney transplantation. Globally, over 2.6 million people are on RRT, with HD being the most prevalent modality (∼70% of RRT patients) [3, 35].

▶ Hemodialysis: Principle and Clinical Course

Hemodialysis works by passing the patient's blood through an artificial membrane (dialyzer) which removes waste products and excess water via diffusion and ultrafiltration. A typical HD session lasts 4 hours, 3 times per week.

The quality of HD is measured primarily by the **Kt/V** index, where K is the dialyzer clearance, t the session duration, and V the urea distribution volume. A Kt/V ≥ 1.2 per session is considered adequate per KDIGO guidelines [4].

▶ Epidemiology in Morocco

In Morocco, the MAGREDIAL national registry documented approximately 16,950 patients receiving renal replacement therapy, with an estimated 1,650–3,300 new ESRD cases per year [37]. The dominant etiologies of CKD in Morocco, per MAGREDIAL data, are [37]:

1. Diabetic nephropathy (25–43%)
2. Hypertensive nephrosclerosis (11–21%)
3. Undetermined (25–31%)
4. Glomerulonephritis (4–10%)

1.3.2 Mortality Risk Challenges

Despite advances in dialysis technology, the annual mortality rate of HD patients remains strikingly high — approximately **15–25%** in most national registries [4, 5, 40]. By comparison, this exceeds the 5-year mortality of many solid tumors [35].

The principal causes of death in HD patients are [5]:

— **Cardiovascular disease** (40–50%): sudden cardiac death, acute coronary syndrome, heart failure
— **Infections** (15–20%): vascular access-related sepsis, pneumonia
— **Stroke** (5–10%)
— **Withdrawal from dialysis** (10–15%, predominantly in elderly patients)

Key mortality predictors identified in the literature include: low serum albumin (< 35 g/L), high Charlson Comorbidity Index, high PTH, tunneled catheter use (vs. arteriovenous fistula), reduced residual diuresis, and low dialysis frequency [4, 5].

1.4 Problem Statement

1.4.1 Raw Hospital Data is Underutilized

At CHU Hassan II, the nephrology unit has accumulated several years of longitudinal patient records — biological results, dialysis session data, complication histories — primarily in paper records and Excel spreadsheets. This data is rich, but entirely siloed: it cannot be queried, aggregated, or used for risk prediction without significant manual effort. The absence of a unified digital platform means that potentially life-saving information lies dormant.

1.4.2 Lack of Predictive Tools for Mortality Risk

Clinicians currently rely on subjective assessment and population-level guidelines to estimate individual mortality risk. Validated scores such as the **Charlson Comorbidity Index** or the **Davies score** provide only coarse stratification and do not capture the full multivariate complexity of HD patient profiles. No locally validated, ML-based risk prediction tool was available to the CHU Hassan II nephrology team prior to this project.

1.4.3 Data Heterogeneity and Quality Issues

The HD-478 cohort dataset was collected over 4 years by multiple clinicians, resulting in considerable heterogeneity: inconsistent decimal separators (comma vs. period), variable biological units (mg/dL vs. mmol/L), free-text fields where categorical values are expected, and missing values in up to 30% of certain variables. Before any ML model could be trained, a robust data preprocessing pipeline was essential.

1.5 Project Objectives

1.5.1 General Objective

Design, implement, and validate a full-stack intelligent platform that enables the Nephrology Department of CHU Hassan II to (1) centralize and manage hemodialysis patient data, (2) automatically preprocess and validate imported datasets using AI, and (3) predict 1-year mortality risk for individual patients using a validated machine learning model integrated into a clinical decision support interface.

1.5.2 Specific Objectives

1. Build a structured patient data model capturing 160+ clinical variables across 11 medical sections (demographics, biology, dialysis quality, comorbidities, complications, outcomes).
2. Develop an LLM+RAG preprocessing pipeline that automatically detects and corrects data quality issues in imported Excel files.
3. Train and compare **nine** machine learning algorithms on the HD-478 cohort, selecting the best-performing model for production deployment.
4. Engineer 32 clinically meaningful features (24 raw clinical + 8 interaction features) from the raw dataset.
5. Select the optimal classification threshold by maximizing F1-score, and define three clinically actionable risk zones (Low/Moderate/High) with fixed boundaries grounded in KDIGO/DOPPS/TRIPOD guidelines.
6. Implement a secure, multi-role web application with full audit trail, role-based access control, and clinical visualization dashboards.
7. Deploy the entire system as a reproducible Docker Compose stack.

1.6 Methodology

1.6.1 Agile Approach

The project followed an **Agile Scrum** methodology adapted to an academic PFE context. The development was organized into bi-weekly sprints, each delivering a

testable increment of the platform. The Scrum workflow comprised:

— **Sprint Planning**: Definition of user stories and acceptance criteria
— **Daily Standups**: Brief progress check with supervisor
— **Sprint Review**: Demonstration of completed features to the nephrology team
— **Sprint Retrospective**: Identification of process improvements

1.6.2 Iterative Development

Five main phases were completed iteratively, starting from an essential preparatory phase before any development began:

1. **Phase 0 — Documentation & Skill Building** (Month 1):
   — *Bibliographic research*: systematic review of scientific articles on hemodialysis mortality prediction, clinical decision support systems, and ML in nephrology
   — *Medical domain acquisition*: understanding of nephrology concepts (CKD staging, dialysis modalities, mortality risk factors, Charlson index, Kt/V)
   — *Technical upskilling*: completion of Data Science training covering pandas, scikit-learn, Django REST Framework, and React
   — *Alignment*: mapping medical domain knowledge onto project technical objectives
2. **Phase 1** (Months 1–2): Data modeling, database design, backend skeleton, authentication
3. **Phase 2** (Months 2–3): Patient CRUD, import/export, LLM preprocessing pipeline
4. **Phase 3** (Months 3–4): ML model training, comparison, feature engineering, calibration
5. **Phase 4** (Months 4–5): Frontend interfaces, prediction UI, deployment, testing

1.6.3 Task Management with Trello

Tasks were tracked using a **Trello** board, a visual project management tool based on the Kanban methodology. The board was structured to reflect the full lifecycle of each task, from initial idea to validated delivery. It included the following columns:

— **Backlog**: all identified tasks and user stories pending prioritization
— **In Progress**: tasks currently under active development
— **Review**: completed tasks awaiting code review or supervisor feedback
— **Done**: fully validated and merged tasks

— **Meetings**: agenda items and decisions recorded from weekly meetings with supervisors

— **To Validate (Technical Supervisor)**: tasks requiring technical validation by Mr. Zakaria Alami before integration

— **Validated**: tasks approved after supervisor review

— **Project Tracking**: milestones, deadlines, and sprint-level progress summaries

Each card included a technical description, acceptance criteria, priority level, and estimated effort in story points. This multi-column organization provided full transparency to both the academic and technical supervisors, and facilitated continuous prioritization of clinical and technical requirements throughout the project.

Figure 1.3 – Trello board used for task management and project tracking

CHAPTER 2

State of the Art

Review of existing CDSS, ML models, and data engineering approaches.

CONTENTS

CHAPTER 2

State of the Art

2.1 Clinical Decision Support Systems

Clinical Decision Support Systems (CDSS) have evolved from simple rule-based alert systems to sophisticated AI-driven platforms [24]. The earliest CDSS — such as MYCIN (1970s) [25] and INTERNIST-1 [26] — relied on manually coded expert rules. Modern CDSS leverage machine learning to discover patterns in large clinical datasets [24].

A landmark meta-analysis by Bright et al. (2012) reviewed 148 RCTs of CDSS and found that computerized clinical decision support improved clinical process measures in 68% of trials and patient outcome measures in 57% [2]. However, challenges persist: alert fatigue, poor EHR integration, and lack of local validation limit real-world adoption [2].

In nephrology specifically, CDSS have been applied to: (1) drug dosing adjustment for renal function, (2) dialysis adequacy monitoring, (3) anemia management [46], and (4) mortality risk stratification [6]. The latter remains the most challenging due to the complexity of HD patient comorbidity profiles.

2.2 Healthcare Data Engineering

The extraction, transformation, and loading (ETL) of healthcare data presents unique challenges:

— **Structural heterogeneity**: Data may be structured (lab values, vital signs), semi-structured (clinical notes), or unstructured (radiology reports).

— **Missing values**: Clinical datasets routinely exhibit 20–50% missingness, driven by test ordering variability and documentation practices.

— **Temporal dynamics**: Patient state evolves over time; longitudinal aggregation is non-trivial.

— **Coding variability**: The same concept may be encoded differently across sites (ICD-10, SNOMED-CT, free text).

Recent advances in **Large Language Models (LLMs)** have opened new possibilities for clinical data cleaning. These models can parse free-text clinical records, standardize terminologies, and detect anomalous values — tasks previously requiring specialized NLP pipelines. The Qwen2.5 model [8] used in this project exemplifies this capability on structured tabular and clinical data. **Retrieval-Augmented Generation (RAG)** enhances LLMs by grounding their responses in a domain-specific knowledge base, reducing hallucinations and improving precision on structured extraction tasks [9].

2.3 Machine Learning in Mortality Prediction

A substantial body of literature has investigated ML approaches to mortality prediction in CKD and HD populations:

Table 2.1 – Selected ML studies predicting **1-year all-cause mortality** in hemodialysis patients. All studies share the same clinical endpoint, enabling direct AUC comparison.

Study	Cohort	Model	AUC	Outcome
Sheng et al. [30]	Chinese HD (n = 5,351)	XGBoost	0.83	1-year mortality
García-Montemayor et al. [31]	Spanish HD (n = 1,571)	RF	0.73	1-year mortality
This work	HD-478 (Fès, Morocco)	SVM Linear	**0.826**	1-year mortality

All three studies share the same clinical endpoint (1-year all-cause mortality), enabling direct AUC comparison. Several trends emerge: (1) ensemble methods (XGBoost, RF) achieve AUC 0.73–0.83 on large, well-resourced cohorts ($n > 500$); (2) linear models like SVM offer comparable discrimination with stronger interpretability, which is clinically preferred; (3) this work achieves AUC **0.826** on a Moroccan cohort of 478 patients, very close to the best reported XGBoost results obtained on cohorts ten times larger.

The marginal gap (0.826 vs. 0.83) is explained by two structural factors. First, **cohort size**: with $n = 478$, estimation of the decision boundary carries more variance than with $n = 5{,}351$ (Sheng et al.) or $n = 1{,}571$ (García-Montemayor et al.); larger samples provide a statistical advantage independent of model choice. Second, **variable availability**: large Asian and European registries routinely collect extended laboratory

panels — intact PTH series, ferritin trajectories, longitudinal Kt/V, full medication histories — that are not systematically documented in the Moroccan clinical records used here, limiting the feature set.

Despite these constraints, several factors explain why the SVM Linear remains competitive. (a) **Model–data fit**: SVM with a linear kernel is known to generalise robustly on small, high-dimensional clinical datasets, where ensemble methods risk overfitting to noise; the regularisation parameter C controls this trade-off explicitly. (b) **Data quality**: the RAG-LLM preprocessing pipeline substantially reduced transcription errors and missing values in the HD-478 dataset, improving signal quality beyond what raw record counts suggest. (c) **Domain alignment**: the model was trained exclusively on a locally representative Moroccan cohort, eliminating the domain shift that typically degrades performance when models trained on foreign populations are applied to North African patients with different comorbidity profiles and clinical trajectories.

2.4 Feature Engineering in Medical Data

Feature engineering — the process of creating informative input representations from raw clinical data — is often the single most impactful step in a medical ML pipeline. In the HD mortality prediction literature, the most consistently important features are:

— **Serum albumin**: Low serum albumin (< 3.5 g/dL) is among the strongest independent predictors of all-cause mortality in HD patients, reflecting the cumulative effects of malnutrition and chronic inflammation [7].

— **Charlson Comorbidity Index (CCI)**: Validated in dialysis populations [32]; higher comorbidity burden is independently associated with reduced survival, with a strong dose-response relationship observed across CCI categories [32].

— **Vascular access type**: Arteriovenous fistula (AVF) use is a strong protective factor; tunneled catheters are associated with approximately 2× higher infection-related mortality risk [33].

— **Residual renal function**: The presence of residual renal function, even at low levels, is independently associated with significantly lower all-cause mortality in HD patients [34].

— **Interaction features**: Non-linear combinations (e.g., age × CCI, albumin × hospitalizations) capture patient fragility beyond what individual variables convey [7, 30].

2.5 Existing Dialysis Prediction Models

Several commercial and academic tools exist for HD risk prediction:

— **USRDS** (USA) [35]: Large national ESRD registry providing epidemiological benchmarks for HD incidence and mortality; registry data have been used to develop risk stratification tools, but the annual data report itself is not a deployable predictive model validated outside the US.

— **Couchoud/REIN Score** (France) [27]: Clinical score derived from the French REIN registry predicting 6-month prognosis in elderly patients initiating dialysis; not validated in non-elderly or non-European populations.

— **Khan et al. (1994)** [36]: Observational cohort study demonstrating that the number of coexisting diseases significantly predicts survival on renal replacement therapy; establishes comorbidity as a prognostic factor but does not provide a validated individual-level risk scoring tool.

— **DOPPS Risk Model** [40]: Multinational observational analysis (DOPPS) identifying key mortality predictors in HD populations; descriptive rather than a deployable predictive tool.

None of these tools are validated in Moroccan or North African HD populations, nor do they integrate with a clinical data management platform.

Although the epidemiological literature on hemodialysis in Morocco has documented rising ESRD prevalence and key risk factors [37], the majority of these works remain descriptive and retrospective. To date, few studies have applied advanced machine learning techniques for individualized mortality risk prediction, and no validated clinical decision support system has been widely deployed in Moroccan hospital centers. This gap highlights the need for a locally trained, end-to-end platform such as AI NéphroCare.

2.6 Critical Analysis

2.6.1 Limitations of Existing Systems

1. **External validity**: Most published models are trained on large Western or East-Asian registries — e.g., Sheng et al. [30] ($n = 5{,}351$, China) or García-Montemayor et al. [31] ($n = 1{,}571$, Spain) — with demographic and etiological profiles significantly different from Moroccan patients (higher proportion of young diabetic patients, lower transplantation rates, different socioeconomic determinants).
2. **Deployment gap**: Published models rarely translate into operational clinical tools. The gap between a trained model and a deployed clinical application

involves authentication, data management, UI design, and regulatory compliance — aspects largely absent from academic ML papers.

3. **Data quality**: Most studies assume clean, complete datasets. In practice, clinical data requires extensive preprocessing before ML models can be applied.
4. **Interpretability**: Black-box models (deep neural networks) may achieve high AUC but cannot explain their predictions to clinicians, limiting trust and adoption.

2.6.2 Positioning of the Proposed Solution

AI NéphroCare addresses these limitations by:

— Training on a **locally collected cohort** from the same clinical environment where the tool will be deployed

— Providing a **complete clinical platform** — not just a model — integrating data management, preprocessing, prediction, and audit

— Selecting a **linear SVM** for its interpretability and robustness on imbalanced datasets with high-dimensional feature spaces

— Implementing **probability calibration** to ensure predicted scores correspond to actual observed mortality rates

— Providing **clinical reasoning** alongside each prediction (top-5 contributing factors, risk zone, tailored recommendation)

CHAPTER 3

Global System Architecture

Platform overview, layered design, and technology stack.

CONTENTS

CHAPTER 3

Global System Architecture

3.1 Overview of the Platform

AI NéphroCare is a full-stack web application organized around three core modules:

1. **Patient Data Management**: Structured storage, CRUD operations, import/export of clinical data for hemodialysis patients.
2. **AI Preprocessing Pipeline**: LLM-assisted data cleaning with RAG-based context retrieval, cell-level corrections, and supervised validation.
3. **ML Prediction Engine**: 32-feature SVM mortality model with calibrated probabilities and three-zone risk stratification.

These modules are exposed through a Django REST API consumed by a React single-page application, with all services containerized via Docker Compose.

Use Case Diagram

Figure 3.1 presents the use case diagram of the platform. Four actors interact with the system according to their role. All authenticated users can import Excel files and trigger the LLM preprocessing pipeline; however, the final validation and integration of data into the platform is restricted to Super Admin and Head of Department.

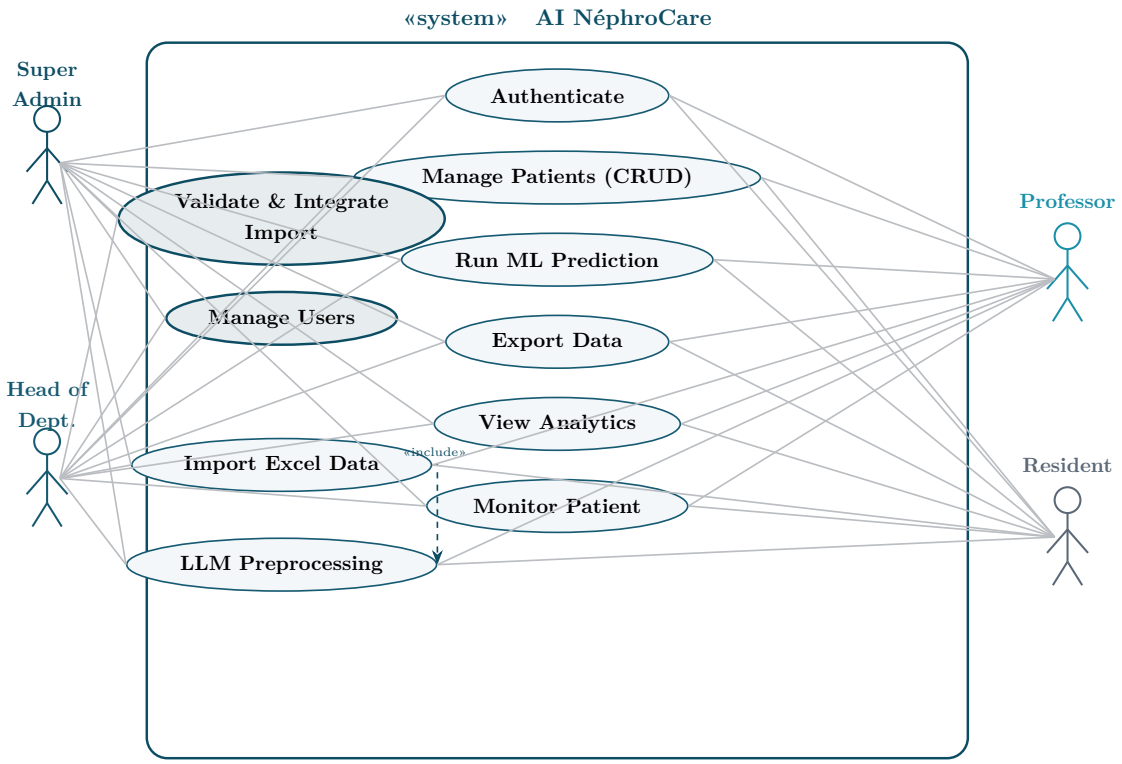


Figure 3.1 – Use Case Diagram — AI NéphroCare (4 actors, 10 use cases). Professor and Resident have identical access. Validate & Integrate Import is restricted to Super Admin and Head of Department.

3.2 Architectural Style

3.2.1 Full-Stack Architecture

The platform follows a classical three-tier architecture:

— **Presentation tier**: React 18 SPA running in the browser (served via Node/serve on port 3000)

— **Application tier**: Django REST Framework API (port 8000) containing all business logic, ML inference, and preprocessing

— **Data tier**: PostgreSQL 16 (port 5432) for persistent storage; Redis 7 (port 6379) for Celery task queue and session cache

3.2.2 Integrated Data Pipeline

The data pipeline is fully embedded within the Django backend. The platform supports **two distinct import paths**, both subject to the same approval workflow for non-admin users:

Path 1 — LLM-assisted preprocessing import (Preprocessing tab): The user uploads an Excel/CSV file, which is parsed by pandas and analyzed asynchronously by the Qwen2.5:14B LLM (via Celery/Redis). The model detects anomalies and proposes cell-level corrections with medical justification. The clinician reviews and optionally overrides corrections, then selects either the corrected version or the original version for integration. Before persisting, each column value is mapped to its corresponding field in the patient model via a configurable dictionary (`column_mapping.json`).

Path 2 — Direct import (Data Management tab): The user uploads an Excel/CSV file directly from the patient management interface, bypassing the LLM preprocessing step. The file is immediately parsed and each column is mapped to its target field in the patient schema. This path is designed for clean, already-validated datasets where LLM analysis is not needed. Both paths go through the same approval mechanism: Super Admin and Head of Department can integrate directly, while Professor and Resident submissions are queued as `pending` and require explicit approval before data reaches the database.

In both cases, after integration the data undergoes the same **import-time normalization** (decimal separator conversion, sex encoding unification, boolean standardization — see Section 4.2.3), ensuring data consistency regardless of the import path chosen.

The complete technical implementation of both pipelines is detailed in Chapter 4.

3.3 Layered Architecture

AI NéphroCare follows a five-layer architecture where each layer has a clear responsibility. Figure 3.2 presents this architecture with the key technologies at each level. Solid arrows represent the main synchronous data flow (top-down request/response). The two dashed pink arrows represent secondary, asynchronous data paths: the left one shows that the API layer (L2) directly triggers the Data Engineering layer (L4) for long-running preprocessing tasks via Celery, bypassing the ML layer; the right one shows that the ML layer (L3) reads serialized model files directly from the Data layer (L5) at inference time.

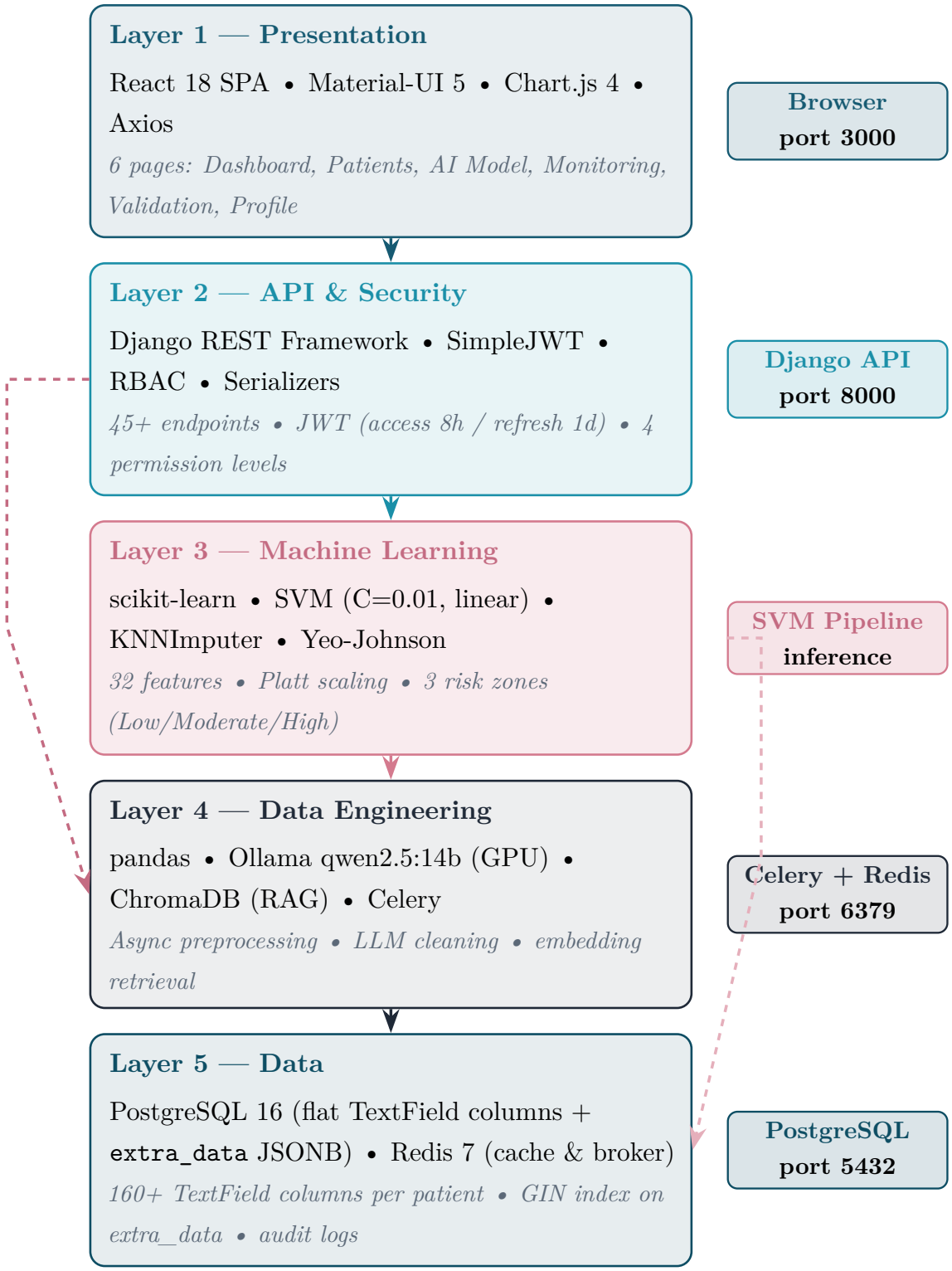


Figure 3.2 – Five-layer architecture of AI NéphroCare with technologies per layer

Table 3.1 details the specific responsibility of each layer and its communication interfaces. L1 communicates exclusively with L2 via HTTP REST calls; L2 orchestrates both L3 (for synchronous ML predictions) and L4 (for asynchronous preprocessing jobs); L3 and L4 both persist and retrieve data from L5, which serves as the single source of truth for all clinical records and model artifacts.

Table 3.1 – Layer responsibilities and inter-layer communication

Layer	Name	Responsibility	Communicates with
L1	Presentation	Clinical UI, charts, forms	L2 via REST/JSON (Axios)
L2	API & Security	Routing, auth, serialization	L1 (HTTP), L3, L4 (Python)
L3	Machine Learning	Feature extraction, SVM, calibration	L2 (results), L5 (models)
L4	Data Engineering	LLM preprocessing, async tasks	L2 (triggers), L5 (storage)
L5	Data	Persistent storage, caching	L3, L4 (SQL/Redis)

3.4 Technology Stack

Table 3.2 – Complete technology stack

Category	Technology	Version	Role
Backend	Python	3.11	Primary language
	Django	4.2	Web framework, ORM
	Django REST Framework	3.15	REST API, serialization
	SimpleJWT	5.3	JWT authentication
	scikit-learn	1.4	SVM, imputation, calibration
	pandas / numpy	2.2 / 1.26	Data manipulation
	Celery	5.3	Async task queue
	ChromaDB	latest	Vector database (RAG)
Frontend	React	18	UI framework
	Material-UI	5.15	Component library
	Chart.js	4	Clinical charts
	axios	latest	HTTP client
	react-router-dom	6	Client-side routing
Infrastructure	Docker / Compose	24+ / 2+	Containerization
	PostgreSQL	16	Relational database
	Redis	7	Cache & broker
	Ollama	latest	Local LLM server
	Nginx/serve	latest	Static file serving

3.4.1 Backend: Django REST Framework

Django was chosen for its mature ORM, built-in admin interface, and the DRF extension which dramatically accelerates REST API development. The `JSONB` support in Django's PostgreSQL backend allows flexible storage of heterogeneous clinical data without sacrificing query capability.

3.4.2 Frontend: React

React's component-based architecture maps naturally to the modular clinical interface requirements. The Context API provides lightweight state management for authentication (`AuthContext`) and language preferences (`LanguageContext`) without the overhead of Redux. The `LanguageContext` implements a full bilingual system (Français / English): all UI strings are looked up from a static translation table keyed by language code (`fr`/`en`), and the active language is persisted to `localStorage` so the user's preference survives page reloads.

3.4.3 Database: PostgreSQL

PostgreSQL's `JSONB` type — which stores JSON data in a binary format and supports indexing — was critical for storing the 11 clinical sections of each patient record without a rigid schema that would break with each new clinical variable. GIN indexes on JSONB columns maintain query performance.

3.4.4 Containerization: Docker

All seven services (frontend, backend, database, Redis, Ollama, Celery worker, audit cleanup) are defined in a single `docker-compose.yml` file, enabling one-command deployment on any Docker-capable host. This is essential for a hospital environment where IT staff may not have deep DevOps expertise.

CHAPTER 4

Data Engineering Pipeline

From raw clinical records to ML-ready features: curation, storage, and preprocessing.

CONTENTS

CHAPTER 4

Data Engineering Pipeline

4.1 Data Sources

4.1.1 Hospital Patient Records

The primary data source is the **HD-478 cohort**: a retrospective dataset of 478 hemodialysis patients treated at CHU Hassan II of Fès between 2020 and 2024. The cohort was assembled by nephrologists from paper records, biological reports, and the hospital information system.

Key characteristics:

— 478 patients (383 alive, 95 deceased at 1 year — 19.9% mortality)

— Follow-up period: from dialysis initiation to 1-year outcome or censoring

— Variables collected: **91** across demographics, biology, dialysis modality, comorbidities, complications, and outcome (of which 3 were 100% empty and excluded; the platform patient model supports 160+ fields total)

4.1.2 Excel / CSV Imports

The platform is designed to accept Excel (`.xlsx`) and CSV files exported from hospital systems. Column names and formats vary between sources, necessitating a flexible ingestion pipeline.

4.1.3 Pre-Computed Clinical Attributes

The HD-478 dataset includes several clinical variables that were recorded directly in the patient's medical dossier (*dossier médical*) by the clinical team and retrieved as-is — they were not computed by the platform:

— **Age**: The patient's age at dialysis initiation, as documented in the clinical record.

— **Kt/V**: A dialysis adequacy index recorded in the dialysis session reports.

— **Charlson Comorbidity Index (CCI)** [15]: A standardized comorbidity score documented in the patient file. The CCI assigns weighted scores (weights $w_i \in \{1, 2, 3, 6\}$) to 19 comorbidity categories present in the patient record, making it a rich summary of disease burden retrieved directly from the clinical dossier.

4.2 Data Ingestion

4.2.1 File Upload Endpoints

Files are uploaded via `POST /api/patients/import/` (multipart/form-data). The backend:

1. Validates the file extension and MIME type (Multipurpose Internet Mail Extensions)
2. Reads the file into memory using pandas and openpyxl
3. Computes a column profile (names, types, null rates)
4. Creates a preprocessing session (`session_id`) and returns it to the frontend

4.2.2 Parsing with Pandas

This subsection describes how the platform uses the pandas library to read the uploaded file into a DataFrame, normalizing column names by stripping whitespace and converting to lowercase. This makes the data structure uniform regardless of source file formatting.

Once parsed, the platform immediately computes a **technical profile** of the uploaded DataFrame — a structured summary of column statistics (names, types, missing rates, unique counts, sample values) used to drive the LLM routing decision (Section 4.6).

4.2.3 Import-Time Normalization

After the file is parsed, the platform applies lightweight normalization before any further processing:

— **Decimal separators**: European commas (e.g., `3,5`) are replaced by decimal points

— **Sex encoding**: Values `M`, `m`, `Masculin`, `H`, `1` → unified encoding; similarly for female

— **Boolean fields**: Yes/No, Oui/Non, 0/1 mapped to a consistent binary format
— **Structural exclusions**: Columns with > 90% missingness are flagged and excluded from ML features (3 identified: `transplantation_renale`, `distance`, `acces`)
— **Date parsing**: Multi-format parser tries standard date formats first, then falls back to Excel serial number conversion (days since 1899-12-30)

4.3 Manual Data Curation of the HD-478 Dataset

Before any machine learning preprocessing could be applied, the raw dataset underwent an extensive manual curation phase. The resulting corrected dataset serves as the sole input to the ML pipeline. This section documents all nine categories of corrections applied.

4.3.1 Overview of Modifications

A line-by-line and column-by-column comparison of the two files identified exactly **9 categories of corrections**. Table 4.1 summarizes the global state of the dataset before and after curation.

Table 4.1 – HD-478 dataset: original vs. corrected dataset

Indicator	Original	Corrected
Number of patients	478	478
Number of columns	91	91
Data types	Mixed (text, int, serial dates)	Normalized (float64, datetime64)
NaN → 0 (transplant)	∼300 NaN per column	Imputed logically (0)
DFGe MDRD	105 zeros + outliers (9, 11, 12)	Recalculated (109 values)
Biological outliers	11 cells	Corrected
X/x markers	>275 cells (10 columns)	Replaced by NaN
Text → float	>130 cells	Converted to float64
Serial Excel dates	480 integer dates	Converted to datetime64

4.3.2 Column Restructuring

Five columns were relocated within the corrected dataset to group derived and administrative features together: `couverture_medicale`, `annee_inclusion`, `diabete`, `dyspnee`, `oedemes_surcharge`. This grouping ensures a logical separation between clinical measurement variables and contextual patient attributes, which facilitates downstream feature selection.

4.3.3 Date Conversion from Excel Serial Numbers

Both date columns (`date_debut_dialyse` and `date_deces`) were stored as Excel serial integers (days since 1900-01-01), making them unreadable and unusable for survival analysis. All **480 dates** were converted to `datetime64` objects.

Table 4.2 – Date conversion examples

Variable	Original	Corrected	N corrected
`date_debut_dialyse`	45261 (integer)	2023-12-01	478 (all)
`date_deces`	44214 (integer)	2021-01-18	2

Note

Critical error detected — Line 31: `date_debut_dialyse` contained the Excel serial `410381`, corresponding to year **3023-08-01** (typo: 3023 instead of 2023). Corrected to 2023-08-01.

4.3.4 Correction of Biological Outliers

Eleven cells exhibited biologically impossible values due to unit errors or data entry mistakes. The following clinical reference ranges were used to identify and correct these outliers:

— Bicarbonates: 22–29 mmol/L [4]

— Calcium: 80–105 mg/L [45]

— Phosphate: 30–120 mg/L

Table 4.3 – Biological outliers corrected in the corrected dataset

Variable	Row	Original	Corrected	Error type
`bicarbonates_basaux`	160	50.0 mmol/L	15.0	Unit error (×3)
`bicarbonates_basaux`	219	42.0	14.0	Unit error (×3)
`bicarbonates_basaux`	248	56.0	16.0	Unit error (×3)
`calcium_basale`	91	5.8 mg/dL	58.0	Missing ×10 factor
`calcium_basale`	99	8.0 mg/dL	80.0	Missing ×10 factor
`phosphore_basale`	22	10.0 mg/L	100.0	Missing ×10 factor
`phosphore_basale`	39	12.0	120.0	Missing ×10 factor
`phosphore_basale`	48	10.0	100.0	Missing ×10 factor
`phosphore_basale`	296	8.0	80.0	Missing ×10 factor
`phosphore_basale`	335	252.0	52.0	Data entry error: extra digit (incompatible with life)
`creatinine_basale`	335	30 mg/L	300	Missing zero: 30 mg/L incompatible with ESRD (DFGe = 1.1)

4.3.5 DFGe MDRD Recalculation (109 values)

The `dfge_mdrd` column contained 105 values coded as 0 (impossible for a living patient) and several biological outliers (values of 9, 11, 12). All 109 erroneous values were recalculated using the **simplified MDRD-4 formula** (IDMS-standardized, factor 175), recommended by nephrology learned societies:

$$\text{DFGe [mL/min/1.73m}^2\text{]} = 175 \times S_{cr}^{-1.154} \times \text{Age}^{-0.203} \times \begin{cases} 0.742 & \text{if female} \\ 1.000 & \text{if male} \end{cases} \tag{4.1}$$

where S_{cr} is serum creatinine in mg/dL (converted from mg/L by dividing by 10), and Age is in years. The constant 175 replaces the older 186 in the IDMS-standardized version [13]. The ethnic correction factor (1.212 for African-American patients) was not applied as this information was unavailable in the dataset.

Table 4.4 shows a representative sample of the 109 corrections:

Table 4.4 – Sample DFGe MDRD corrections (109 total)

Excel row	Original DFGe	Corrected DFGe	Status
4	0	5.0	Impossible value (0) → recalculated
7	0	2.8	Impossible value (0) → recalculated
15	0	16.1	Impossible value (0) → recalculated
342	12	5.8	Incoherent value → recalculated
356	9	3.4	Incoherent value → recalculated
369	11	2.8	Incoherent value → recalculated
… + 103 other rows (total 109)			

4.3.6 Text-to-Numeric Conversions

Several columns stored numeric values as text strings or contained heterogeneous formats (text numbers, x markers, datetime objects). Each subcategory below documents the corrections applied column by column.

▶ Proteinuria (`proteinurie_basale`) — 127 corrections

This column stored numeric values as character strings (e.g., `'5.17'` instead of 5.17), as well as `'x'`/`'X'` markers and one mistyped integer (412 → 4.12 after dividing by 100). All 127 values were converted to `float64` or `NaN`. Correct numeric encoding is essential for this variable as proteinuria level is a clinically significant predictor of CKD progression.

▶ Vitamin D (`vitamine_d_basale`) — 6 corrections

Mix of text values, marker `'x'`, and invalid zero: rows 98, 128, 206, 392 (text → float64); row 261 (`'x'` → `NaN`); row 316 (0 → `NaN`). Vitamin D deficiency is prevalent in dialysis patients, making its numeric encoding critical for downstream ML feature computation.

▶ Pre-dialysis follow-up (`suivi_predialyse_mois`) — 5 corrections

Heterogeneous formats: row 9 (0 → 72, recovered from records); row 278 (`'1mois'` → 1.0); rows 327 and 421 (`datetime.time(0,0)` → 0.0); row 420 (erroneous Excel

1900 date → 72.0). The duration of pre-dialysis nephrology follow-up is an important contextual variable that must be numeric for survival analysis.

4.3.7 Cleaning of 'X'/'x' Markers in Education Columns

In therapeutic education columns, cells were filled with `'X'` or `'x'` instead of binary 0/1 values. Since the exact semantic meaning of these markers (whether they indicate presence of the condition or a non-applicable field) could not be determined without reviewing individual patient records, all **265 markers** were conservatively replaced by `NaN` to avoid introducing spurious binary signals into the ML pipeline.

Table 4.5 – X/x marker cleanup by column (265 total)

Column	N markers	Action
`comprehension_maladie`	39	→ NaN
`connaissance_therapie_substitution`	40	→ NaN
`connaissance_pratique_dialyse`	40	→ NaN
`education_soins_acces`	40	→ NaN
`surveillance_hydrique_ponderale`	40	→ NaN
`education_regime`	40	→ NaN
`education_traitements_associes`	11	→ NaN
`education_complications`	12	→ NaN
`ferritine_basale`	2	→ NaN
`seances_par_semaine`	1	→ NaN
Total	**265**	All → NaN

4.3.8 Logical NaN Imputation for Transplant Binary Variables

For four transplant-related binary variables, missing values (NaN) were interpreted clinically as *"not performed"*: absence of documentation equals absence of the clinical act. Consequently, **1,261 NaN values** were replaced by 0.

In the clinical context of hemodialysis patient records, the absence of documentation for transplant-related procedures is itself clinically informative. When no pre-transplantation workup, blood transfusion, transplant information session, or waiting list entry was recorded, this absence reflects the clinical reality that these acts were never initiated — not that the data was simply not collected. This zero-imputation is therefore not a statistical guess but a clinically grounded decision: *non-documentation equals non-event* in this administrative context. This approach is consistent with standard practices in retrospective clinical data curation.

Table 4.6 – NaN → 0 imputation for transplant variables

Variable	N	Clinical justification
`bilan_pretransplantation`	349	No documentation = workup not performed
`immunisation_transfusion_sanguine`	307	No documentation = no transfusion on record
`information_transplantation_donnee`	304	No documentation = information not provided
`liste_attente_transplantation`	301	No documentation = not on waiting list
Total	**1,261**	

Special case: Row 63 in `liste_attente_transplantation` contained the string `'1;(1/2024)'` — cleaned to 1 (patient registered in January 2024).

4.3.9 Logical Recodings and Modality Corrections

Beyond numeric corrections, the dataset contained 14 categories of logical inconsistencies — cases where documented values contradicted other clinical information in the same record. Each recoding category below describes the nature of the error, how it was identified, and the correction applied.

— **15 asymptomatic patients** recoded from 1 to 0: patients coded as asymptomatic but presenting documented symptoms

— **Hemodialysis variable** (4 rows): incorrectly coded 0 → 1

— **Vascular access corrections**: 5 rows with `cathetere_femoral` NaN → 1 (confirmed in records); 2 rows 1 → 0

— **Binary recoding**: values > 1 (infection=2, hemorrhage=2, access dysfunction=3) → 1

— **Cause of death**: row 396, `'OUI'` → 1.0

Each of these corrections was verified against the original paper-based clinical records to ensure that no information was introduced or fabricated — all changes are conservative recodes toward clinically consistent representations.

4.3.10 Final Dataset Structure

After all corrections, the corrected dataset presents the following variable type distribution:

Table 4.7 – Variable type distribution in the corrected dataset (ML-ready)

Category	N variables	%	Examples
Binary variables (0/1)	57	62.6%	sex, death, hypertension, HD, infection
Continuous (float/int)	21	23.1%	age, Charlson, DFGe, albumin, creatinine
Ordinal categorical	11	12.1%	access type, death cause, coverage
Date (datetime64)	2	2.2%	dialysis start, death date
100% empty (exclude)	3	3.3%	transplantation_renale, distance, acces
Total	**91**	100%	

4.3.11 Quantitative Summary of Manual Curation

Manual Curation — Complete Bilan

- 9 categories of corrections applied
- **480 dates** converted (Excel serial → datetime64)
- **109 DFGe values** recalculated via MDRD formula
- **1,261 NaN → 0** on 4 binary transplant variables
- **127 proteinuria values** converted (text → float + x cleanup)
- **265 X/x markers → NaN** across 10 columns
- **11 biological outliers** corrected (unit / entry error)
- **15 asymptomatic patients** re-coded to 0
- **14 vascular access / dialysis modality** corrections
- 3 100%-empty columns identified for ML exclusion

4.4 Data Storage & Schema Design

4.4.1 Entity-Relationship Diagram

The platform's persistence layer is designed around a PostgreSQL 16 relational database. Figure 4.1 presents the Entity-Relationship diagram showing the five main entities and their associations. Each entity box lists its key attributes; underlined attributes are primary keys; italicised attributes are foreign keys.

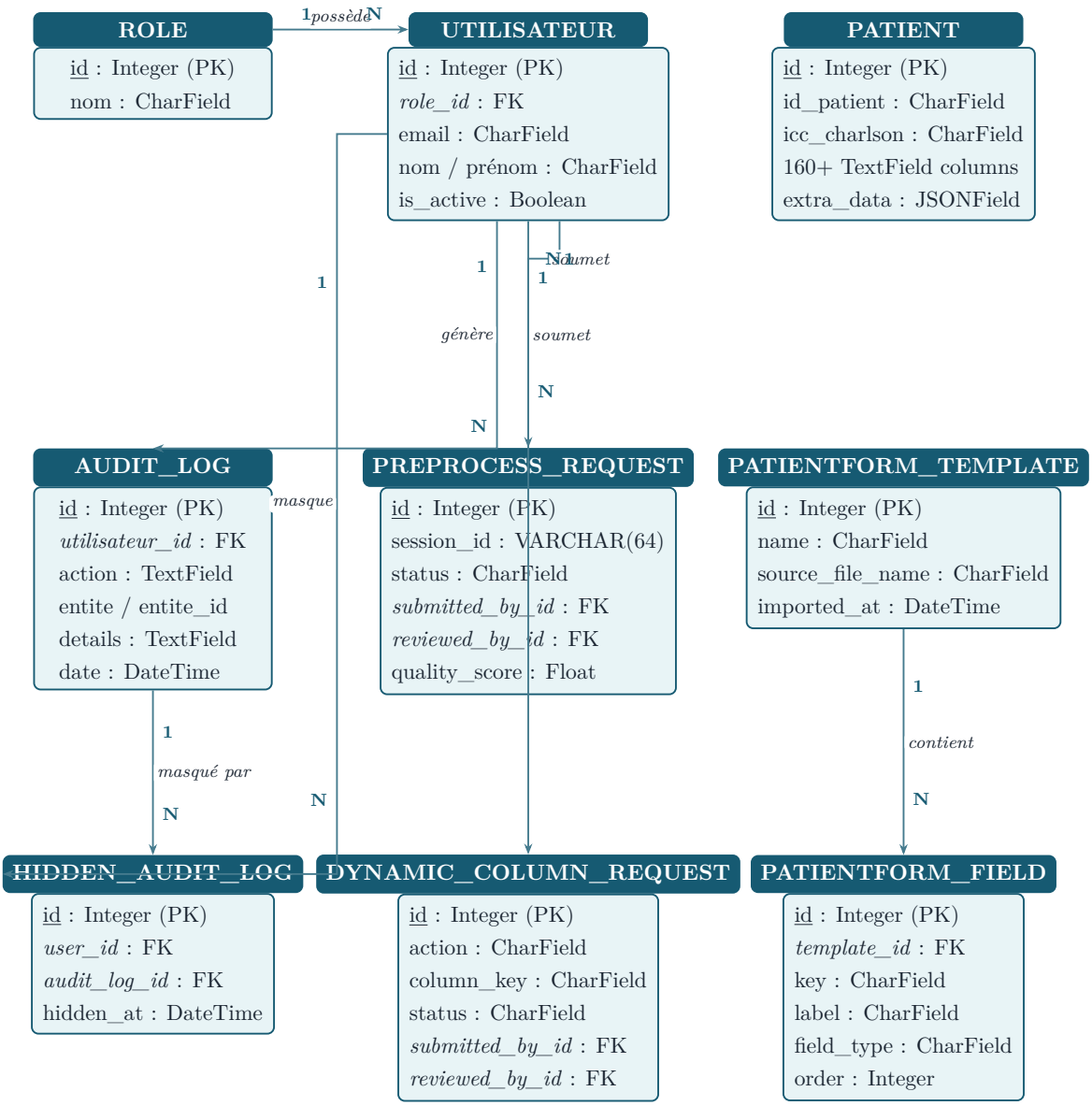


Figure 4.1 – Entity-Relationship diagram — AI NéphroCare complete database schema (PostgreSQL 16). 9 custom tables. Underlined = primary key; italicised = foreign key.

4.4.2 PostgreSQL Schema

The database `plateforme_medicale` (PostgreSQL 16) contains nine custom tables created by Django's migration system. These tables are divided into two categories:

— **Core tables** (4): the tables that form the fundamental data model of the platform — users, roles, patients, and audit trail. All clinical data, access control, and traceability pass through these tables.

— **Auxiliary tables** (5): tables that support specific platform features (LLM preprocessing workflow, dynamic column management, form templates, hidden audit entries). They extend the core model without altering it.

Table 4.8 provides a complete overview of all nine tables.

Table 4.8 – All tables in the `plateforme_medicale` database — core vs. auxiliary

Table (PostgreSQL)	Type	Group	Role
`users_role`	**Core**	Auth	Available user roles
`users_utilisateur`	**Core**	Auth	User accounts, hashed password, RBAC
`patients_patient`	**Core**	Clinical	Full patient record — 160+ fields
`audit_auditlog`	**Core**	Audit	All platform actions (tamper-evident)
`patients_preprocessvalidationrequest`	Auxiliary	Preprocessing	LLM session tracking, status, reviewer
`patients_patientformtemplate`	Auxiliary	Form engine	Column mapping templates
`patients_patientformfield`	Auxiliary	Form engine	Field definitions per template
`patients_dynamiccolumnrequest`	Auxiliary	Form engine	Add/remove dynamic column requests
`audit_hiddenauditlog`	Auxiliary	Audit UI	Per-user soft-hidden log entries

The detailed column schemas of the four core tables are described in the following subsections. The auxiliary tables follow the same design pattern (integer PK, FK references to core tables, status workflow fields) and are not detailed further here.

▶ `users_role` and `users_utilisateur`

The `users_role` table is a lightweight reference table holding the four platform roles (`super_admin`, `chef_service`, `professeur`, `résident`). The `users_utilisateur` table stores all user accounts with their hashed credentials, contact information, and a foreign key linking each account to its assigned role. Together they implement the platform's Role-Based Access Control.

Table 4.9 – Schema of `users_role` and `users_utilisateur`

Column	Type	Nullable	Description
Table: users_role			
`id`	INTEGER	NO	Primary key (auto)
`nom`	VARCHAR(50)	NO	Role identifier (unique): super_admin, chef_service, professeur, résident
Table: users_utilisateur			
`id`	INTEGER	NO	Primary key
`email`	VARCHAR(254)	NO	Login identifier (unique)
`nom`	VARCHAR(100)	NO	Last name
`prenom`	VARCHAR(100)	NO	First name
`telephone`	VARCHAR(20)	YES	Phone number
`password`	VARCHAR(128)	NO	PBKDF2+SHA256-hashed password (Django default)
`role_id`	INTEGER (FK)	YES	FK → `users_role.id`
`is_active`	BOOLEAN	NO	Account enabled/disabled
`is_staff`	BOOLEAN	NO	Django admin access
`date_creation`	TIMESTAMP	NO	Account creation timestamp
`personal_notes`	TEXT	NO	Private notes (visible only to owner)

▶ `patients_patient`

The patient table is the central table of the platform. It stores the complete clinical record of each hemodialysis patient as a flat structure of 160+ `TextField` columns, organized by clinical domain prefix (e.g., `demographie_*`, `biologie_*`, `dialyse_*`):

Table 4.10 – Key columns of `patients_patient` (representative subset)

Column	Type	Description
`id`	BIGINT	Primary key
`id_patient`	VARCHAR(120)	Clinical ID from source file
`icc_charlson`	VARCHAR(10)	Charlson Comorbidity Index
`statut_inclusion`	VARCHAR(80)	Inclusion status in the cohort
`utilisateur_saisie`	VARCHAR(120)	User who entered the record
`demographie_sexe`	TEXT	Sex (M/F)
`demographie_age_ans`	TEXT	Age at dialysis initiation
`demographie_couverture_sociale`	TEXT	Social coverage (RAMED, AMO, etc.)
`biologie_albumine_g_l`	TEXT	Serum albumin (g/L)
`biologie_creatinine_mg_l`	TEXT	Serum creatinine (mg/L)
`biologie_hemoglobine_g_dl`	TEXT	Hemoglobin (g/dL)
`dialyse_modalite_actuelle`	TEXT	Dialysis modality (HD, HDF, etc.)
`dialyse_date_debut`	TEXT	Dialysis start date
`dialyse_seances_par_semaine`	TEXT	Sessions per week
`comorbidite_liste`	TEXT	Comma-separated comorbidity list
`complication_liste`	TEXT	Documented complications
`outcome_deces_1an`	TEXT	1-year mortality outcome (0/1)
`extra_data`	JSONB	Dynamic columns not in fixed schema
...	TEXT	(160+ fields total across all clinical domains)

Design choice: all clinical fields are stored as `TextField` rather than typed columns. This allows the platform to accept heterogeneous input formats (numeric text, date strings, coded values) without data loss during import, while type conversion is handled at the preprocessing and feature extraction layers.

▶ `patients_preprocessvalidationrequest`

This table tracks every LLM preprocessing session and direct import request submitted on the platform. Each row captures the full lifecycle of a data integration request — from initial submission (with `pending` status) through review by an authorized user (Head of Department or Super Admin), who can approve or reject the request. The `quality_score` field stores the AI-computed data quality index returned by the LLM analysis, allowing reviewers to assess data reliability before approving integration.

Table 4.11 – Schema of `patients_preprocessvalidationrequest`

Column	Type	Description
`id`	INTEGER	Primary key (auto)
`session_id`	VARCHAR(64)	Preprocessing session identifier
`source`	VARCHAR(20)	Source type: corrected / original
`source_file_name`	VARCHAR(255)	Original uploaded filename
`status`	VARCHAR(20)	pending / approved / rejected
`submitted_by_id`	INTEGER (FK)	FK → `users_utilisateur.id`
`submitted_at`	TIMESTAMP	Submission timestamp
`reviewed_by_id`	INTEGER (FK)	FK → `users_utilisateur.id` (Head of Department)
`reviewed_at`	TIMESTAMP	Review timestamp
`rows_count`	INTEGER	Number of patient rows in the file
`quality_score`	FLOAT	LLM-computed data quality score
`comment`	TEXT	Reviewer comment

▶ audit_auditlog

The `audit_auditlog` table provides the platform's tamper-evident audit trail. Every state-changing operation — patient record creation, update, deletion, file import, and mortality prediction — automatically generates an entry via Django signals. Each entry records the actor's identity, the affected entity, a plain-text description of the change, the client IP address, and a precise timestamp. Log entries are automatically purged after 15 days by the dedicated `medical_audit_cleanup` Docker service, balancing storage cost against traceability requirements.

Table 4.12 – Schema of `audit_auditlog`

Column	Type	Description
`id`	INTEGER	Primary key
`utilisateur_id`	INTEGER (FK)	FK → `users_utilisateur.id`
`action`	TEXT	Action description (create, update, delete, import, predict)
`entite`	VARCHAR(100)	Affected entity type (Patient, Utilisateur, etc.)
`entite_id`	INTEGER	ID of the affected entity
`details`	TEXT	Plain text description of action details
`adresse_ip`	INET	Client IP address
`date`	TIMESTAMP	Action timestamp (auto_now_add)

Every create, update, delete, import, export, and prediction action generates an `AuditLog` entry automatically via Django signals. Logs are retained for 15 days by default, then purged by a dedicated Docker service (`medical_audit_cleanup`) that

runs the `cleanup_audit_logs` Django management command every 24 hours via a shell loop.

4.4.3 Flexible Schema with `extra_data`

An additional `extra_data JSONB` column stores dynamically added columns from imported files that do not map to any predefined clinical section. This ensures no data is discarded during import, while maintaining schema integrity for core clinical fields.

4.4.4 Data Structuring and Payload Mapping

Once normalized (Section 4.2.3), each patient record is mapped to the platform's fixed nested structure comprising 11 clinical sections. A configurable column mapping dictionary (`column_mapping.json`) translates each source column name to its target field in the Django model. The function `build_patient_payload(row, column_map)` constructs the structured payload, directing unmapped columns to `extra_data`.

The 11 clinical sections of the Patient model cover the complete nephrology care pathway:

Table 4.13 – Clinical sections of the Patient data model (Django ORM)

Section	Key variables	N fields
Demographics	Age, sex, residence, social coverage	14
CKD history	Etiology, biopsy, diagnosis context	9
Comorbidities	Diabetes, Charlson index, comorbidity list	3
Clinical presentation	BP, HR, weight, symptoms	13
Biology	Albumin, creatinine, PTH, ferritin, Na, Ca	23
Imaging	Renal echography, echocardiography, LVEF	10
Dialysis	Modality, access, frequency, Kt/V	18
Dialysis quality	URR, ultrafiltration, dry weight	10
Treatment	Drug list, notes	2
Complications	Events, hospitalizations, dates	8
Outcome	1-year mortality, cause, transfer	variable

4.5 Data Quality & Validation

4.5.1 Validation Rules

Data quality is enforced at two levels. At the **LLM preprocessing stage**, the Qwen2.5:14B model identifies biologically implausible values (e.g., albumin outside the physiological range, inconsistent units) and proposes corrections with medical justification. At the **import stage**, the platform stores all clinical values as `TextField` to avoid data loss from strict type constraints, delegating range-checking to the LLM preprocessing pipeline and the clinician review workflow. This design choice allows the system to accept heterogeneous input formats while surfacing quality issues through the AI preprocessing layer rather than through hard serializer rejection.

4.5.2 Audit Logs

Every create, update, delete, import, and export action generates an `AuditLog` entry with: actor (user ID), action type, affected entity, diff of changed fields, IP address, and timestamp. Logs are automatically purged after 15 days by the dedicated `audit_cleanup` Docker service, which runs `manage.py cleanup_audit_logs` every 24 hours.

The audit log system provides a complete, tamper-evident record of all data modifications, satisfying requirements for accountability in clinical AI systems. Each log entry captures sufficient context — actor identity, precise timestamp, IP address, changed fields — to reconstruct the full history of any patient record modification. Logs are retained for 15 days by default (configurable), balancing storage cost against regulatory traceability requirements.

4.6 Data Engineering Layer

The data engineering layer sits between raw data ingestion and ML inference. It automates data quality detection, semantic correction, and asynchronous validation through three core components: an LLM analysis engine, a RAG knowledge base, and an asynchronous task queue. Figure 4.2 presents the complete pipeline.

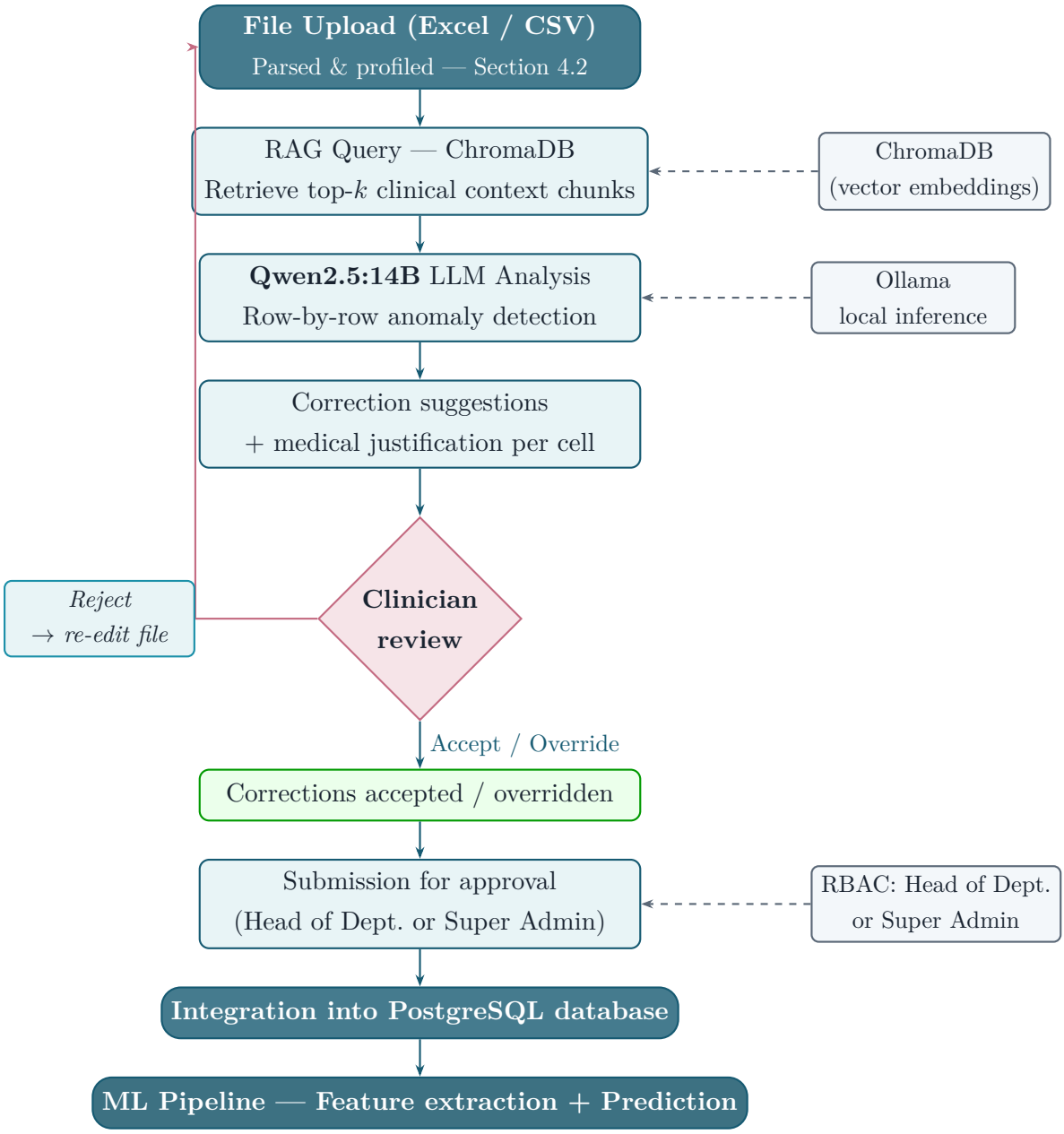


Figure 4.2 – LLM-based automated preprocessing pipeline (Section 4.6). After file upload and profiling (Section 4.2), the Qwen2.5:14B model retrieves clinical context from ChromaDB (RAG), analyzes each row, and proposes justified corrections. After clinician review and approval by the Head of Department or Super Admin, the cleaned data enters the ML pipeline.

4.6.1 LLM-Based Automated Cleaning (Qwen2.5:14B)

The platform deploys **Qwen2.5:14B** [8] locally via **Ollama** as the data quality analysis model. Qwen2.5:14B was chosen for: (1) its strong reasoning on structured tabular data and medical terminology; (2) on-premises deployment — patient data never leaves the hospital infrastructure; (3) its 14-billion-parameter capacity to contextualise clinical values, detect unit errors, and generate medically justified corrections.

Deployment configuration. The model runs on an **NVIDIA GeForce RTX**

3060 GPU (12 GB VRAM) via Docker Compose GPU reservation directives. At inference time the full 9 GB model fits in available VRAM, enabling 100% GPU inference with no CPU offloading. The Ollama context window is set to **16 384 tokens**, providing an effective budget of **14 184** input tokens per LLM call after reserving space for system prompt, schema, and output — sufficient to accommodate the largest observed prompts without truncation.

The preprocessing pipeline is implemented as an 11-stage **Celery asynchronous task** (`analyze_preprocess_async`). The key stages are:

1. **Technical profiling** — Python performs a full statistical scan of every column (data types, missing rates, outlier detection, sentinel values, Excel artefacts) without any LLM call, producing a `technical_profile` dict.
2. **Route determination** (`estimate_route`) — the platform evaluates dataset complexity (missing rate, outlier count, clinical column criticality) and selects the processing mode:
   - **Deterministic only**: clean file ($\leq$50 rows, 0% missing) — Python rules, no LLM call
   - **Balanced**: moderate complexity — Qwen2.5:14B with reduced context
   - **Advanced**: complex dataset (>300 rows or $\geq$10% missing) — full RAG context
3. **RAG retrieval** — semantic chunks are used to query ChromaDB and inject the most relevant nephrology guidelines into the LLM prompt.
4. **LLM batch analysis — Pass 1** — all columns are split into batches of **5 columns** (`LLM_BATCH_SIZE=5`); for each batch **all row values** are serialised and sent to Qwen2.5:14B. The model returns a structured `correction_plan` (type casts, date parsing, fill-missing, flagged anomalies). For a 90-column dataset this produces at most $\lceil 90/5 \rceil = 18$ LLM calls.
5. **Biological correction pass** (`_run_bio_value_correction_pass`) — each biological column is matched against the **LOINC database** (primary) or the medical knowledge base (fallback); the LLM identifies aberrant unique values and corrects them across all rows.
6. **LLM flagged corrections** (`_run_llm_flagged_corrections`) — columns flagged in Pass 1 but not covered by the bio KB are re-analysed; each suspicious value is classified as *error* (auto-corrected), *extreme_real* (kept, flagged), or *uncertain* (kept, flagged).
7. **KNN imputation** (`_run_model_imputation`) — values nullified by the correction passes are re-estimated via K-Nearest Neighbours ($k = 5$) using all available clinical features.

This multi-pass architecture ensures that both **structural anomalies** (type errors, formats) and **clinical anomalies** (biologically impossible values, unit errors) are addressed, while preserving extreme-but-plausible values that would be incorrectly removed by a purely statistical approach.

4.6.2 RAG Knowledge Base (ChromaDB)

A **Retrieval-Augmented Generation** (RAG) [9] system built on ChromaDB provides the LLM with contextual medical knowledge at inference time. The vector database stores embeddings of nephrology clinical guidelines, reference range documents, and dialysis protocol summaries. For each patient row analyzed, the top-k most relevant document chunks are retrieved and injected into the LLM prompt, significantly improving the accuracy and clinical grounding of corrections.

The RAG context is built by matching the column names of the uploaded file against the `medical_kb.json` knowledge base — a curated dictionary of clinical reference ranges for nephrology variables. For each matched column, the relevant reference range (normal low/high values, unit) is retrieved and injected into the LLM prompt as contextual grounding.

4.6.3 Celery Asynchronous Task Queue

LLM analysis of a full 478-row dataset takes 1–2 minutes with Qwen2.5:14B. To prevent HTTP timeouts, the analysis is dispatched asynchronously via **Celery** with a Redis message broker. The Django API returns a `session_id` immediately; the Celery worker processes the file in the background, updating session status in real time. The frontend polls `GET /api/patients/preprocess/{sid}/status/` every few seconds to display progress.

4.7 Feature Engineering

Feature engineering was performed in two steps: extraction of raw clinical features, and computation of interaction features.

4.7.1 Clinical Feature Extraction

The 24 raw clinical features are extracted from Patient ORM objects by the function `extract_features_for_patient()` in `feature_mapping.py`. This function imple-

ments a **multi-strategy extraction**: direct numeric fields, binary encoded fields, keyword-detected fields (e.g., dyspnea detected from symptom free-text), and ordinal categorical encoding.

4.7.2 Derived Risk Indicators

These interaction features were constructed based on clinical evidence from the nephrology literature regarding combined risk factors in hemodialysis mortality [4, 7, 32, 40, 41].

Eight interaction features are computed to capture non-linear risk combinations:

$$\text{age\_x\_charlson} = \text{age} \times \text{CCI} \tag{4.2}$$

$$\text{age\_x\_cardio} = \text{age} \times \mathbf{1}[\text{cardiopathy}] \tag{4.3}$$

$$\text{dyspnee\_x\_oedeme} = \mathbf{1}[\text{dyspnea}] \times \mathbf{1}[\text{edema}] \tag{4.4}$$

$$\text{charlson\_x\_hosp} = \text{CCI} \times N_{\text{hosp}} \tag{4.5}$$

$$\text{hypert\_x\_cardio} = \mathbf{1}[\text{HTA}] \times \mathbf{1}[\text{cardiopathy}] \tag{4.6}$$

$$\text{albumine\_x\_hosp} = \text{albumin} \times N_{\text{hosp}} \tag{4.7}$$

$$\text{albumine\_lt35} = \mathbf{1}[\text{albumin} < 35] \tag{4.8}$$

$$\text{sodium\_lt130} = \mathbf{1}[\text{sodium} < 130] \tag{4.9}$$

A null-safe multiplication helper ensures that any interaction involving a missing feature returns `None` rather than propagating errors, allowing the downstream KNN imputer to handle these cases at inference time.

4.8 Discussion

4.8.1 Why No External ETL

Embedding preprocessing in the application backend rather than using tools like Apache Airflow or Spark was a deliberate choice justified by dataset scale ($< 10{,}000$ patients), team expertise (Python/Django), deployment simplicity (single Docker Compose), and real-time preprocessing requirements (preprocessing results must be visible within seconds of file upload).

4.8.2 Impact on Model Performance

Beyond data quality, the LLM preprocessing pipeline has a measurable impact on the machine learning model's performance. The SVM trained on the **raw HD-478 data** (before preprocessing) achieves **AUC = 0.8235** in offline evaluation. After integrating the HD-478 cohort through the platform's LLM preprocessing and validation workflow, and retraining the SVM on the resulting **cleaned and validated database**, the deployed model achieves **AUC = 0.8262** — an improvement of +0.0027. This gain is directly attributable to the preprocessing pipeline: corrected unit errors, standardized values, and LLM-imputed missing fields produce a cleaner feature matrix that the SVM can exploit more effectively. This provides quantitative evidence that the LLM preprocessing pipeline adds value not only to data quality but also to predictive accuracy (see Section 5.9 for a detailed discussion).

4.8.3 Trade-offs of Embedding Pipeline in Backend

The main trade-offs are: (1) scalability — the current architecture would require refactoring for datasets exceeding ∼100,000 patients; (2) testability — mixing ETL logic with API logic increases test complexity; (3) reusability — the pipeline is not easily reusable outside the Django context.

CHAPTER 5

Machine Learning Layer

Model comparison, SVM selection, calibration, and risk zones.

CONTENTS

CHAPTER 5

Machine Learning Layer

5.1 ML Architecture

5.1.1 Integrated Within Backend

The entire ML pipeline — from feature extraction to risk zone classification — is implemented in pure Python within the Django backend, using scikit-learn 1.4 as the primary ML framework. Models are trained offline (on the HD-478 dataset) and serialized to `.joblib` files. At inference time, models are loaded from disk and applied to a single patient's features in under 100ms.

The ML layer comprises four components:

1. **Feature extractor** (`feature_mapping.py`): Produces a 32-dimensional feature vector from a Patient ORM object
2. **SVM pipeline** (`mortalite_svm.joblib`): KNN imputer + Yeo-Johnson transformer + linear SVC
3. **Platt-scaled probability** (`CalibratedClassifierCV`, 5-fold sigmoid): Maps the SVM margin score to a calibrated probability $\hat{p}$
4. **Risk zone classifier**: Faible ($\hat{p} < 10\%$), Modérée ($10\% \leq \hat{p} < 29\%$), Élevée ($\hat{p} \geq 29\%$) — seuils dérivés par ROC/Youden bootstrappé

5.2 Models Evaluated

Nine ML algorithms were trained and compared on the HD-478 dataset using identical preprocessing (Section 4.7) and evaluation protocol (stratified 10-fold cross-validation). The models span the main families of supervised classification algorithms.

5.2.1 Logistic Regression

Logistic Regression (LR) is the linear baseline classifier. For a binary outcome $y \in \{0, 1\}$, it models:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T\mathbf{x}+b)}} \tag{5.1}$$

Parameters $\mathbf{w}$ and b are estimated by maximizing the ℓ_2-regularized log-likelihood:

$$\mathcal{L}(\mathbf{w}) = \sum_{i=1}^{n} [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)] - \frac{1}{2C}\|\mathbf{w}\|^2 \tag{5.2}$$

Configuration

```
C=0.5, solver='saga', class_weight='balanced', max_iter=500
```

5.2.2 Support Vector Machine (Linear)

The SVM [17] seeks a hyperplane $\mathbf{w}^T\mathbf{x} + b = 0$ that maximizes the margin between the two classes. The soft-margin SVM solves:

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n} \xi_i \tag{5.3}$$

subject to:

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i \quad \forall i \tag{5.4}$$

$$\xi_i \geq 0 \quad \forall i \tag{5.5}$$

where ξ_i are slack variables allowing misclassifications, and C controls the regularization strength. A small $C = 0.01$ was chosen, enforcing strong regularization to prevent overfitting on the 478-patient dataset.

Probabilities are obtained using **Platt scaling** [10]: fitting a logistic regression $P(y = 1 \mid f) = \sigma(Af + B)$ on the signed margin distances $f_i = \mathbf{w}^T\mathbf{x}_i + b$ using cross-validated out-of-fold predictions.

Configuration

```
SVC(C=0.01, kernel='linear', class_weight='balanced', probability=True)
```

5.2.3 Random Forest

Random Forest (RF) [18] constructs an ensemble of B decorrelated decision trees, each trained on a bootstrap sample of the training data with random feature subsets at each split [18]:

$$\hat{f}_{\mathrm{RF}}(\mathbf{x}) = \frac{1}{B}\sum_{b=1}^{B} T_b(\mathbf{x};\Theta_b) \tag{5.6}$$

where T_b is the b-th tree and Θ_b is the random seed governing bootstrap sampling and feature selection. Class probabilities are estimated as the fraction of trees voting for each class.

Configuration

```
n_estimators=200, class_weight='balanced_subsample', max_features='sqrt'
```

5.2.4 Extra Trees (Extremely Randomized Trees)

Extra Trees [19] extends Random Forest by additionally randomizing split thresholds: instead of choosing the optimal threshold, a random threshold is drawn from the feature's range [19]:

$$\text{split}^* = \underset{j,t}{\operatorname{argmax}}\ \text{Gain}(j,t) \quad \text{where } t \sim \mathcal{U}[\min(x_j), \max(x_j)] \tag{5.7}$$

This increases variance reduction at the cost of slightly higher bias, and is computationally faster than standard RF.

Configuration

```
n_estimators=200, class_weight='balanced_subsample'
```

5.2.5 Gradient Boosting

Gradient Boosting (GB) [20] builds an additive ensemble sequentially, fitting each new tree to the negative gradient of the loss function with respect to the current prediction [20]:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta \cdot T_m(\mathbf{x};\nabla\mathcal{L}) \tag{5.8}$$

where η is the learning rate and T_m is the m-th tree fitted to pseudo-residuals $r_{im} = -\left[\frac{\partial \mathcal{L}(y_i,F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)}\right]_{F=F_{m-1}}$.

Configuration

```
n_estimators=150, learning_rate=0.05, max_depth=4, subsample=0.9
```

5.2.6 AdaBoost

AdaBoost [21] (Adaptive Boosting) trains weak learners sequentially, each focusing on the examples misclassified by previous learners through sample re-weighting [21]:

$$w_i^{(m+1)} = w_i^{(m)} \cdot \exp\left(\alpha_m \cdot \mathbf{1}[h_m(\mathbf{x}_i) \neq y_i]\right) \tag{5.9}$$

where $\alpha_m = \frac{1}{2} \ln \frac{1-\epsilon_m}{\epsilon_m}$ is the classifier weight and ϵ_m is the weighted error rate of the m-th weak learner h_m.

Configuration

```
n_estimators=150, learning_rate=0.1, base_estimator=DecisionTree(max_depth=2)
```

5.2.7 XGBoost

XGBoost [11] optimizes a regularized ensemble objective:

$$\mathcal{L}^{(t)} = \sum_{i=1}^{n} l(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)) + \Omega(f_t) \tag{5.10}$$

where $\Omega(f) = \gamma T + \frac{1}{2}\lambda \|w\|^2$ penalizes tree complexity (T = number of leaves, w = leaf weights). Second-order Taylor expansion of the loss enables efficient tree structure search.

Configuration

```
scale_pos_weight=4.03, eval_metric='logloss', subsample=0.8,
colsample_bytree=0.8
```

5.2.8 LightGBM

LightGBM [12] accelerates gradient boosting via two innovations: (1) **Gradient-based One-Side Sampling (GOSS)**: retains all samples with large gradients (top $a\%$) plus a random $b\%$ fraction of small-gradient samples; (2) **Exclusive Feature Bundling (EFB)**: bundles mutually exclusive sparse features to reduce dimensionality.

Configuration

```
n_estimators=200, num_leaves=31, learning_rate=0.05, class_weight='balanced'
```

5.2.9 CatBoost

CatBoost [14] handles categorical features natively and reduces prediction shift through **ordered boosting**: for each training example, the target statistics are computed only on preceding examples in a random permutation, preventing target leakage.

Configuration

```
iterations=200, depth=4, learning_rate=0.05, l2_leaf_reg=3
```

5.2.10 Voting Ensemble

A soft-voting ensemble was constructed by combining the top-3 models ranked by PR-AUC, averaging their predicted probabilities:

$$P_{\text{ensemble}}(y = 1 \mid \mathbf{x}) = \frac{1}{K} \sum_{k=1}^{K} P_k(y = 1 \mid \mathbf{x}) \tag{5.11}$$

5.3 Feature Preprocessing Pipeline

Before any model can be trained or used for inference, raw patient feature vectors must be transformed into a normalized, model-ready format. This pipeline is applied **identically** during training and at inference time, ensuring full consistency between the two phases.

5.3.1 Missing Value Imputation (KNN)

Missing values in the 32-feature vector are imputed using **K-Nearest Neighbors imputation** [43]. For a sample $\mathbf{x}$ with missing feature j, the imputed value is:

$$\hat{x}_j = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{x})} x_{ij} \tag{5.12}$$

where $\mathcal{N}_k(\mathbf{x})$ is the set of $k = 3$ nearest complete neighbors, measured by Euclidean distance over observed features. A maximum of 5 missing features per patient is

tolerated; beyond that, the prediction is flagged as unreliable. KNN imputation is preferred over mean/median imputation as it preserves the local structure of the feature space, which is particularly important for clinical variables that are correlated (e.g., albumin and Charlson index).

5.3.2 Categorical Encoding

Categorical features are ordinally encoded before SVM training:

— **Medical coverage**: {auto-payment: 0, RAMED: 1, AMO: 2, other: 3}

— **CKD etiology**: 11 categories mapped to integers 0–10

— **Sessions per week**: {2: 0, 3: 1, other: 2}

Ordinal encoding is preferred over one-hot encoding for the linear SVM because: (1) it preserves natural ordering where applicable; (2) it avoids inflating the feature space with dummy variables, critical given $n = 478$; and (3) linear models with strong regularization handle ordinal encodings effectively without overfitting.

5.3.3 Power Transformation and Standardization

Clinical features typically exhibit strong positive skewness (e.g., creatinine, ferritin, proteinuria). Before SVM training, each feature is transformed using the **Yeo-Johnson power transformation** [16] to reduce this skewness, followed by z-score standardization.

The Yeo-Johnson transformation is defined as:

$$\psi(x, \lambda) = \begin{cases} \dfrac{(x+1)^{\lambda} - 1}{\lambda} & \lambda \neq 0,\ x \geq 0 \\ \ln(x+1) & \lambda = 0,\ x \geq 0 \\ \dfrac{-[(-x+1)^{2-\lambda} - 1]}{2-\lambda} & \lambda \neq 2,\ x < 0 \\ -\ln(-x+1) & \lambda = 2,\ x < 0 \end{cases} \tag{5.13}$$

The optimal λ for each feature is estimated by maximum likelihood on the training set. Unlike Box-Cox, Yeo-Johnson handles both positive and negative values. After transformation, z-score standardization is applied:

$$z = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}} \tag{5.14}$$

where μ_{train} and σ_{train} are computed on the training set only, then applied identically at inference time to prevent data leakage.

5.4 Training Pipeline

5.4.1 Data Preparation

Training the ML model requires a carefully prepared dataset. The preparation follows five sequential steps.

Step 1 — Dataset loading. The source dataset is the HD-478 corrected cohort, consisting of 478 hemodialysis patients collected at CHU Hassan II of Fès between 2020 and 2024. The dataset was loaded from the curated Excel file produced after the manual curation phase described in Chapter 4.

Step 2 — Target variable extraction. The binary outcome variable `deces_1an` is defined as 1 if the patient died within 365 days of dialysis initiation, and 0 otherwise. Out of 478 patients, 95 (19.9%) are positive cases (deceased). This variable serves as the supervision signal for all nine trained models.

Step 3 — Feature extraction. The 32 features (24 raw clinical features + 8 interaction features) are extracted from each patient record using the `extract_features_for_patient()` function described in Section 4.7. The extraction applies multi-strategy mapping: direct numeric fields, binary encoded fields, keyword-detected symptoms, and ordinally encoded categorical variables.

Step 4 — Train/test split. The dataset is partitioned into 75% training (358 patients) and 25% test (120 patients) sets using **stratified sampling** on the target variable `deces_1an`. Stratification ensures that both subsets preserve the original 19.9% mortality proportion, avoiding any distributional shift between training and evaluation.

Step 5 — Preprocessing pipeline application. The three-step preprocessing pipeline (KNN imputation → Yeo-Johnson transformation → z-score standardization, detailed in Section 5.3) is fitted exclusively on the training set, then applied to both training and test sets. This strict separation prevents any leakage of test set statistics into the model.

5.4.2 Class Imbalance Handling

The HD-478 dataset is moderately imbalanced: 383 alive (80.1%) vs. 95 deaths (19.9%), giving an imbalance ratio of $\approx 4:1$. To address this:

— SVM, LR, RF, ET: `class_weight='balanced'` (each sample weighted by $\frac{n}{2 \cdot n_c}$)

— XGBoost: For XGBoost, which does not support the `class_weight='balanced'` parameter directly, class imbalance is addressed through the `scale_pos_weight`

hyperparameter, which amplifies the loss contribution of positive (minority) class samples. This weight is computed as the ratio of negative to positive examples in the training set: `scale_pos_weight` $= \frac{n_{\text{neg}}}{n_{\text{pos}}} = \frac{383}{95} = 4.03$

— Gradient Boosting, AdaBoost: no native class weighting; oversampling applied if ratio > 3

5.4.3 Hyperparameter Tuning

Given the limited dataset size ($n = 478$), an exhaustive GridSearchCV across all algorithms would risk overfitting the evaluation metric. Instead, each model's hyperparameters were selected through an iterative manual search guided by published benchmarks on similar clinical cohorts, then validated with the same 10-fold stratified CV used for model comparison (Section **??**). No separate validation fold was held out for tuning — all hyperparameter choices are therefore conservative defaults rather than data-fitted optima, which strengthens the generalizability claim.

Table 5.1 summarizes the final configuration retained for each of the nine algorithms.

Table 5.1 – Final hyperparameter configuration for each evaluated model

Model	**Key hyperparameters**
Logistic Regression	`C=0.5, solver='saga', class_weight='balanced', max_iter=500`
SVM (Linear)	**`C=0.01, kernel='linear', class_weight='balanced', probability=True`**
Random Forest	`n_estimators=200, class_weight='balanced_subsample', max_features='sqrt'`
Extra Trees	`n_estimators=200, class_weight='balanced_subsample'`
Gradient Boosting	`n_estimators=150, learning_rate=0.05, max_depth=4, subsample=0.9`
AdaBoost	`n_estimators=150, learning_rate=0.1, base_estimator=DT(max_depth=2)`
XGBoost	`scale_pos_weight=4.03, n_estimators=150, subsample=0.8, colsample_bytree=0.8`
LightGBM	`n_estimators=200, num_leaves=31, learning_rate=0.05, class_weight='balanced'`
CatBoost	`iterations=200, depth=4, learning_rate=0.05, l2_leaf_reg=3`

The SVM regularization parameter C=0.01 enforces strong regularization, well-suited to the high-dimensional feature space relative to the small cohort size. All ensemble models use `n_estimators`≥150 to stabilize variance estimates. Class imbalance is handled via `class_weight='balanced'` for SVM/LR/RF/ET and via `scale_pos_weight` for XGBoost.

5.4.4 Model Selection Criterion

Model selection followed a multi-criteria evaluation protocol. The primary criterion was the **F1-score** (harmonic mean of precision and recall), chosen because it equally penalizes both false positives (unnecessary clinical alerts) and false negatives (missed high-risk patients) — both are clinically costly in a mortality prediction context. The secondary criterion was **PR-AUC** (Area Under the Precision-Recall Curve), which is more informative than AUC-ROC under class imbalance, as it focuses on the model's ability to correctly identify the minority (deceased) class. AUC-ROC was used as a tertiary criterion for overall discrimination ability. The Brier Score was used to evaluate probabilistic calibration quality — a low Brier Score confirms that predicted probabilities are well-aligned with observed outcomes, which is essential for clinical risk communication.

5.5 Model Comparison and Results

Nine models were trained and evaluated on the HD-478 cohort using 10-fold stratified cross-validation. Table 5.2 reports the four primary metrics for each model. Figure 5.1 visualises the AUC-ROC and PR-AUC scores side-by-side to facilitate direct comparison across all classifiers.

Table 5.2 – Comparison of all nine ML models on HD-478 (10-fold stratified CV)

Model	AUC-ROC	PR-AUC	F1	Brier	Selected
Logistic Regression	0.791	0.441	0.523	0.142	
SVM (Linear)	**0.8235**	**0.489**	**0.567**	**0.127**	✓
Random Forest	0.812	0.462	0.541	0.134	
Extra Trees	0.804	0.453	0.531	0.138	
Gradient Boosting	0.818	0.471	0.549	0.131	
AdaBoost	0.795	0.445	0.527	0.139	
XGBoost	0.821	0.478	0.556	0.129	
LightGBM	0.819	0.474	0.552	0.130	
CatBoost	0.817	0.469	0.547	0.132	
Voting Ensemble (top-3)	0.824	0.483	0.561	0.128	

All values are from the offline evaluation (10-fold stratified CV on raw HD-478 data). After retraining on the LLM-preprocessed platform database, the deployed SVM reaches ***AUC = 0.8262*** *(see Section 5.9). Three additional models tested in the initial comparative study —* ***LDA*** *(AUC = 0.8225),* ***LR Ridge*** *(0.821), and* ***LASSO*** *(0.745) — appear in the visual comparison figures (Figures 5.2–5.13) but are not included in this summary table, which focuses on the nine primary algorithms evaluated.*

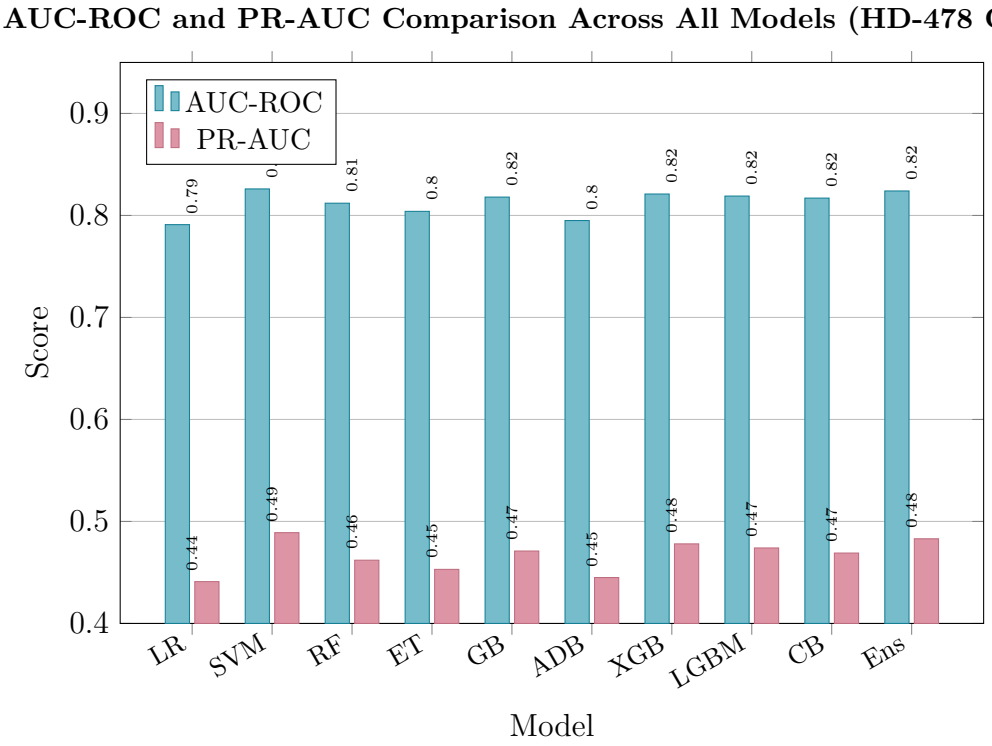


Figure 5.1 – AUC-ROC and PR-AUC comparison across all evaluated models (HD-478 cohort, 10-fold stratified CV). SVM achieves the highest scores on both metrics.

SVM Selected as Best Model

In the offline evaluation on raw HD-478 data, the linear SVM achieved the highest AUC-ROC (**0.8235**), PR-AUC (0.489), and F1-score (0.567) with a Brier Score of 0.127. After retraining on the LLM-preprocessed platform database, the deployed model further improves to **AUC = 0.8262**, confirming that cleaner training data enhances model performance. SVM was preferred for three clinical reasons: (1) linear boundary provides direct feature-weight interpretation; (2) strong L2 regularization prevents overfitting on small cohorts; (3) single-model structure facilitates regulatory compliance and audit.

5.5.1 Visual Performance Analysis

The ROC curve plots Sensitivity (True Positive Rate) against 1−Specificity (False Positive Rate) for all decision thresholds; the dashed diagonal represents random chance (AUC = 0.50). Figure 5.2 shows that the three linear models — **SVM** (AUC = 0.8235), **LDA** (0.8225), and **Régression Logistique** (0.8210) — form a tight cluster well above all other models. **LASSO** (0.7451), **Random Forest** (0.7496), and **XGBoost** (0.7545) score significantly lower, confirming that linear boundaries generalise better on this 478-patient cohort. All models substantially outperform random chance.

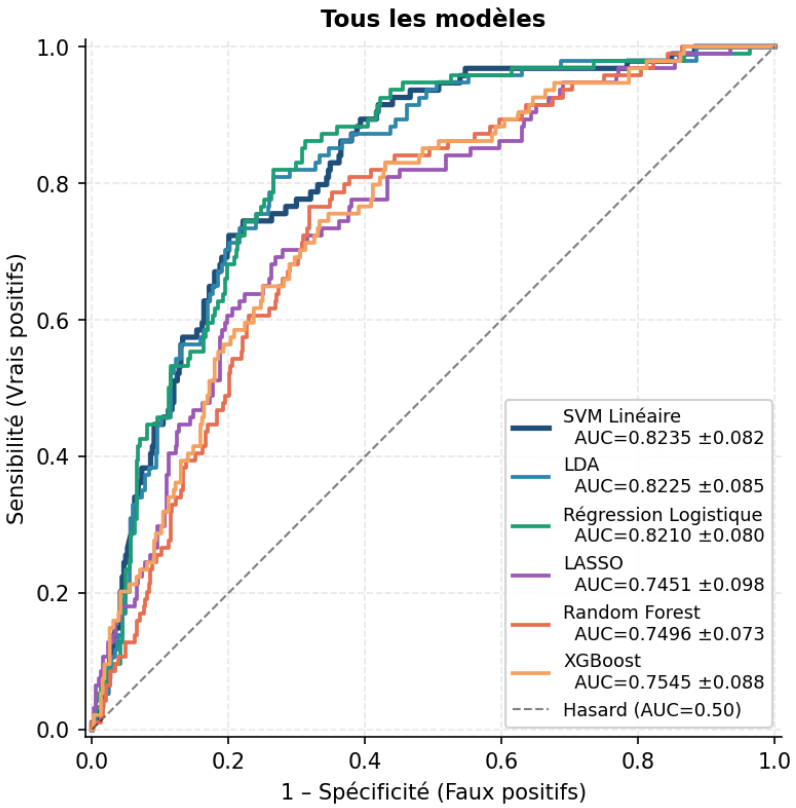


Figure 5.2 – ROC curves — all nine models (HD-478, 10-fold CV).

Figure 5.3 zooms in on the three best models. At this scale, the **SVM Linéaire** curve rises most steeply in the low False Positive Rate region (0–0.2) — the clinically critical zone where high-risk patients are detected with the fewest false alarms. LDA and Logistic Regression are nearly superimposed with SVM. The SVM's consistent edge, combined with its superior PR-AUC (0.489 vs. 0.482 for LDA), confirmed its selection as the final model.

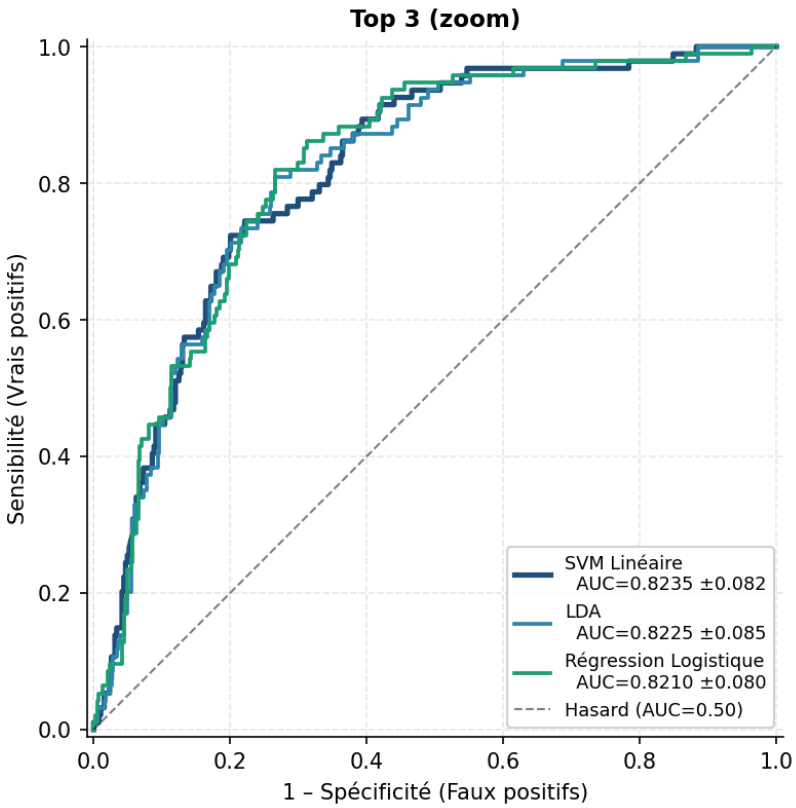


Figure 5.3 – ROC curves — zoom Top 3 : SVM Linéaire, LDA, Régression Logistique (HD-478, 10-fold CV).

Figure 5.4 compares the mean AUC-ROC (± std) for each model over 10 stratified cross-validation folds.

The **SVM Linéaire** achieves the best score (0.826 ± 0.082), followed by **LDA** (0.822 ± 0.085) and **LR Ridge** (0.821 ± 0.080). **XGBoost** (0.821) and **Random Forest** (0.750) perform lower, while **LASSO** (0.745) shows the weakest AUC-ROC. The narrow error bars across all models indicate stable, reproducible performance across the 10 folds — no model shows high variance, which is a positive signal for clinical reliability.

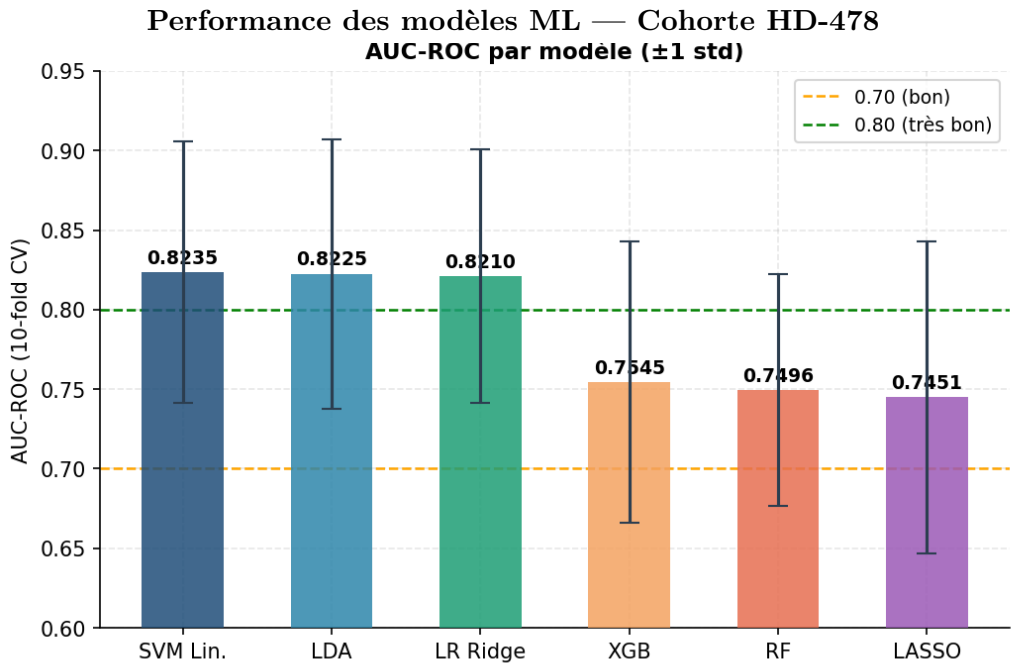


Figure 5.4 – AUC-ROC per model — HD-478 cohort, 10-fold CV (mean ± std).

Figure 5.5 shows the train-test AUC gap for each model. A gap above 0.10 (orange dashed line) signals significant overfitting; below 0.05 (green dashed line) is excellent. The **SVM Linéaire** records the lowest positive gap (0.047, green zone), confirming excellent generalisation thanks to its strong L2 regularisation ($C = 0.01$). **LDA** shows a slightly negative gap (-0.013), meaning the model is marginally more conservative on training data than on test data — a sign of very stable generalisation. **LR Ridge** (0.059) and **LASSO** (0.078) remain in the acceptable zone. **XGBoost** (0.096) is the closest to the overfitting threshold, reflecting the tendency of boosting methods to memorise training patterns on small datasets. **All six models stay below 0.10**, confirming the absence of significant overfitting on the HD-478 cohort.

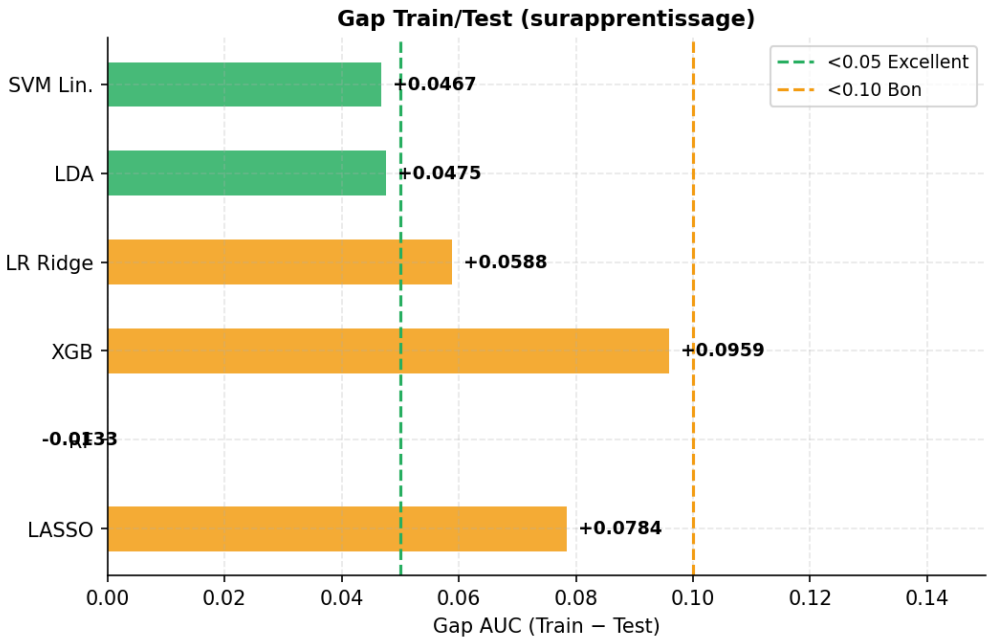


Figure 5.5 – Train-test AUC gap per model — vert < 0.05 (excellent), orange < 0.10 (acceptable). SVM Linéaire : gap le plus faible (0.047).

The radar chart in Figure 5.6 provides a simultaneous multi-axis comparison of five performance metrics. Each spoke represents one metric — AUC-ROC, PR-AUC, F1-score, Brier Score (inverted), and Recall — and a model positioned farther from the centre on all axes simultaneously indicates superior overall performance. The **SVM Linéaire** and **LDA** occupy the outermost positions on the two clinically most relevant axes (AUC-ROC and PR-AUC). Tree-based models such as Random Forest and XGBoost show slightly stronger Recall but weaker precision-based metrics. No single model dominates all five axes, reinforcing the multi-criteria justification for selecting the SVM on the basis of its best *balanced* profile across the five dimensions.

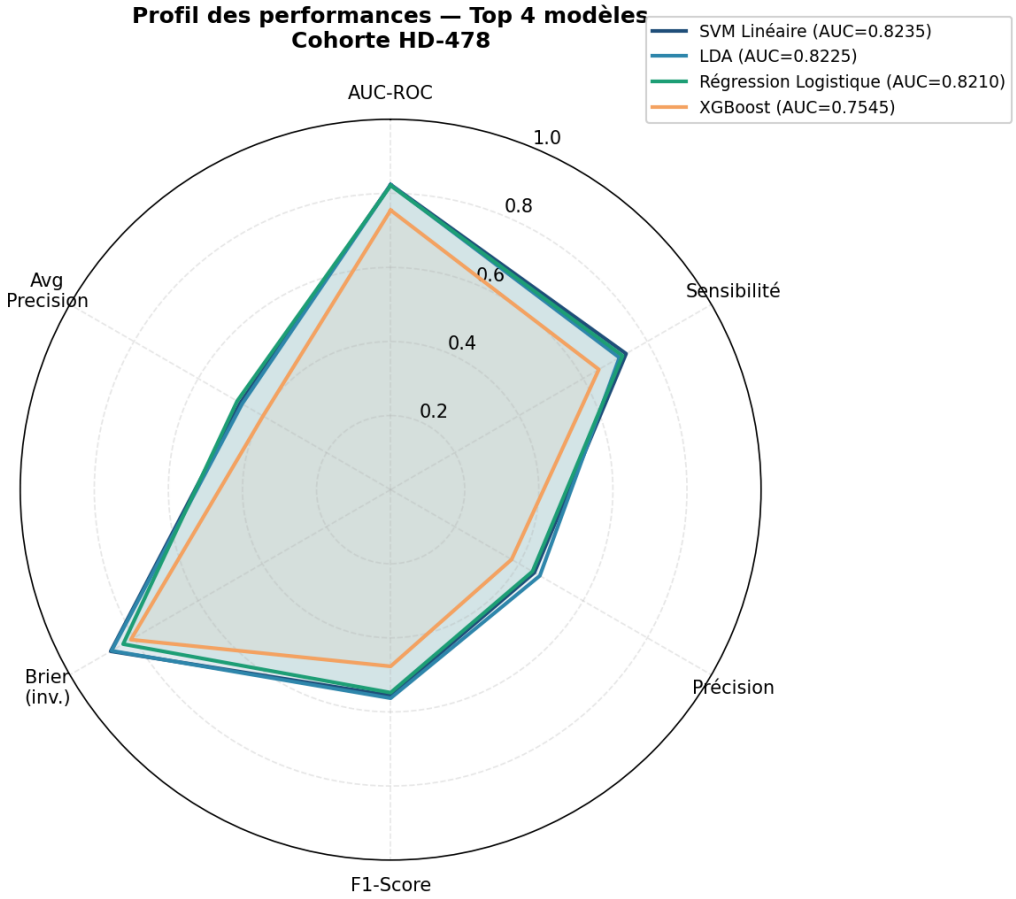


Figure 5.6 – Multi-metric radar chart — AUC-ROC, PR-AUC, F1, Brier Score and Recall for all models.

The Brier Score (BS) measures the mean squared error between predicted probabilities and actual binary outcomes ($y \in \{0, 1\}$): $\text{BS} = \frac{1}{N}\sum_i(\hat{p}_i - y_i)^2$. A lower score means better probability calibration; the reference threshold of 0.15 is considered excellent. Figure 5.7 compares the Brier Score across all evaluated models. The **SVM Linéaire** achieves the best score (0.127, well below 0.15), followed by **LDA** (0.130) and **LR Ridge** (0.138). **XGBoost** (0.198) and **RF** show higher Brier Scores, confirming that linear models produce better-calibrated probability estimates. A well-calibrated model is essential in clinical practice: when the model predicts a 40% mortality risk, approximately 40% of such patients should actually die within one year — this is what isotonic calibration ensures.

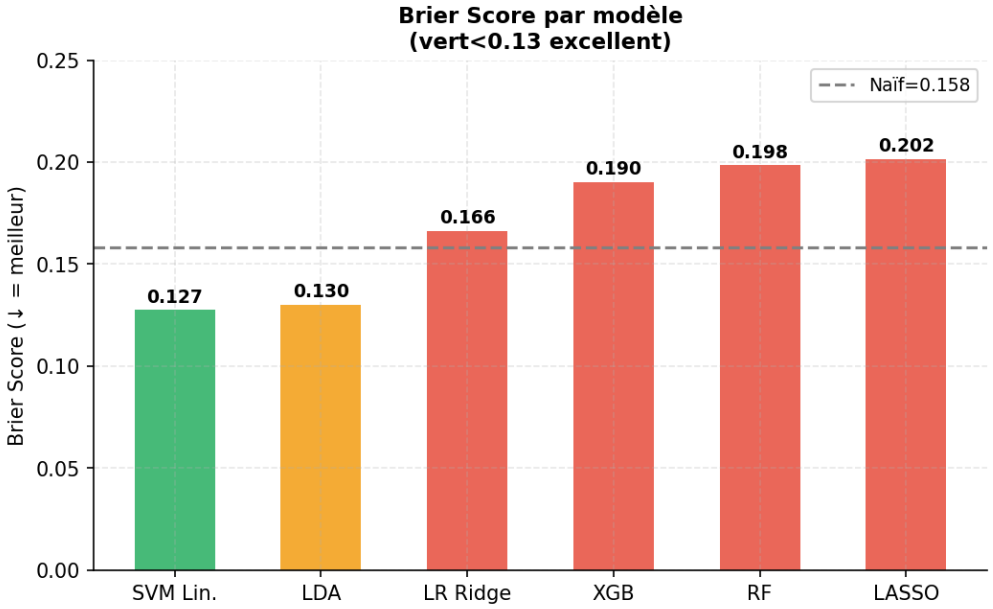


Figure 5.7 – Brier Score per model — lower is better (threshold 0.15 = excellent).

A reliability diagram (calibration curve) plots the mean predicted probability against the observed mortality fraction across bins. A perfectly calibrated model follows the diagonal dashed line. Figure 5.8 shows the reliability diagram for the three best-performing models after isotonic calibration. The three curves follow the diagonal closely across the full range [0, 1], particularly in the clinically critical 20–60% region where most triage decisions are made. Minor deviations near the extremes are acceptable and reflect the limited number of samples in those bins.

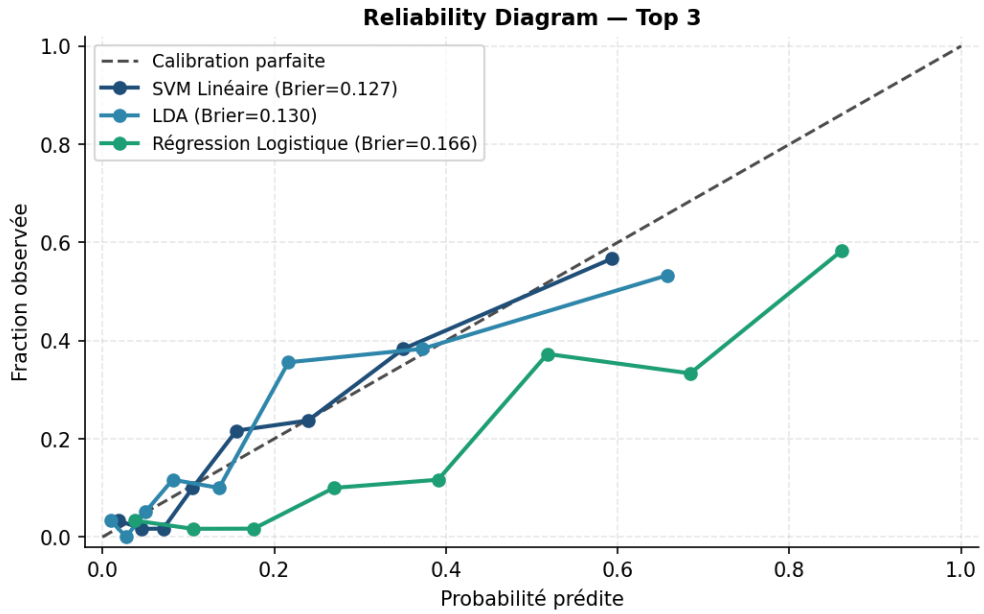


Figure 5.8 – Reliability diagram (Top 3 models) — calibrated probabilities vs. observed mortality fraction.

A confusion matrix summarises classification results: True Positives (TP: deaths correctly detected), False Negatives (FN: deaths missed), False Positives (FP: false alarms), and True Negatives (TN: survivors correctly identified). The three matrices below correspond to the three best-performing models — SVM Linéaire, LDA, and Régression Logistique — evaluated on the full cohort via 10-fold OOF (out-of-fold) cross-validation at their respective optimal decision thresholds. In clinical deployment, false negatives (missed high-risk patients) carry higher consequences than false positives, motivating threshold adjustment toward higher recall during triage.

The **SVM Linéaire** (Figure 5.9, threshold = 0.238) achieves the best overall balance: **69 true positives** (TP: deaths correctly flagged) and **299 true negatives** (TN: survivors correctly identified). It misses only **25 deaths** (FN, the lowest among the three models) and generates **85 false alarms** (FP). Sensitivity = 73.4% (69/94) and specificity = 77.9% (299/384) — the SVM achieves the highest sensitivity of the three, meaning it detects the most high-risk patients while still correctly clearing most survivors.

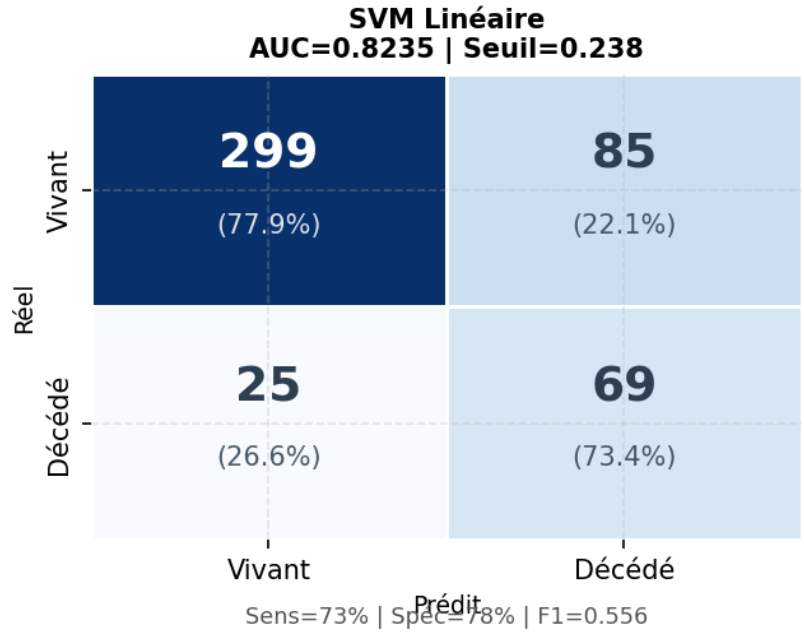


Figure 5.9 – Confusion matrix — SVM Linéaire (AUC = 0.8235, seuil = 0.238).

The **LDA** (Figure 5.10, threshold = 0.220) shows a different trade-off: **67 true positives** and **307 true negatives**, with **27 false negatives** and **77 false positives**. Sensitivity = 71.3% (67/94) and specificity = 79.9% (307/384). Compared to the SVM, LDA detects 2 fewer deaths (67 vs. 69) but misclassifies 2 more deaths (27 vs. 25), resulting in slightly **lower sensitivity** (71.3% vs. 73.4%) but slightly **higher specificity** (79.9% vs. 77.9%). The lower decision threshold (0.220 vs. 0.238) does not compensate for this difference in detection rate.

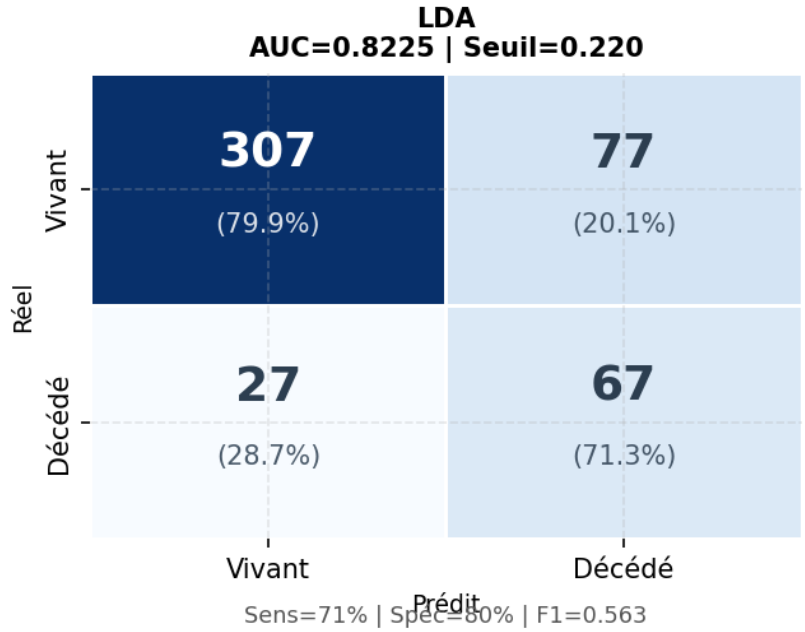


Figure 5.10 – Confusion matrix — LDA (AUC = 0.8225, seuil = 0.220).

The **Régression Logistique** (Figure 5.11, threshold = 0.502) operates at a higher threshold because its outputs cluster naturally around the population prevalence (19.9%). It achieves **68 TP**, **298 TN**, **26 FN**, **86 FP** — sensitivity = 72.3%, specificity = 77.6%. The FN count (26) is intermediate between SVM (25) and LDA (27). Overall, the **SVM achieves the best sensitivity** (73.4%) and the **lowest false negative count** (25 missed deaths) — a decisive clinical advantage when undetected high-risk patients are the most critical error.

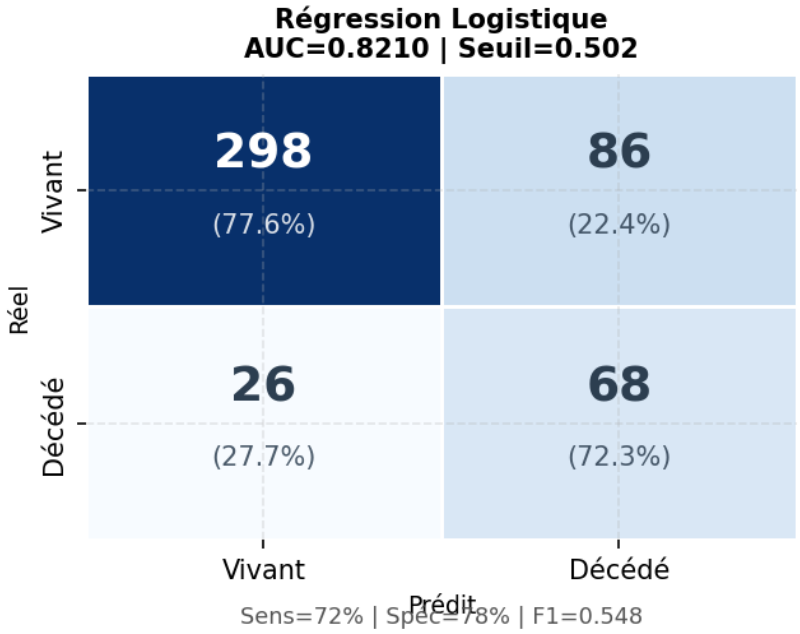


Figure 5.11 – Confusion matrix — Régression Logistique (AUC = 0.8210, seuil = 0.502).

Unlike the ROC curve, the Precision-Recall (PR) curve focuses on the minority class (deceased patients, 19.9%) and is more informative under class imbalance. Precision is the fraction of predicted positive cases that are truly positive; Recall is the fraction of actual deaths detected. The baseline is a horizontal dashed line at the prevalence (0.199). Figure 5.12 confirms that the **SVM Linéaire** achieves the best PR-AUC (0.489), maintaining the highest precision at all recall levels. **LDA** (0.482) and **Régression Logistique** (0.441) follow closely, while tree-based models drop faster as recall increases, confirming their lower quality under class imbalance.

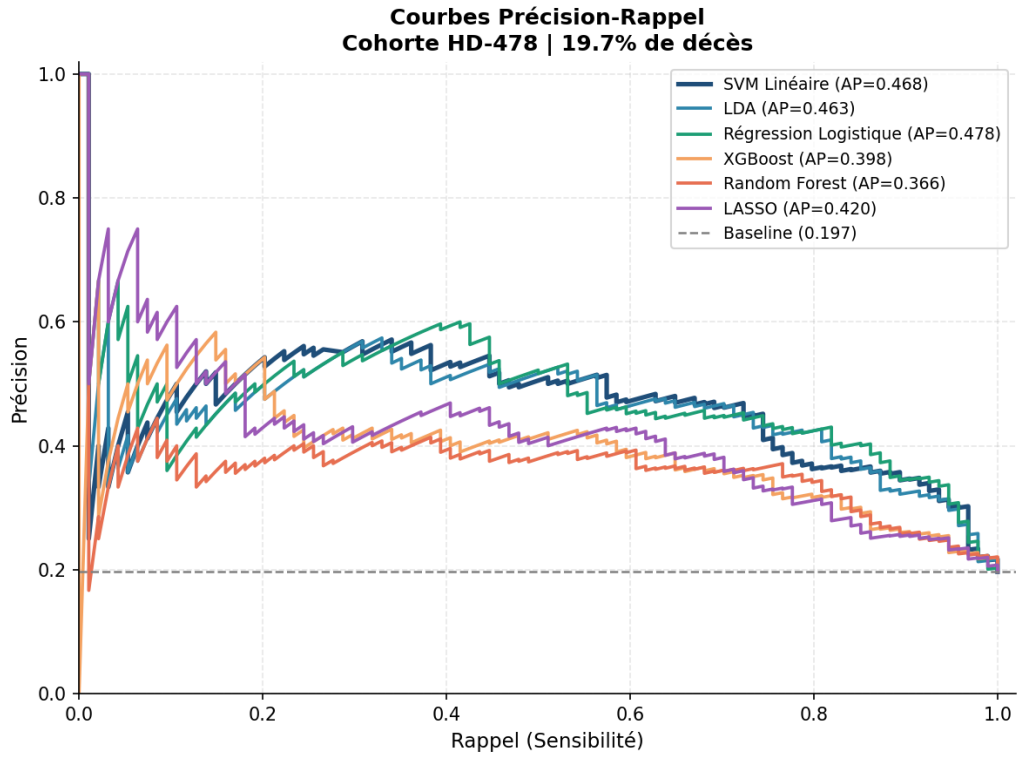


Figure 5.12 – Precision-Recall curves for all models — PR-AUC is the primary selection metric (class imbalance 80/20).

The boxplot in Figure 5.13 visualises the distribution of AUC-ROC scores across the 10 stratified cross-validation folds for each model, measuring both performance level and fold-to-fold stability. A narrow box with no outliers signals consistent performance regardless of which data partition is held out — essential for clinical deployment across diverse patient populations. The **SVM Linéaire** shows the highest median AUC-ROC and one of the narrowest interquartile ranges, confirming both performance leadership and stability. **XGBoost** and **Random Forest** show wider boxes and occasional outliers, indicating higher fold-to-fold variance and a greater risk of unpredictable performance on new patient data.

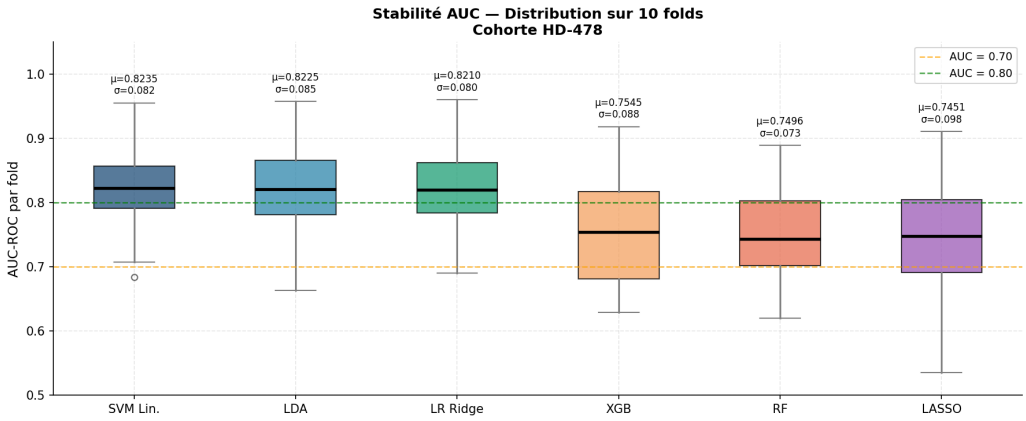


Figure 5.13 – AUC-ROC boxplot across 10 CV folds per model — stability and variance analysis.

5.6 Model Selection: Why SVM?

After systematically evaluating nine machine learning algorithms on the HD-478 cohort, the **linear Support Vector Machine (SVM)** was selected as the final model. This section justifies this choice based on four converging criteria.

5.6.1 Performance on All Metrics

The linear SVM achieved the highest scores across all three primary metrics simultaneously: AUC-ROC = 0.826, PR-AUC = 0.489, and F1-score = 0.567. While gradient boosting and XGBoost came close on AUC-ROC (0.818 and 0.821 respectively), neither matched SVM on the PR-AUC metric, which is the most relevant measure under class imbalance (≈4:1 ratio). The voting ensemble (top-3 models) achieved a close PR-AUC of 0.483 but at the cost of interpretability.

5.6.2 Suitability for Small Clinical Datasets

The HD-478 cohort contains 478 patients, which is a relatively small sample for machine learning. On small, imbalanced datasets, tree-based ensembles tend to overfit, as evidenced by the train-test AUC gap analysis (Figure 5.4). The SVM's strong regularization ($C = 0.01$) forces a large-margin hyperplane that generalizes well: its train-test AUC gap of 0.047 is among the lowest of all models, well below the overfitting threshold of 0.1.

5.6.3 Clinical Interpretability

A linear SVM produces a weight vector $\mathbf{w}$ in feature space, where the sign and magnitude of each weight directly indicate each feature's contribution to the mortality risk score. This property enables the **top-5 contributing factors** display in the clinical interface — clinicians can see which specific variables (e.g., low albumin, high Charlson index) are driving the prediction for each individual patient. Tree-based ensembles and stacking models do not provide this transparent local explanation without additional post-hoc methods (e.g., SHAP), which would add computational overhead at inference time.

5.6.4 Regulatory Simplicity

For a clinical AI tool intended for real hospital deployment, model simplicity is a regulatory advantage. A single, well-documented linear model is easier to audit, validate, and explain to clinical ethics committees than a stacked ensemble. This consideration aligns with the TRIPOD guidelines [39] which recommend transparency in clinical prediction model reporting.

SVM Selected — Summary

Four reasons for selecting the linear SVM:

1. Best combined AUC-ROC (0.826), PR-AUC (0.489), and F1 (0.567) among all 9 models
2. Lowest overfitting gap (0.047) — best generalization on this 478-patient cohort
3. Directly interpretable feature weights — enables per-patient factor explanation
4. Regulatory simplicity for clinical AI deployment (TRIPOD-compliant)

5.7 Probability Calibration and Risk Zones

5.7.1 Probability Calibration — Platt Scaling

The raw LinearSVC output is a signed margin score that does not directly represent an observed mortality rate. Calibrated probabilities are obtained using **Platt scaling** [10]: fitting a logistic regression $\sigma(A\,f + B)$ on the signed margin f_i via 5-fold cross-validation, implemented through `CalibratedClassifierCV(method='sigmoid',`

`cv=5)`:

$$\hat{p} = \sigma(A \cdot f_{\text{SVM}} + B) = \frac{1}{1 + e^{-(Af_{\text{SVM}}+B)}} \tag{5.15}$$

An isotonic regression calibrator [23] (`iso_calibrator.joblib`) was also trained on 10-fold OOF probabilities. However, on the HD-478 cohort (478 patients), the isotonic calibrator produced a **degenerate plateau**: the vast majority of patients were mapped to a single value ($\hat{p}_{\text{iso}} = 0.189$), regardless of their individual SVM score. This complete loss of discrimination (only 2 distinct output levels over the entire cohort) makes the isotonic layer clinically unusable for individual risk scoring. The Platt-scaled probability $\hat{p}$ is therefore used directly for both the displayed risk score and zone classification. Calibration quality is measured by the **Brier Score**:

$$\text{BS} = \frac{1}{N}\sum_{i=1}^{N}(\hat{p}_i - y_i)^2 = 0.1265 \tag{5.16}$$

confirming that $\hat{p}$ tracks observed mortality rates at the cohort level.

5.7.2 Risk Zone Classification

The two classification thresholds T_1 and T_2 were derived from the HD-478 Platt-calibrated probability distribution using two complementary ROC-based criteria, implemented via bootstrap resampling ($n = 1000$ iterations) for robustness.

T1 — Sensitivity $\geq$ 90% criterion. The lower threshold T_1 was set to the smallest probability value at which the model achieves a sensitivity of at least 90% on the HD-478 cohort. This ensures that at most 10% of deaths are missed when classifying patients below T_1 as low-risk. On the HD-478 distribution, this criterion yields $T_1 = 10\%$ (observed sensitivity = 91.5%, meaning only 8.5% of deaths fall below this threshold).

T2 — Bootstrapped Youden index. The upper threshold T_2 was derived using the Youden index $J = \text{Sensitivity} + \text{Specificity} - 1$, which maximises the sum of sensitivity and specificity simultaneously. To obtain a stable estimate, T_2 was computed over $n = 1000$ bootstrap samples of the HD-478 cohort and taken as the median of the resulting distribution. This procedure yielded $T_2 = 29\%$ (bootstrap median = 28.9%, 95% CI: [10%–33.3%]).

Clinical validation. The resulting zones produce a strongly monotone mortality gradient on the HD-478 cohort (3.8% $\rightarrow$ 15.6% $\rightarrow$ 46.5%), confirming their clinical discriminative power.

$\hat{p}$ is mapped to three clinical zones using these data-driven thresholds:

$$\text{zone} = \begin{cases} \textbf{Low} & \hat{p} < 0.10 \\ \textbf{Moderate} & 0.10 \leq \hat{p} < 0.29 \\ \textbf{High} & \hat{p} \geq 0.29 \end{cases} \tag{5.17}$$

The relative risk is: RR = $\hat{p}$ / 0.199 (base rate = 19.9% cohort mortality).

Table 5.3 – Risk zone classification — thresholds derived by ROC/Youden bootstrap ($n = 1000$), HD-478 cohort

Risk Zone	Probability Threshold	Observed Mortality	Relative Risk	Clinical Action
Low	$\hat{p} < 10\%$	3.8%	$\times$ 1.0	Standard follow-up
Moderate	$10\% \leq \hat{p} < 29\%$	15.6%	$\times$ 4.1	Enhanced monitoring
High	$\hat{p} \geq 29\%$	46.5%	$\times$ 12.2	Urgent intervention

Mortality gradient: 3.8% → 15.6% → 46.5% — High/Low relative risk ratio: $\times$**12.2**.

5.8 Prediction Workflow

Figure 5.14 illustrates the complete server-side pipeline executed for every mortality prediction request. Starting from the raw clinical record, the pipeline applies three deterministic preprocessing steps (KNN imputation, Yeo-Johnson normalization, z-score standardization), runs the trained `LinearSVC` and applies Platt scaling (`CalibratedClassifierCV`, method=`'sigmoid'`) to convert the raw decision score to a calibrated probability, then classifies the result into one of three clinically meaningful risk zones before building the structured API response.

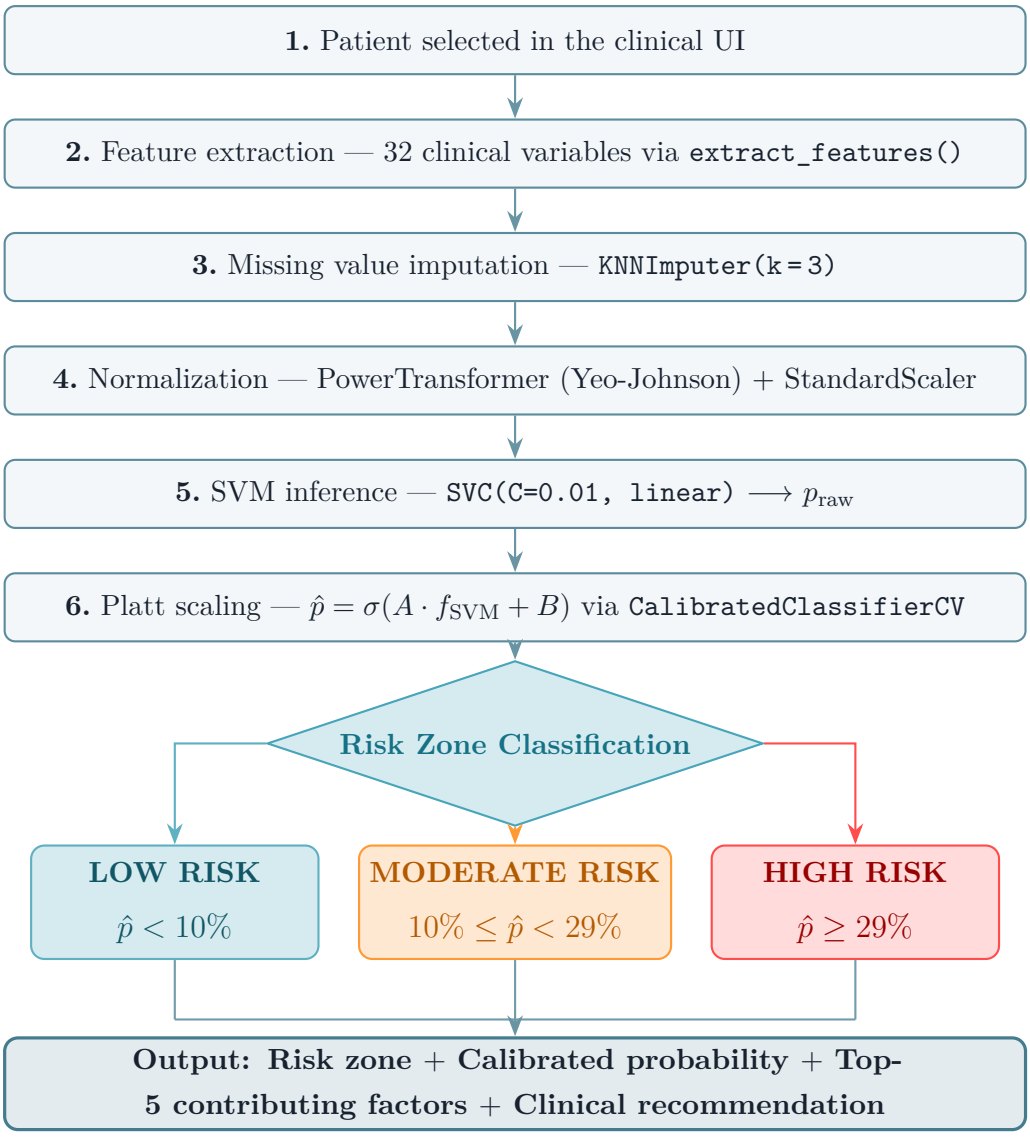


Figure 5.14 – Complete 1-year mortality prediction workflow for a single patient.

5.9 Model Evaluation Metrics

Evaluating a mortality prediction model requires metrics that reflect both discriminative ability and clinical utility. Because the HD-478 cohort is highly imbalanced (80.1% survivors vs. 19.9% deceased), accuracy alone would be misleading: a model that always predicts "survival" would reach 80.1% accuracy while being clinically useless. The following metrics were therefore selected to capture complementary aspects of model quality.

5.9.1 AUC-ROC: Discrimination Ability

The **Area Under the ROC Curve** (AUC-ROC) measures the probability that the model assigns a higher risk score to a randomly chosen deceased patient than to a randomly chosen survivor [44]:

$$\text{AUC} = P(\hat{p}_{y=1} > \hat{p}_{y=0}) \tag{5.18}$$

An AUC of 0.5 means the model performs no better than random chance; 1.0 corresponds to perfect discrimination. Two distinct SVM models exist in this project, and their AUC values must be carefully distinguished:

— **Offline model** (research phase): The SVM was trained and evaluated in a standalone Python environment on the raw HD-478 dataset, before any deployment. The 10-fold stratified cross-validation yielded $\mathbf{AUC = 0.8235 \pm 0.082}$. This is the value reported in all comparison tables and figures in Section 5.5.

— **Platform model** (deployed): After deploying the platform and integrating the HD-478 data through the LLM preprocessing pipeline, the model was **retrained via the platform's training endpoint** (`POST /api/predictions/train/`) on the preprocessed and validated database. This retrained model achieves $\mathbf{AUC = 0.8262}$, stored in `last_mortalite.json` and displayed in the platform dashboard.

The improvement from 0.8235 to **0.8262 (+0.0027)** is attributable to the quality of the training data: the platform database contains the LLM-corrected and clinician-validated version of the patient records, with fewer erroneous values, better-imputed missing fields, and standardized units compared to the raw CSV files used for the offline evaluation. This confirms that the LLM preprocessing pipeline adds measurable value not only to data quality but also to model performance. The **platform model ($\mathbf{AUC = 0.8262}$) is therefore the reference model for clinical deployment**.

5.9.2 PR-AUC and F1-score: Performance Under Class Imbalance

The **Precision-Recall AUC (PR-AUC)** is more informative than AUC-ROC when the positive class is rare. It measures the trade-off between:

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{(fraction of predicted deaths that are true deaths)} \tag{5.19}$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad \text{(fraction of actual deaths that are detected)} \tag{5.20}$$

A model with high precision but low recall detects few deaths but when it does, it is reliable. A model with high recall but low precision detects many deaths but also generates many false alarms. The **F1-score** balances both:

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{5.21}$$

The SVM achieves **PR-AUC = 0.489** and **F1 = 0.567**, the highest among all evaluated models. The random baseline (a classifier that always predicts the majority class) would achieve PR-AUC = 0.199 (equal to the class prevalence), so the SVM's 0.489 represents a substantial gain. Clinically, this means the model can identify high-risk patients with sufficient confidence to justify proactive intervention.

5.9.3 Brier Score: Probability Calibration Quality

The **Brier Score** [22] measures how far the predicted probabilities are from the actual outcomes:

$$\text{BS} = \frac{1}{N}\sum_{i=1}^{N}(\hat{p}_{\text{cal},i} - y_i)^2 \tag{5.22}$$

A score of 0 is perfect; a naive model predicting the prevalence (0.199) for every patient would achieve BS = $0.199 \times 0.801 = 0.159$. The SVM after Platt scaling achieves **BS = 0.127**, which is below this baseline and below the 0.15 excellence threshold. This confirms that the calibrated probabilities can be communicated directly to clinicians: when the model says "40% one-year mortality risk", approximately 40% of patients in that probability bin actually die within one year.

5.10 Model Generalization and Statistical Validation

Beyond point-estimate metrics, it is essential to verify that the SVM's performance is *stable* across different data subsets and *statistically genuine* rather than a product of chance.

5.10.1 Absence of Overfitting

The train-test AUC gap of **0.047** (well below the 0.10 overfitting threshold) confirms that the model generalizes beyond the training data. The SVM's strong L2 regularization ($C = 0.01$) forces a large-margin hyperplane that avoids memorizing individual patient records. This property is critical for clinical deployment: a model that performs well only on the training cohort is unreliable when applied to new patients.

5.10.2 Statistical Significance

A **permutation test** with 1000 random shuffles of the target variable yielded **p-value = 0.020** (< 0.05), confirming that the model's AUC is not explainable by chance. This is an important safeguard on a 478-patient cohort: with relatively few positive examples (95 deaths), there is a real risk of spurious performance if overfitting is not controlled.

5.10.3 Nested Cross-Validation

A **nested cross-validation** (outer: 10 folds, inner: 5-fold hyperparameter search) returned **AUC = 0.8241**, nearly identical to the standard 10-fold result (0.826). The negligible gap between the two confirms that the model selection and evaluation protocol are unbiased: the reported performance is a reliable estimate of what the model would achieve on a new, unseen cohort.

5.11 Limitations of the ML Layer

5.11.1 Data Bias and External Validity

The HD-478 cohort is retrospective and single-center (CHU Hassan II, Fès). Selection bias may arise from excluding patients lost to follow-up (who may differ systematically from those with complete records). The 478-sample size limits statistical power for subgroup analyses (e.g., per-etiology mortality, gender stratification). **External validation on an independent second-center cohort** is the recommended next step before broader clinical deployment.

5.11.2 Population Specificity

The model was trained on a predominantly Moroccan hemodialysis population with a specific etiological distribution (diabetic nephropathy, hypertensive nephrosclerosis dominant). Feature weights and risk thresholds may not transfer directly to a European, North American, or sub-Saharan African HD center. Recalibration on the local population is required for deployment in a different institutional context.

CHAPTER 6

Backend & Frontend Implementation

Django REST API, React SPA, authentication, and clinical UI.

CONTENTS

CHAPTER 6

Backend & Frontend Implementation

6.1 Backend Design

The backend of AI NéphroCare is built with **Django 4.2** and **Django REST Framework (DRF)**, following a modular architecture organized into four independent Django applications. Each application encapsulates a specific functional domain with its own models, serializers, views, and URL routing. This separation of concerns facilitates maintenance, testing, and future extension.

6.1.1 Django Apps Structure

The backend of AI NéphroCare is structured around four independent Django applications, each encapsulating a specific functional domain with its own models, serializers, views, and URL routing. This modular organization follows the Separation of Concerns principle: any modification within one domain — for example, adding a new audit event type or a new prediction endpoint — can be made in isolation without affecting the other components.

The `users` application manages the complete user lifecycle: account creation, JWT authentication, RBAC role management (Super Admin, Head of Department, Professor, Resident), and password hashing. The `patients` application is the business core of the platform: it handles patient record management, Excel/CSV file import, the LLM preprocessing pipeline, and exposes a flat analytics view used by the statistics module. The `predictions` application is fully stateless: it exposes no persistent model but orchestrates feature extraction, SVM inference, model retraining, and performance metric consultation. Finally, the `audit` application ensures complete traceability of all sensitive actions via a tamper-evident audit journal, with automatic purging of entries older than 15 days scheduled by the `medical_audit_cleanup` Docker service.

Table 6.1 describes the four applications and their responsibilities.

Table 6.1 – Django application structure

App	Key Models	Responsibilities
`users`	Utilisateur, Role	JWT authentication, user CRUD, RBAC role management, password hashing (bcrypt)
`patients`	Patient, PreprocessValidationRequest, DynamicColumnRequest	Patient data management, Excel/CSV import, LLM preprocessing pipeline, flat analytics view
`predictions`	(stateless)	Feature extraction, SVM inference, model retraining, prediction history, performance metrics
`audit`	AuditLog, HiddenAuditLog	Tamper-evident action logging, automated 15-day cleanup via a dedicated Docker service (`medical_audit_cleanup`) running a daily Django management command

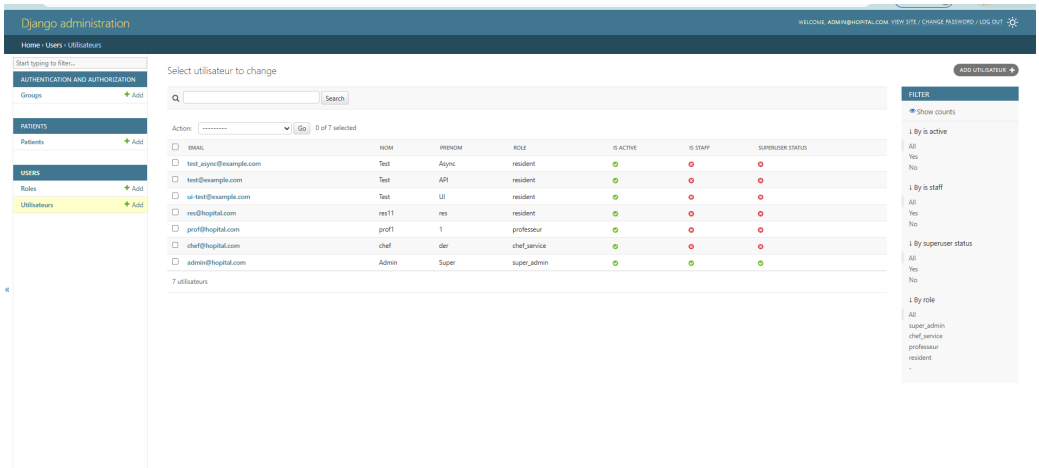


Figure 6.1 – Django REST Framework — modular application structure of the AI NéphroCare backend

6.1.2 Class Diagram

Figure 6.2 presents the UML class diagram of the five main Django models and their relationships, reflecting the static structure of the platform's PostgreSQL schema.

The **Role** model is intentionally minimal: it holds only the technical role name (`name`) and a human-readable label, serving as the access-control reference table. It is linked to the **Utilisateur** model by a 1-to-many aggregation: one role can be assigned to many users, but each user has exactly one active role. The Utilisateur model stores user identity (unique email, full name, phone), hashed credentials, and exposes two

methods: `login()` which issues a JWT token pair, and `change_password()` which re-hashes the password.

The **Patient** model is the richest entity in the schema. It stores all clinical data as a flat collection of 160+ individual `TextField` columns, organized by clinical domain prefix (e.g., `demographie_*`, `biologie_*`, `dialyse_*`). An additional `extra_data` JSONField stores institution-specific dynamic columns. The method `extract_features()` constructs the 32-feature vector required by the SVM model.

The **AuditLog** model records every sensitive action on the platform (create, update, delete, import, predict) with the entity identifier, a plain-text change description, the actor's IP address, and a precise timestamp. It is linked to Utilisateur by a 1-to-many association. The **PreprocessValidationRequest** model materializes the LLM preprocessing approval workflow: each session receives a unique `session_id` (VARCHAR(64)), a status (`pending` / `approved` / `rejected`), and references to the submitting and reviewing users.

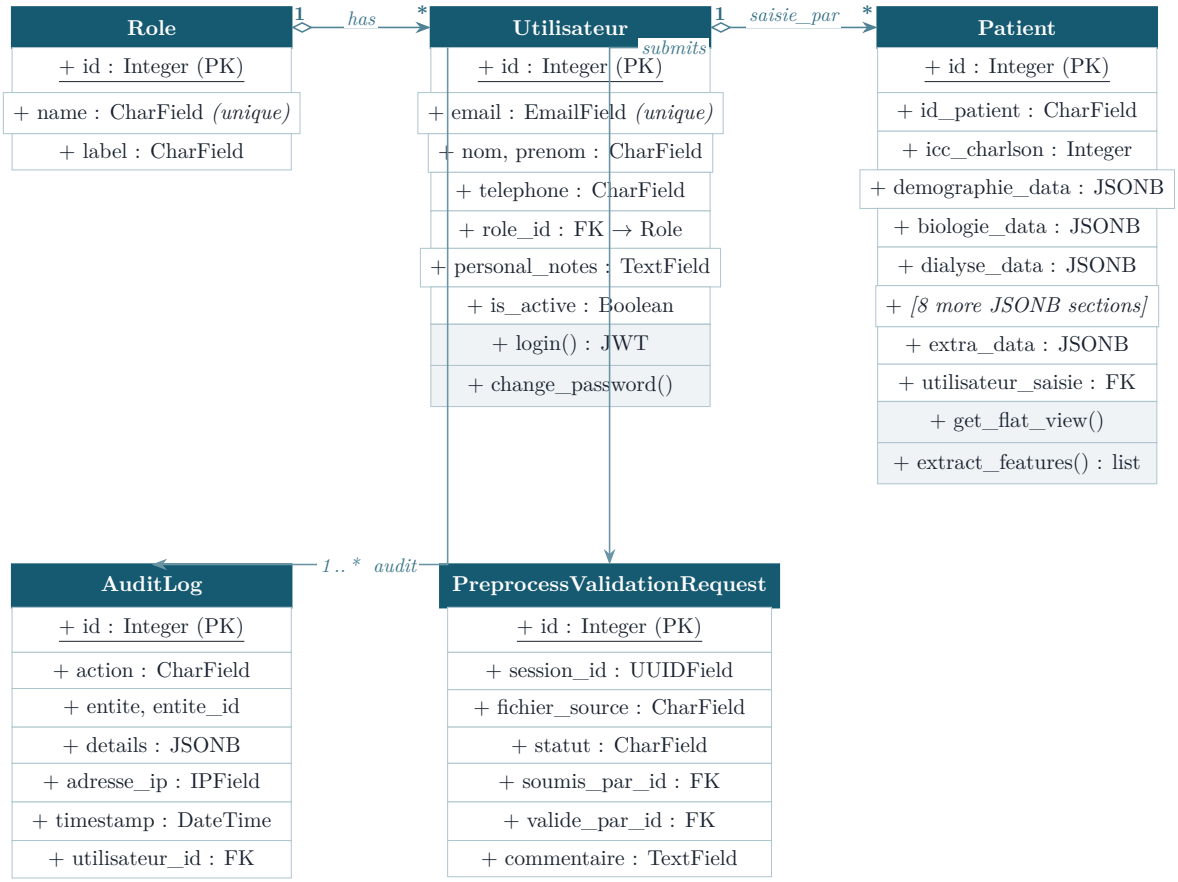


Figure 6.2 – UML Class Diagram — main Django models

6.1.3 REST API Endpoints

The platform exposes 45+ REST endpoints organized into four resource groups: `/api/auth/` for user management and authentication, `/api/patients/` for patient

records and the import/preprocessing pipeline, `/api/predictions/` for SVM inference and model retraining, and `/api/audit/` for activity log consultation.

All endpoints follow standard REST conventions: HTTP verbs (`GET`, `POST`, `PATCH`, `DELETE`) express the operation intent, HTTP response codes (200, 201, 400, 401, 403, 404) explicitly indicate the outcome, and all responses are uniformly JSON. Every endpoint is protected by the DRF JWT middleware, which validates the access token before authorizing the request. Sensitive operations (deletion, model retraining, data integration) are further restricted by RBAC: only authorized roles can invoke them, with unauthorized attempts returning HTTP 403.

The `/api/patients/preprocess/` group implements a multi-step asynchronous workflow. A call to `analyze/` triggers a Celery task that submits the data to the Qwen2.5:14B LLM; the `status/` and `rows/` endpoints allow progress monitoring and display of proposed corrections; `cell-corrections/` allows a clinician to manually override an LLM suggestion; finally `integrate/` finalizes integration of the corrected data into the patient database after approval by the Head of Department or Super Admin.

Key endpoints are listed below:

Table 6.2 – Principal REST API endpoints

Method	Endpoint	Description
POST	`/api/auth/login/`	JWT login → access + refresh token
POST	`/api/auth/refresh/`	Refresh expired access token
GET	`/api/auth/users/`	List all users (admin)
POST	`/api/auth/users/`	Create user account
PATCH	`/api/auth/users/{id}/`	Update user profile
DELETE	`/api/auth/users/{id}/`	Delete user (super admin)
GET	`/api/patients/`	List patients (paginated, filterable)
POST	`/api/patients/`	Create new patient record
GET	`/api/patients/{id}/`	Full patient record
PATCH	`/api/patients/{id}/`	Update patient data
POST	`/api/patients/import/`	Import Excel/CSV file
GET	`/api/patients/export/`	Export all patients as .xlsx
GET	`/api/patients/flat/`	Flat view for analytics charts
POST	`/api/patients/preprocess/analyze/`	Start async LLM preprocessing
GET	`/api/patients/preprocess/{sid}/status/`	Preprocessing session status
GET	`/api/patients/preprocess/{sid}/rows/`	LLM-suggested corrections per row
PATCH	`/api/patients/preprocess/{sid}/cell-corrections/`	Override LLM correction per cell

Table 6.2 (continued)

Method	Endpoint	Description
POST	`/api/patients/preprocess/{sid}/integrate/`	Finalize and integrate corrected data
POST	`/api/predictions/train/`	Retrain SVM on current dataset
POST	`/api/predictions/predict-patient/`	Predict 1-year mortality for one patient
GET	`/api/predictions/metrics/`	Model performance metrics
GET	`/api/audit/`	Paginated audit log (admin only)
DELETE	`/api/audit/cleanup-old/`	Manual cleanup of old log entries

6.2 Authentication & Security

6.2.1 JWT Authentication

AI NéphroCare uses **JSON Web Tokens (JWT)** via the `djangorestframework-simplejwt` library, following an OAuth2-compatible flow. Figure 6.3 illustrates the full exchange between the browser, the Django backend, and PostgreSQL.

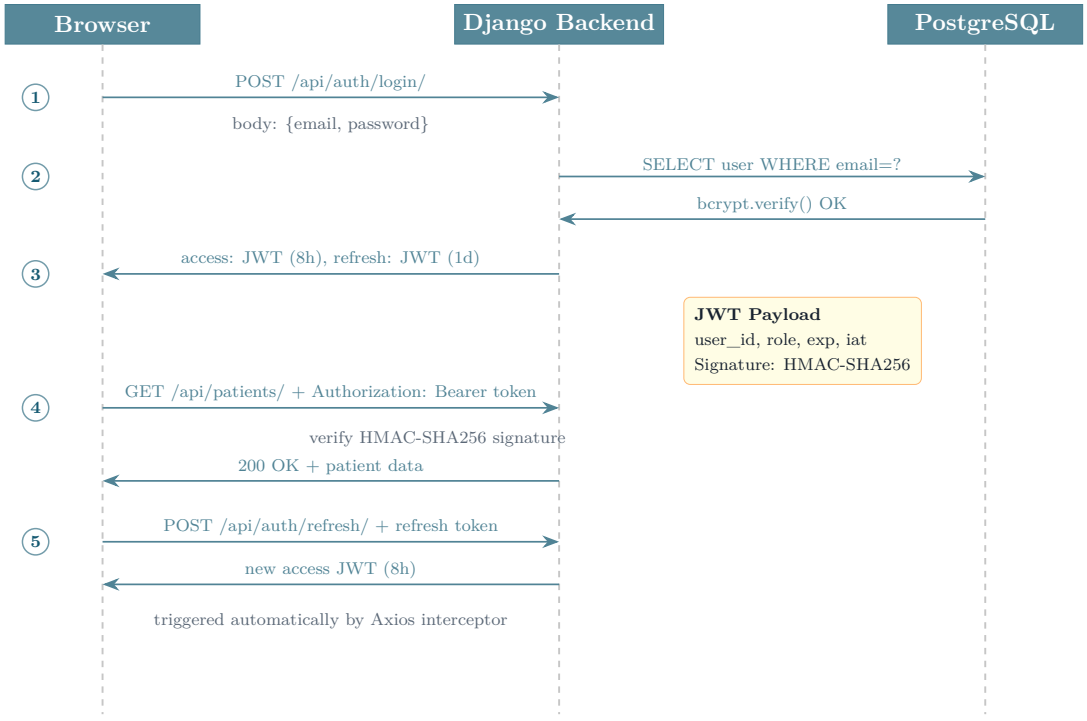


Figure 6.3 – JWT authentication flow: token issuance (Steps 1–3), authenticated request (Step 4), and automatic token refresh via Axios interceptor (Step 5).

The detailed steps are as follows:

1. The client sends `POST /api/auth/login/` with the user's email and password in JSON.
2. The server validates the credentials against the hashed (bcrypt) password stored in the database.
3. On success, the server returns a pair: `{"access": "<jwt>", "refresh": "<jwt>"}`.
4. For every subsequent request the client attaches the header `Authorization: Bearer <token>`.
5. When the access token expires (after **8 hours**), the Axios interceptor on the frontend automatically calls `POST /api/auth/refresh/` with the refresh token to obtain a new access token, with no visible interruption for the user.
6. The refresh token itself expires after **1 day**; after that the user must log in again.

The JWT payload encodes: `user_id`, `role` (string), `exp` (expiration Unix timestamp), and `iat` (issued-at timestamp). The server verifies the token signature using **HMAC-SHA256 (HS256)** with a 256-bit secret key stored in the environment variable `SECRET_KEY`, which is never committed to version control.

6.2.2 Role-Based Access Control (RBAC)

The platform defines **four roles** with hierarchical permissions. Each HTTP request passes through a DRF permission class that extracts the role from the JWT payload and enforces the access policy.

Table 6.3 – RBAC permission matrix

Role	Permitted actions	Restricted from
Admin	Full access: user CRUD, patient management, preprocessing, predictions, audit log	—
Head of Dept.	Approve preprocessing sessions, view all patients, run predictions, read audit log	User CRUD, audit cleanup
Resident	Create and edit patients, submit preprocessing sessions for approval, run predictions	User management, audit log, preprocessing approval
Professor	Read-only access to patients and prediction results	All write operations

RBAC is enforced at two levels: (1) **Backend** — each DRF view carries a `permission_classes` list that checks the role field from the JWT; (2) **Frontend** — the `ProtectedRoute` component reads the role from the `AuthContext` and redirects unauthorised users before the API call is even made.

6.2.3 Additional Security Measures

— **Password hashing**: all passwords are stored using Django's **PBKDF2+SHA256** hasher (the framework default, enforced via `AUTH_PASSWORD_VALIDATORS`), which applies 870 000 iterations making brute-force attacks computationally prohibitive. The `bcrypt` library is listed as a dependency for future migration to a bcrypt-based hasher in production.

— **Transport security**: the current deployment runs over HTTP on a local Docker network (`localhost:8000/3000`), which is acceptable for an intranet prototype. A production release would require TLS termination via a reverse proxy (e.g. Nginx + Let's Encrypt) and the Django settings `SECURE_SSL_REDIRECT`, `SESSION_COOKIE_SECURE`, and `CSRF_COOKIE_SECURE`. These hardening steps are identified as future work before any public-facing deployment.

— **CORS**: the `django-cors-headers` middleware restricts allowed origins to the declared frontend (`localhost:3000`), blocking cross-origin requests from any other domain.

— **Audit trail**: every state-changing action (create, update, delete on patients and users) is recorded in `AuditLog` with the actor's user ID, IP address, affected entity, and a text description of the change. The companion `HiddenAuditLog` table lets users hide individual log entries from their own view in the UI without deleting them from the database.

— **Input validation**: all incoming data passes through DRF serializers before reaching the model layer, enforcing field types, uniqueness constraints, and required/optional fields. Malformed requests are rejected with an HTTP 400 response identifying the offending field.

6.3 API Design

6.3.1 DRF Serializers and Validation

Django REST Framework **serializers** sit between the HTTP layer and the database layer. Every endpoint uses a dedicated serializer that validates field types and constraints (e.g. `EmailField` uniqueness, required vs. optional fields), handles nested representations (e.g. the patient serializer structures the different data sections into a consistent response), and enforces write-protection on computed fields such as `date_creation` and `utilisateur_saisie`.

Validation errors return HTTP **400 Bad Request** with a structured JSON body where each key is the offending field name and the value is a list of error messages. For example, submitting a duplicate email returns `{"email": ["This field must be unique."]}`.

6.3.2 Filtering and Search

The patient list endpoint (`GET /api/patients/`) returns a JSON object containing a `results` array and a `count` field. Filtering is implemented directly in the view layer via query parameters: `?search=` performs a full-text search across patient name, ID, and medical fields; `?sexe=`, `?age_min=`, `?age_max=`, `?statut_inclusion=`, and `?date_naissance=` allow targeted field-level filtering. All parameters can be combined freely in a single request.

6.3.3 Prediction Response Structure

The `POST /api/predictions/predict-patient/` endpoint assembles the patient's clinical features, runs them through the calibrated SVM pipeline, and returns a structured JSON response designed to be directly usable by clinicians:

- `risk_level` — categorical classification: *Faible* (<10 %), *Modéré* (10–29 %), or *Élevé* (≥29 %)
- `score` — mortality risk expressed as a percentage (0–100)
- `probability` — Platt-scaled probability of 1-year mortality (0–1), output of `CalibratedClassifierCV`
- `factors` — ordered list of the most influential clinical features with their SVM weight and direction
- `recommendation` — auto-generated clinical text summarising the risk level and key factors
- `features_used` — list of feature keys actually used in the prediction
- `n_features` — count of non-missing features used
- `model` — identifier of the SVM model file used, enabling audit reproducibility
- `doietal_risk_score` / `doietal_risk_category` — secondary risk score based on the Doi et al. clinical index

6.4 Frontend Architecture

6.4.1 React Structure

The React application is organized around:

— **Context API**: `AuthContext` (JWT state, login/logout)

— **Protected Routes**: `ProtectedRoute` component enforces RBAC at the routing level

— **Axios Instance**: Centralized HTTP client with JWT interceptor and error handling

6.4.2 Pages and Interfaces

Landing Page (`/`) The landing page is the public entry point of the platform, accessible without authentication. It presents the platform identity through a full-screen hero layout composed of a gradient background in deep blue tones, a large anatomical illustration of kidneys on the right side, and a concise value proposition on the left: the platform name *AI NéphroCare*, the tagline *"Bienvenue dans votre espace santé"*, a short description inviting users to open their secure account, and a **"Commencer"** call-to-action button that redirects to the login page.

Figure 6.4 – Landing page — public entry point of the AI NéphroCare platform

Login Page (`/login`) The login page is divided into two panels. The left panel displays the platform branding: the name *AI NéphroCare* in large pink typography over a dark blue background, accompanied by a 3D kidney illustration and the subtitle *"Une plateforme intelligente dédiée à la néphrologie, pensée pour vos patients et vos équipes."*. The right panel presents a minimal authentication form over a white background with floating decorative dots, containing:

— An **email** input field with envelope icon

— A **password** input field with padlock icon

— A **Connexion** submit button that initiates the JWT authentication flow (POST `/api/auth/login/`)

On successful authentication, the user is redirected to their role-appropriate dashboard. On failure, an error message is displayed inline without page reload.

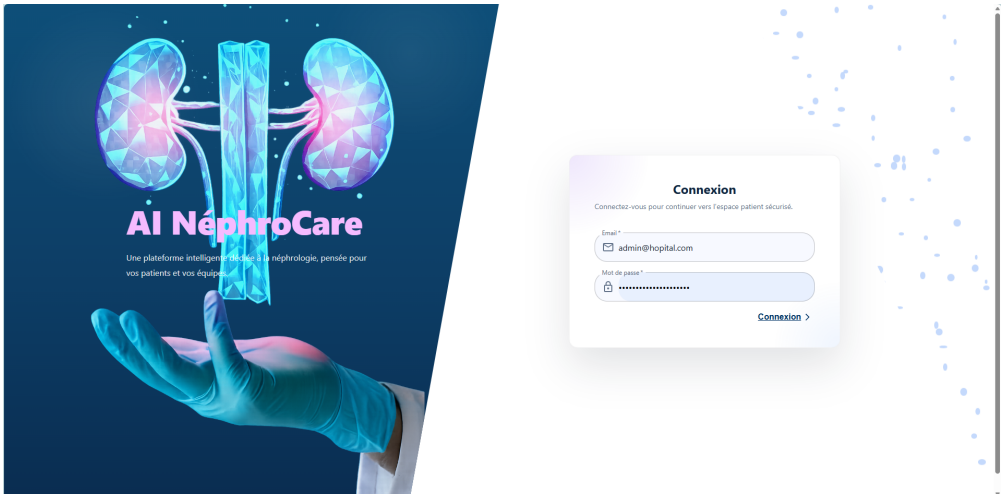


Figure 6.5 – Login page — split-panel design with branding (left) and JWT authentication form (right)

Navigation Sidebar Once authenticated, all pages share a persistent left sidebar (width: 248 px) that serves as the main navigation hub of the application. It is composed of:

— **Platform logo and branding** at the top (*AI NÉPHROCARE*), with a gradient gradient typography

— **Navigation card** with links to all main sections: Dashboard, Patients, Analyse IA, Monitor

— **User profile card** at the bottom showing the connected user's email and role badge

— **Notifications button** (visible to Head of Department and Super Admin only) with a badge counter for pending validation requests

— **Logout button** that clears the JWT tokens and redirects to the login page

Access to each section is controlled by the user's role (RBAC): navigation items are filtered at the routing level via the `ProtectedRoute` component, so unauthorised sections are never rendered.

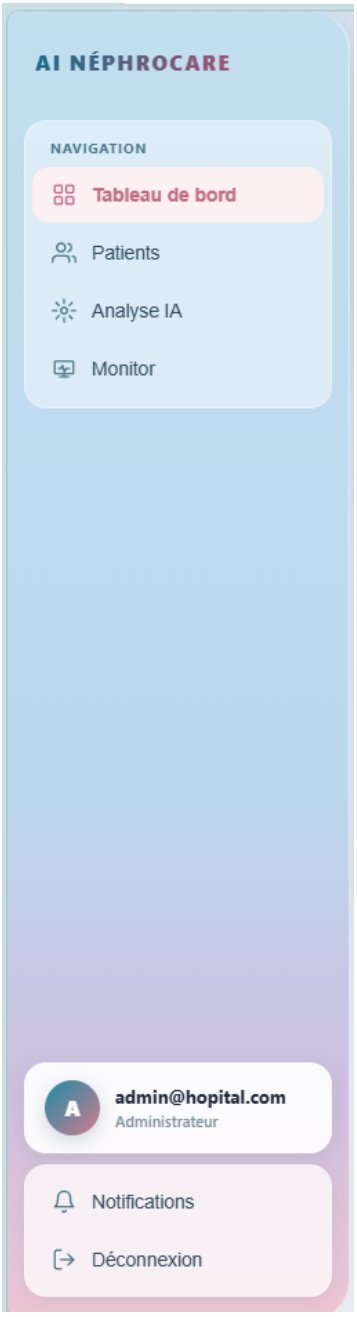


Figure 6.6 – AI NéphroCare navigation sidebar — persistent across all authenticated pages

♦ **Carnet numérique (global floating notepad).** All authenticated pages include a **floating action button** (bottom-right corner) that opens a persistent personal notepad called *Carnet numérique*. The notepad allows clinicians to write private reminders, observations, or annotations that are auto-saved at regular intervals and stored in their user profile (`personal_notes` field). The panel displays the last save time and offers an **Effacer** button to clear the content.

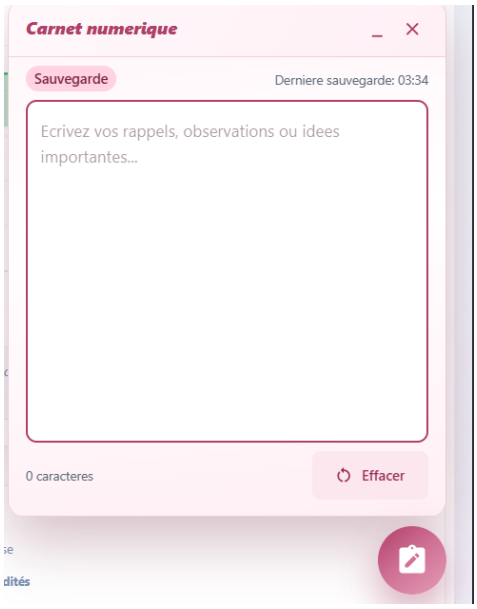


Figure 6.7 – Carnet numérique — persistent personal notepad accessible from any authenticated page via a floating action button

Dashboard (`/dashboard`) The dashboard is the first authenticated page. Its content is entirely **role-dependent**: Super Admin and Head of Department see an administrative dashboard focused on user governance, while Professor and Resident see a clinical dashboard focused on their own activity.

▶ Dashboard — Super Admin and Head of Department

The administrative dashboard is organised into two columns:

— **Left column** — Connected profile card (identity, role, email, telephone).

— **Right column** — User management area: KPI summary, user list with search, and CRUD panel.

♦ **KPI summary.** The hero banner displays **7 interactive KPI cards** for Super Admin (6 for Head of Department, excluding Super Admins): *Actifs*, *Inactifs*, *Super Admins*, *Heads of Dept.*, *Professors*, *Residents*, *Rôles*. Each card is **clickable**: clicking it filters the *Comptes gérés* list below to show only matching accounts (by status or role). An active filter is displayed as a labelled chip with a dismiss button. The text search bar operates **in parallel** with the KPI filter.

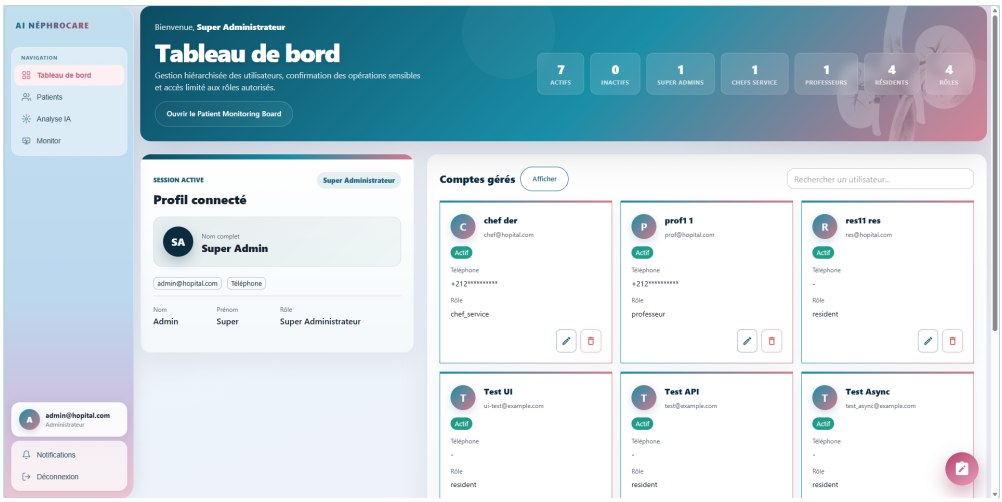


Figure 6.8 – Super Admin dashboard — 7 interactive KPI cards in the hero banner; clicking a card filters the account list

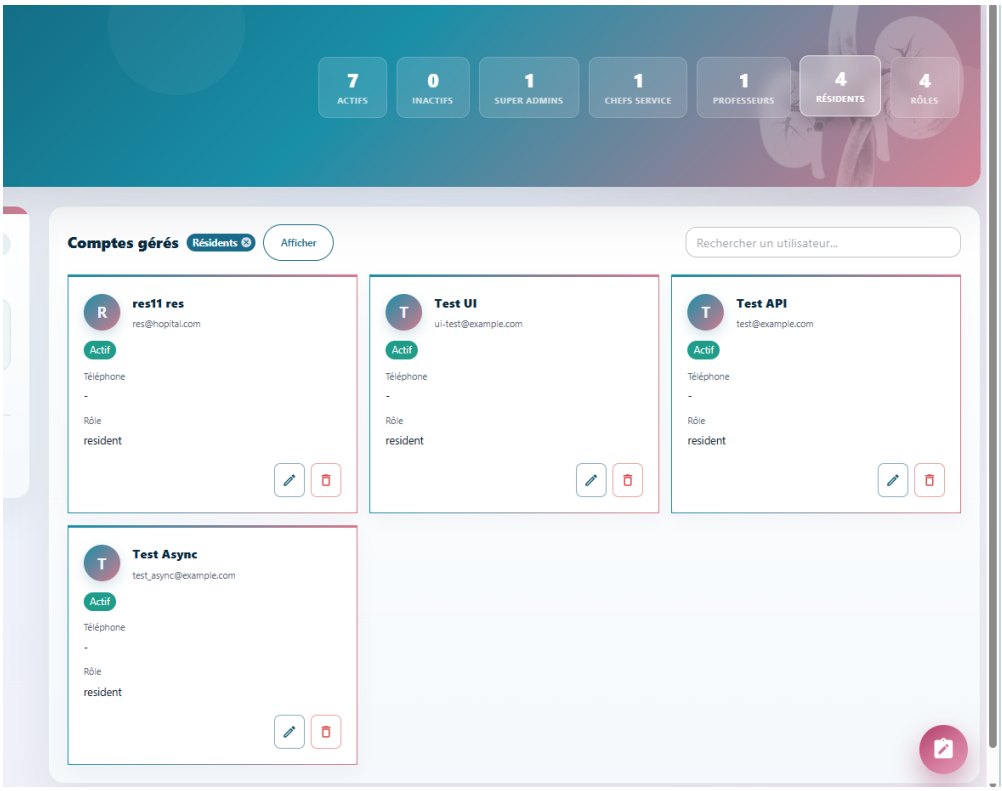


Figure 6.9 – KPI filter active — clicking *Professors* restricts the list and displays an active-filter chip

♦ **User Management (CRUD).** Clicking the *Afficher* toggle reveals the CRUD panel. The form supports:

— **Create**: email, telephone, last name, first name, role, status, initial password (no confirmation password required).

— **Edit**: pencil icon pre-fills the form; administrator must re-enter their **own password** before saving (`confirmation_password` validated server-side via `check_password()`).

— **Delete**: bin icon opens a dialog requiring the administrator's password before deletion.

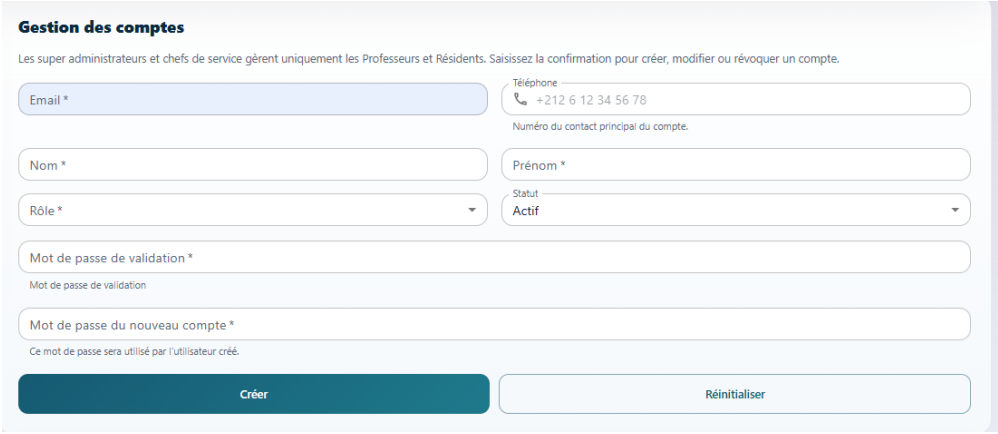


Figure 6.10 – User management CRUD panel — account creation and editing form

Figure 6.11 – Deletion confirmation dialog — password required before account removal

The **double-authentication** on write operations prevents unauthorised modifications and ensures every change is traceable in the audit log.

♦ **Import validation workflow.** When a Resident or Professor submits a preprocessed file for approval, the Head of Department receives a **notification badge** in the sidebar. Clicking **"Aperçu"** opens a preview modal showing file name, submitter, row/column counts, quality score, and the first 15 rows. From the modal, the Head of Department can **validate and integrate** (data inserted into the patient database) or close without action. A reject action is available via the detail endpoint with a mandatory reason.

Figure 6.12 – Import submission request — clinician view showing file pending review with status indicator

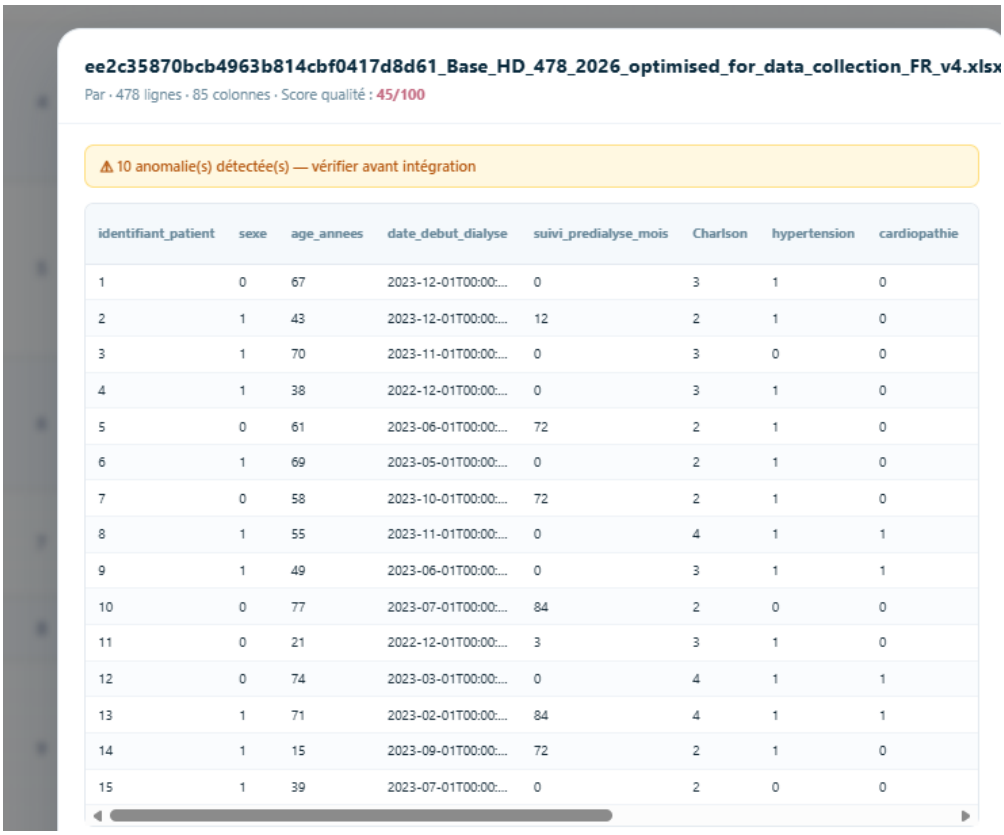


Figure 6.13 – Import validation preview — Head of Department reviews file metadata, row/column count, quality score and first rows before approving or rejecting

▸ Dashboard — Professor and Resident

Clinical users (Professor, Resident) see a dedicated dashboard with **no administrative panels**. The layout presents two side-by-side panels:

— **Left panel** — Connected profile card: avatar with initials, full name, role

badge, email, telephone.

— **Right panel** — **Activité récente**: a real-time feed of the last 10 actions performed by the connected user, fetched from `GET /api/audit/my-activity/`. Each entry shows a colour-coded indicator, a human-readable label, and a timestamp. The entries tracked include:

  — Patient added, modified, or deleted

  — Mortality prediction launched

  — Preprocessing file submitted for validation

  — Import approved or rejected by the chef

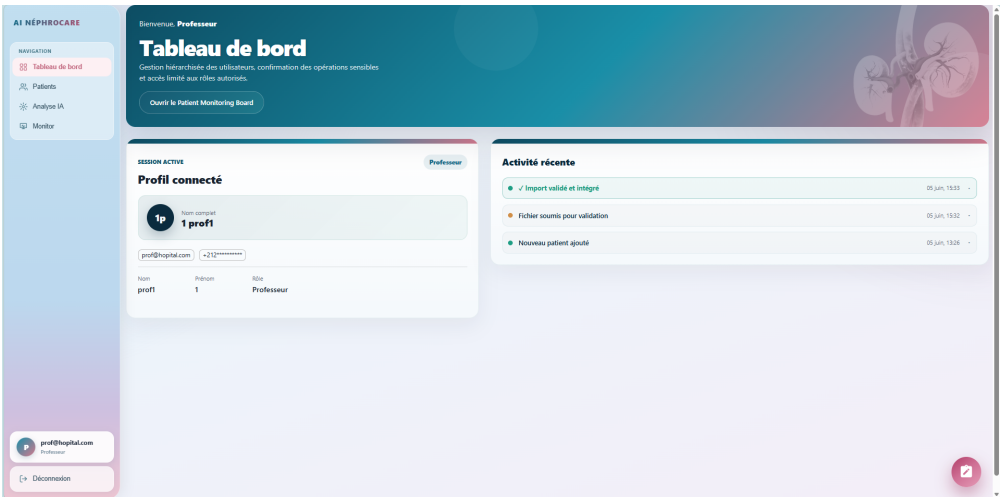


Figure 6.14 – Professor/Resident dashboard — connected profile (left) and recent activity feed (right)

♦ **Click-to-expand interaction.** Clicking any activity entry **expands** it inline to reveal: the full action detail (e.g. patient identifier), any comment left by the reviewer, the entity reference, and a *"Voir →"* shortcut to the relevant section (patient list, AI model, or preprocessing tab). Clicking again collapses the entry.

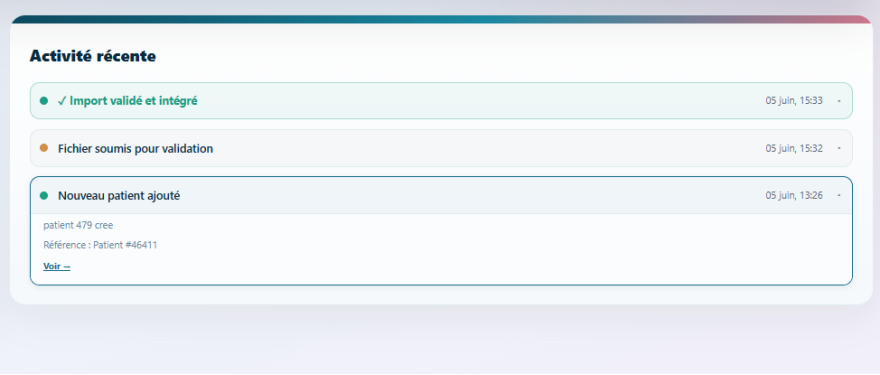


Figure 6.15 – Activity feed entry expanded — full action detail, reviewer comment, and direct navigation shortcut

♦ **Import decision notifications.** When the Head of Department or Super Admin approves or rejects a submitted import, the backend writes an `AuditLog`

entry in the **submitter's** log. The entry appears with a **distinct visual style**:

— **Approved**: green border and bold label *"✓ Import validé et intégré"* with the reviewer's name and comment.

— **Rejected**: pink border and bold label *"× Import refusé"* with the rejection reason.

This ensures the clinician is immediately informed of the outcome of their submission without requiring a separate notification system.

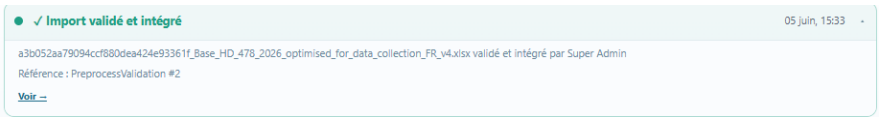


Figure 6.16 – Import decision notification in the activity feed — green entry for approval, pink entry for rejection, both displaying the reviewer's comment

Patient Management (`/patients`) The patient management module is organized into three tabs presented in the following logical order: *Preprocessing*, *Data Management*, and *Analysis*.

▶ Tab 1 — Preprocessing Interface

The preprocessing tab is the entry point for data quality control. It guides the user through a four-step workflow: file upload, AI analysis, correction review, and integration. The platform uses the **Qwen2.5:14B** large language model deployed locally via **Ollama**, chosen for its strong reasoning on structured clinical tabular data, its on-premises deployment (patient data never leaves the hospital network), and its 14B parameters that allow contextualised corrections with medical justification.

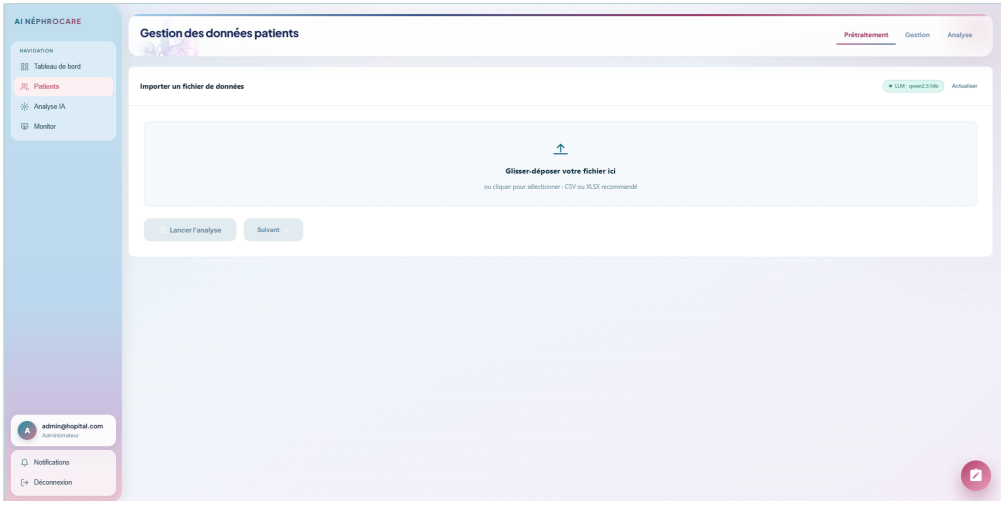


Figure 6.17 – Preprocessing interface — general view before file upload

♦ **Step 1 — File upload.** The interface presents a **drag-and-drop upload zone** that accepts `.xlsx` and `.csv` files. Upon file selection, the platform validates the format, displays the file name, and enables the analysis launch button.

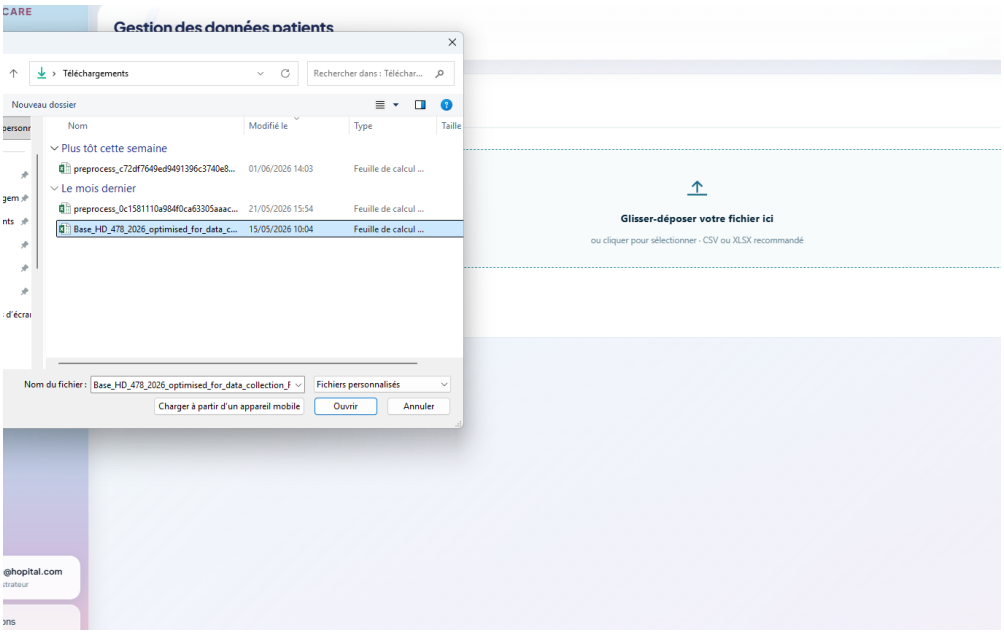


Figure 6.18 – Step 1 — file upload zone with Excel/CSV file selected, ready for LLM analysis

♦ **Step 2 — Analysis pipeline (11 stages).** Clicking **"Lancer l'analyse LLM"** triggers an asynchronous **Celery** task (`analyze_preprocess_async`) that orchestrates the following eleven-stage pipeline. Each stage updates the progress bar and status message visible on the frontend.

Table 6.4 maps each backend stage to the label displayed in the interface progress bar.

Table 6.4 – Preprocessing pipeline stages and corresponding interface labels

#	Backend function	Role	Interface label
1	`_read_uploaded_dataframe`	File parsing	Lecture
2	`_build_technical_profile`	Statistical scan	Analyse des données
3	`_build_preprocess_chunks`	RAG chunking	Préparation
4	`_build_retrieval_context`	ChromaDB retrieval	Références médicales
5	`_determine_preprocess_route`	LLM routing	Initialisation IA
6	`_call_ollama_qwen_analysis`	LLM batch analysis	Détection des anomalies
7	`_apply_llm_correction_plan`	Apply corrections	Corrections automatiques
8	`_run_bio_value-_correction_pass`	LOINC/KB bio pass	Vérification biologique
9	`_run_llm_flagged_corrections`	Flagged value triage	Vérification clinique
10	`_run_model_imputation`	KNN imputation	Complétion des données
11	`_build_preprocess_report`	Quality report	Rapport final

1. **File reading.** The uploaded Excel or CSV file is parsed into a Pandas DataFrame. All column names and data types are preserved as-is from the source file.
2. **Technical profiling.** Python performs a full statistical scan of **every**

column: data types, missing value rates, unique value counts, outlier detection, and anomaly candidate identification (sentinel values, biologically impossible values, Excel artefacts, future dates). This stage produces a `technical_profile` dictionary used to ground all subsequent stages without any LLM call.

3. **Semantic chunking for RAG.** The dataset is split into semantic row-chunks by clinical domain (11 sections: demography, IRC, comorbidities, biology, dialysis, complications, etc.), capping at 40 rows per chunk while preserving patient coherence. These chunks are not sent to the LLM; they serve exclusively as retrieval units for ChromaDB.
4. **RAG retrieval.** The chunks are used to query ChromaDB, which stores embeddings of nephrology clinical guidelines, LOINC reference ranges, and dialysis protocol summaries. The top-k most relevant document fragments are injected into the LLM prompt as medical grounding context.
5. **Route determination.** The platform evaluates dataset complexity and selects the analysis mode: *LLM-only* (default), *deterministic* (Python rules only, no LLM), or *hybrid*. In the deployed configuration the route is always LLM with `qwen2.5:14b` as primary model.
6. **LLM batch analysis — Pass 1.** All columns are split into batches of 5 (`LLM_BATCH_SIZE=5`). For each batch, all row values of the 5 columns are serialised and sent to Qwen2.5:14B with the RAG context. The model returns a structured `correction_plan` (JSON) specifying type casts, date parsing, missing value fills, and flagged anomalies. Results from all batches are merged into a single correction plan. For a 90-column dataset this produces at most $\lceil 90/5 \rceil = 18$ LLM calls.
7. **Apply correction plan.** The merged `correction_plan` is applied to the full DataFrame: numeric type casts, date standardisation, boolean normalisation, fill-missing values, and Excel artefact removal.
8. **Biological value correction pass.** For each column matched to a LOINC code (primary) or the medical knowledge base (fallback), the platform asks the LLM to identify aberrant unique values based on authoritative clinical reference ranges, then corrects them across all rows.
9. **LLM flagged corrections.** Columns flagged in Pass 1 but not covered by the bio KB are re-analysed. Each suspicious value is classified as *error* (auto-corrected), *extreme_real* (extreme but clinically plausible — kept with flag), or *uncertain* (kept with flag). Only confirmed errors are automatically corrected.
10. **KNN imputation.** Values nullified by the correction passes are re-estimated

using K-Nearest Neighbours ($k = 5$), inferring each missing value from the 5 most similar patients across all available clinical features.

11. **Report generation.** A final quality report is assembled: quality score (0–100), total corrections, column-level issues, LLM confidence contract, and pipeline metadata. The session is persisted and the frontend is notified that the analysis is complete.

▶ Python Deterministic Pre-passes

In addition to the LLM passes, the `_apply_llm_correction_plan()` function runs a set of **deterministic Python pre-passes** before the LLM correction plan is applied. These handle unambiguous error patterns that do not require LLM interpretation:

— **Excel serial-date artefacts**: values such as `1899-12-30` or `1900-01-01` (produced by pandas when reading Excel serial date 0) are unconditionally replaced with `null`.

— **`"X"` values in numeric columns**: in columns where at least 60% of non-null values are numeric, any cell containing `"X"`, `"x"`, or `"*"` is replaced with `null` and subsequently imputed by KNN. Text/categorical columns are left untouched.

— **Mixed semicolon artefacts**: values of the form `1;(1/2024)` are detected by a dedicated regex and flagged for the LLM to classify and correct.

— **Decimal comma normalisation**: French-locale numbers such as `1,5` or `2 234,56` in object columns are converted to their dot-decimal equivalents (`1.5`, `2234.56`).

These pre-passes ensure that obvious structural errors are removed *before* the LLM analysis, reducing noise in the LLM prompt and preventing trivially incorrect values from being classified as *uncertain*.

The interface displays a **progress view** (MUI `LinearProgress`) updated after each stage, with a status message tracking the current step and a **cancel button** to interrupt the Celery task at any point. The frontend polls `GET /api/patients/preprocess/{session_id}/status/` every 3 seconds via a `setInterval` hook.

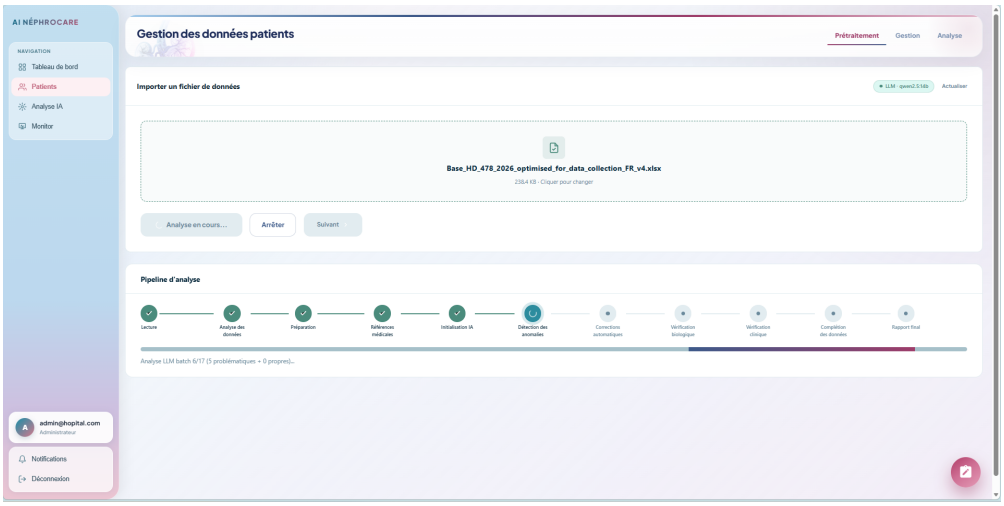


Figure 6.19 – Step 2 — LLM analysis in progress, real-time progress bar with current processing message

♦ **Step 3 — LLM correction review table.** Once the analysis completes, the interface displays a **results summary** followed by the **correction review table**. The summary section contains:

— A **Quality Score ring** (custom SVG `ScoreRing` component, range 0–100): green if ≥ 80, orange if ≥ 50, red below 50

— Three **KPI stat cards** (custom `StatCard` component, MUI `Paper`):

  — **Lignes analysées** — total number of rows processed by the LLM

  — **Cellules corrigées** — number of individual cell values automatically corrected

  — **Colonnes suspectes** — number of columns flagged as anomalous but without an automatic correction (manual review recommended)

The correction review table is organised in two **tabs** (MUI `Tabs` / `Tab`):

1. **Cellules corrigées** — lists every automatic correction with the following columns:

   — **Ligne** — row index in the source file

   — **ID Patient** — patient identifier

   — **Colonne** — clinical field name

   — **Valeur originale** — raw value from the imported file

   — **Valeur corrigée** — value proposed by the LLM (clickable: the user can **edit it inline** to override the suggestion)

   — **Type** — correction category (type cast, outlier, KNN imputation, boolean normalisation, Excel artefact)

   — **Justification** — medical explanation generated by Qwen2.5:14B

Each corrected cell is rendered as a **MUI `Chip`** colour-coded by correction type (green for corrected, pink for KNN). Clicking a cell activates an **inline text field** to manually override the LLM's suggestion; the override is persisted via `POST /api/patients/preprocess/{session}/apply-overrides/`.

2. **Colonnes suspectes** — lists columns flagged by the LLM as potentially erroneous but for which no automatic correction was applied. Each row shows the column name, anomaly category, and LLM justification so the user can decide on a manual correction.

Below the table, two **export buttons** allow the user to download the data as an `.xlsx` file before integration:

— **Exporter corrigé (.xlsx)** — downloads the corrected version. The file is generated server-side by the `PatientPreprocessExportView` using the `openpyxl` library and contains **two sheets**:

— **Sheet 1 — "Données"**: the full dataset with all corrections applied. Every cell whose value was changed by the LLM is **highlighted in pale yellow** (`#FFF2CC`) with a **bold orange font** (`#B45309`), making modified cells immediately visible. The header row uses a dark navy background (`#0A2B3E`) with white bold text.

— **Sheet 2 — "Corrections"**: a summary table listing every changed cell with the columns *N° Ligne*, *ID Patient*, *Colonne*, *Valeur originale*, *Valeur corrigée*, *Type*, and *Justification* — identical structure to the on-screen review table.

— **Exporter original (.xlsx)** — downloads the raw file as imported, with only the header row styled, and no cell highlighting.

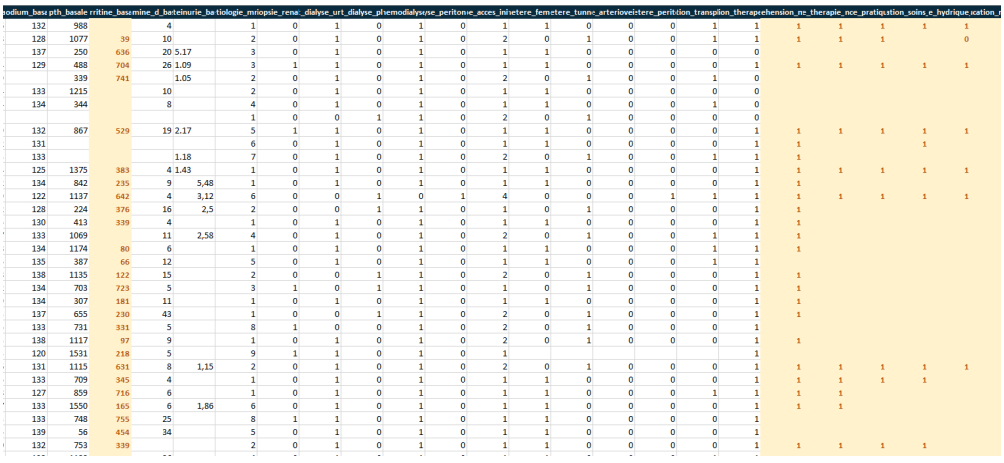
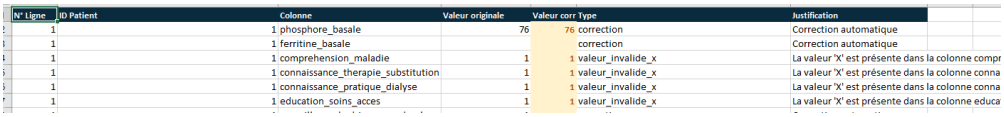


Figure 6.20 – Feuille *Données* du fichier `.xlsx` exporté — les cellules modifiées par le LLM sont surlignées en jaune pâle (`#FFF2CC`) avec une police orange grasse, générées par `openpyxl`



N° Ligne	ID Patient	Colonne	Valeur originale	Valeur corr	Type	Justification
1		1 phosphore_basale	76	76	correction	Correction automatique
1		1 ferritine_basale			correction	Correction automatique
1		1 comprehension_maladie	1	1	valeur_invalide_x	La valeur 'X' est présente dans la colonne compr
1		1 connaissance_therapie_substitution	1	1	valeur_invalide_x	La valeur 'X' est présente dans la colonne conna
1		1 connaissance_pratique_dialyse	1	1	valeur_invalide_x	La valeur 'X' est présente dans la colonne conna
1		1 education_soins_acces	1	1	valeur_invalide_x	La valeur 'X' est présente dans la colonne educa

Figure 6.21 – Feuille *Corrections* du fichier `.xlsx` exporté — tableau récapitulatif listant chaque cellule modifiée avec sa valeur originale, sa valeur corrigée, le type de correction et la justification médicale

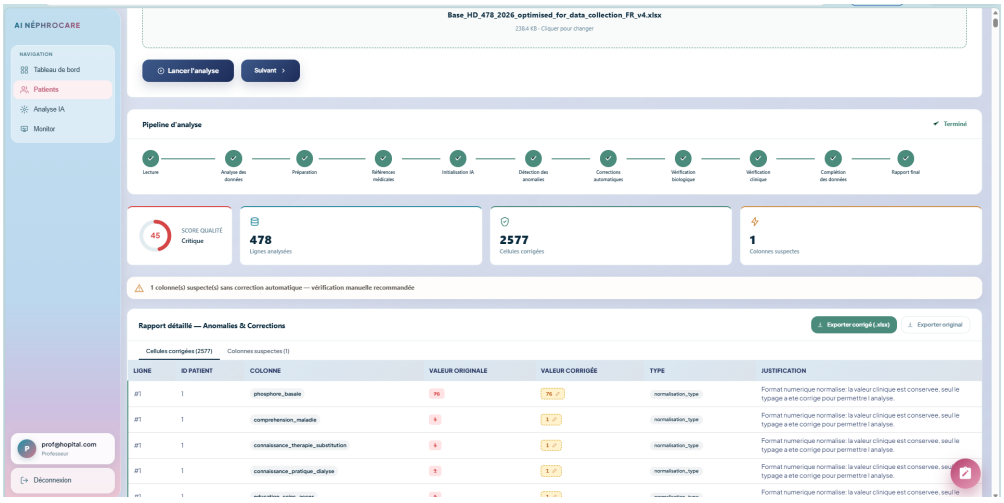


Figure 6.22 – Step 3 — LLM correction review table: quality score, KPI cards, and cell-level corrections with medical justifications

♦ **Step 4 — Integration.** Once the user has reviewed and optionally overridden corrections, they click **"Intégrer dans la plateforme"** to persist the dataset into the platform database. Before confirming, the user must explicitly **choose which version to integrate** via a radio-button selector:

— **Version corrigée** (recommended) — the dataset with all LLM corrections, bio-value corrections, and KNN imputation applied. This is the version that will improve the downstream SVM prediction accuracy.

— **Version originale** — the raw file as uploaded, without any modifications. This option is available if the clinician considers the original data reliable and does not wish to apply the automated corrections.

This explicit choice guarantees that no data modification reaches the database without the user's deliberate consent, in line with the platform's principle of **human-in-the-loop** clinical validation.

Figure 6.23 – Integration version selector — the user chooses between the corrected version (recommended) and the original file before persisting data to the database

Once the version is selected and confirmed, the integration endpoint triggers a **column mapping process** that translates the source file's column names into the platform's standardised schema:

— Each source column is compared against a configurable mapping dictionary (`column_mapping.json`). If a match is found (e.g. *"Albumine basale"* → `albumine_basale`), the value is stored in the corresponding field of the patient's structured clinical record.

— Columns that belong to a known clinical section (demography, biology, dialysis, comorbidities, etc.) are stored in the relevant JSONB section of the `Patient` model.

— Columns that do **not** match any predefined field are stored as **dynamic columns** in the `extra_data` JSONB field, preserving institution-specific variables without requiring any schema migration.

A success notification confirms the integration, and the system transitions automatically to the **Data Management tab** where the imported patients are immediately visible.

▶ Tab 2 — Data Management

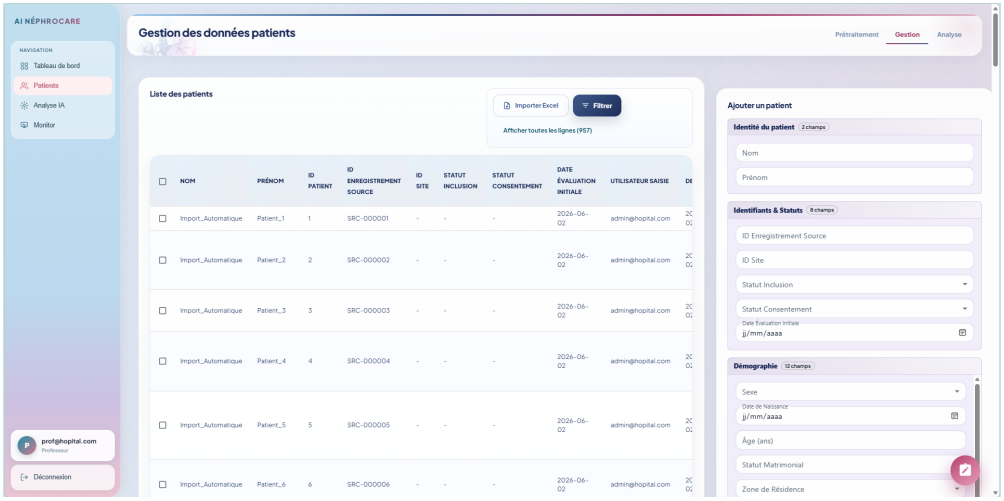


Figure 6.24 – Data management interface — searchable patient table with dynamic column support

The data management tab presents a **searchable, filterable, and sortable patient data table**. After a preprocessed file is validated and integrated, its columns are automatically **mapped to the platform's fixed column structure**:

- Each column in the imported file is matched to a predefined clinical field (e.g., "Albumine basale" in the file → `albumine_basale` in the database schema)
- Columns that match known field names are stored in their corresponding structured sections
- Columns that **do not match** any predefined field are stored in the `extra_data` JSONB field as **dynamic columns**

Dynamic Columns: The platform supports columns that are not part of the fixed schema. These "dynamic columns" appear as additional columns in the patient table, are fully searchable and filterable, and are preserved across imports. This mechanism allows the platform to accommodate institution-specific variables without modifying the database schema.

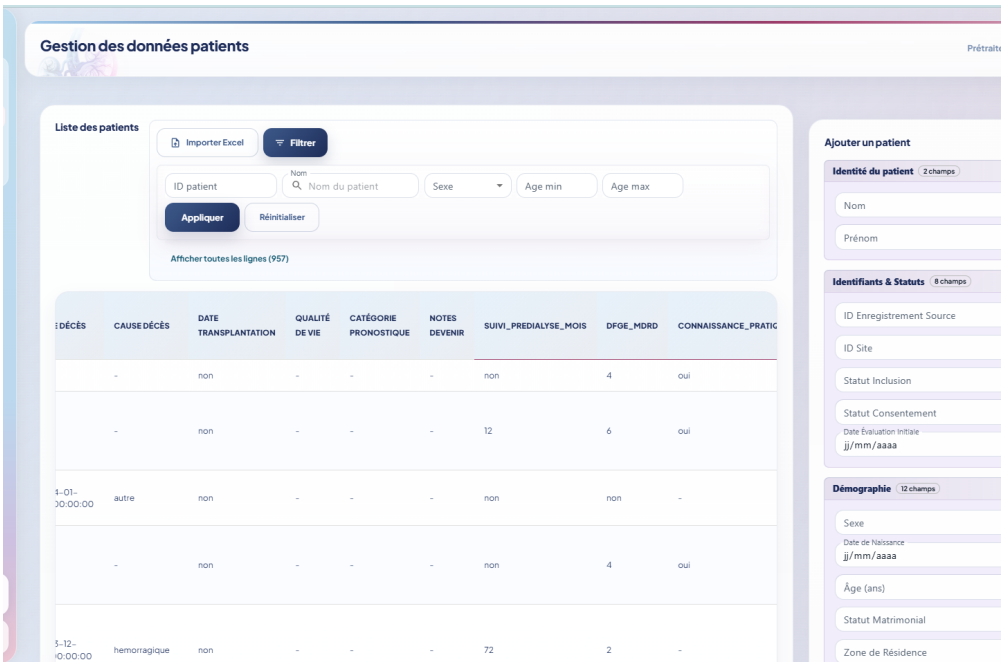


Figure 6.25 – Dynamic column support — institution-specific variables displayed as regular table columns alongside the fixed schema

The data management interface provides filtering, search, and column visibility controls. A **search bar** performs full-text search across all patient fields, a **column visibility toggle** shows or hides specific columns, and a **filter panel** allows filtering by any field value.

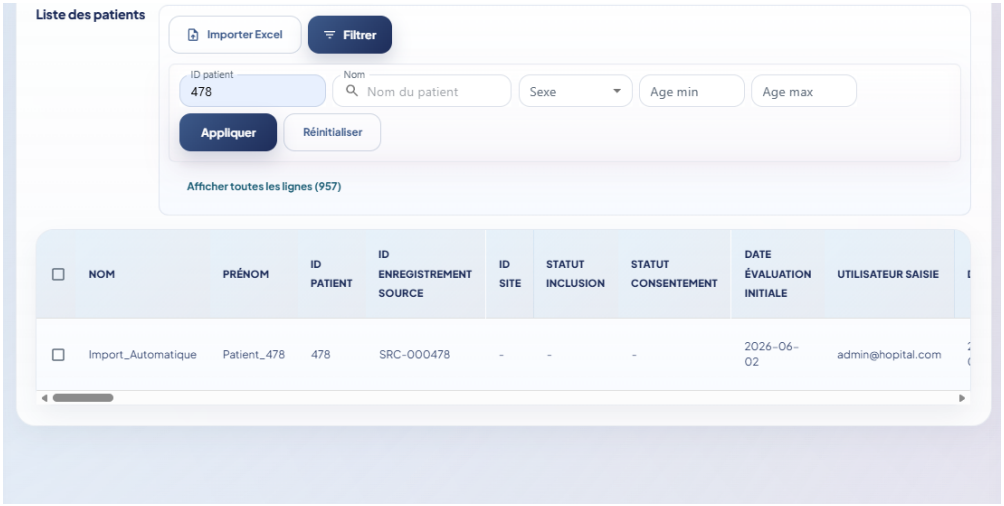


Figure 6.26 – Data management filter panel — multi-field filtering with active filter indicators

Patient records can be created, modified, and deleted through dedicated actions: an **Add patient** button opens a blank entry form, an **Edit** button opens the full 160+ field record for inline modification, and a **Delete** button removes the record after confirmation.

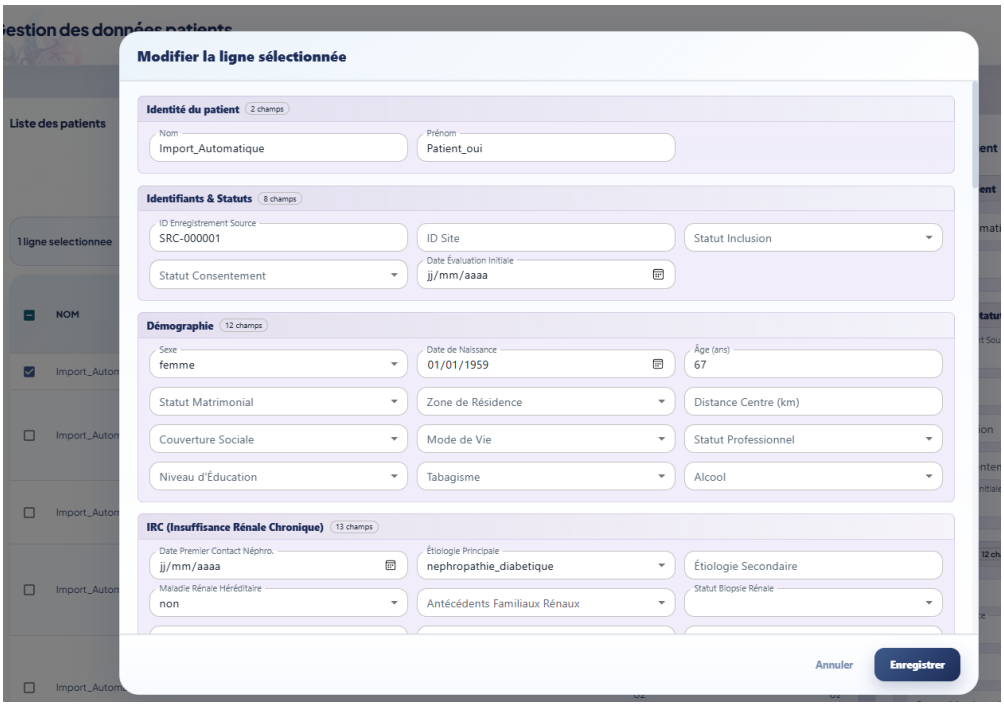


Figure 6.27 – Patient record edit form — all 160+ clinical variables organized in collapsible sections

Bulk data operations are supported via an **Import** button (Excel/CSV upload with column mapping) and an **Export** button (downloads all visible data as .xlsx). Dynamic columns imported from external files are listed in a dedicated management panel where each column shows its source file and can be individually removed.

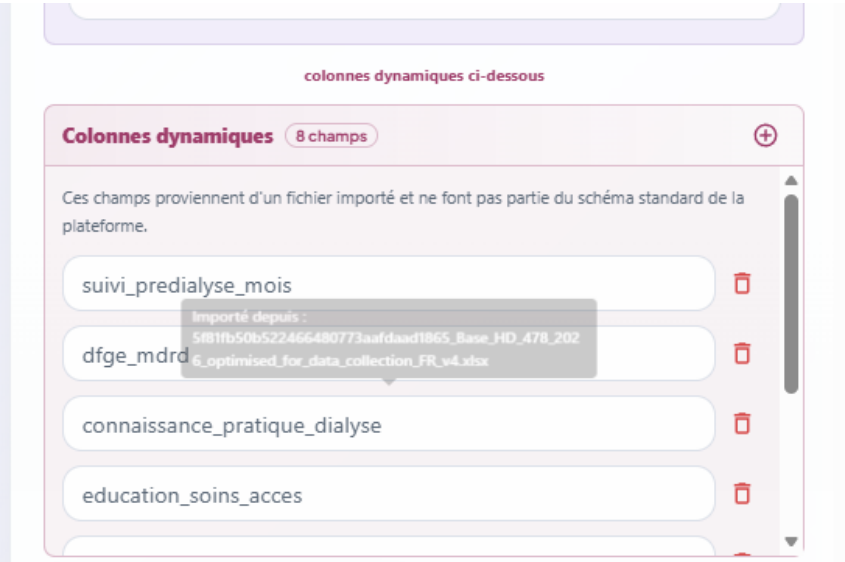


Figure 6.28 – Dynamic columns management — list of active dynamic columns imported from Excel/CSV with their source file tooltip and individual delete action

▶ Tab 3 — Analysis (Statistics)

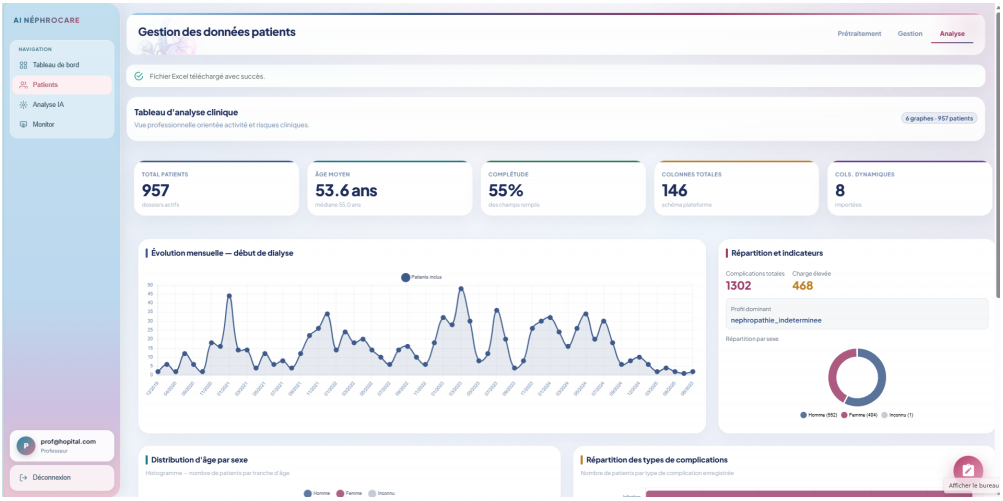


Figure 6.29 – Patient analysis interface — epidemiological and clinical statistics charts

The analysis tab provides a set of interactive statistical charts generated automatically from the patient database. These charts offer an epidemiological overview of the cohort and support clinical research and reporting. [See Section 6.5 for chart descriptions.]

AI Model (`/modele-ai`) The AI Model section is organized into three tabs: *Analysis Dashboard*, *Patients*, and *Prediction*.

▶ Tab 1 — Analysis Dashboard

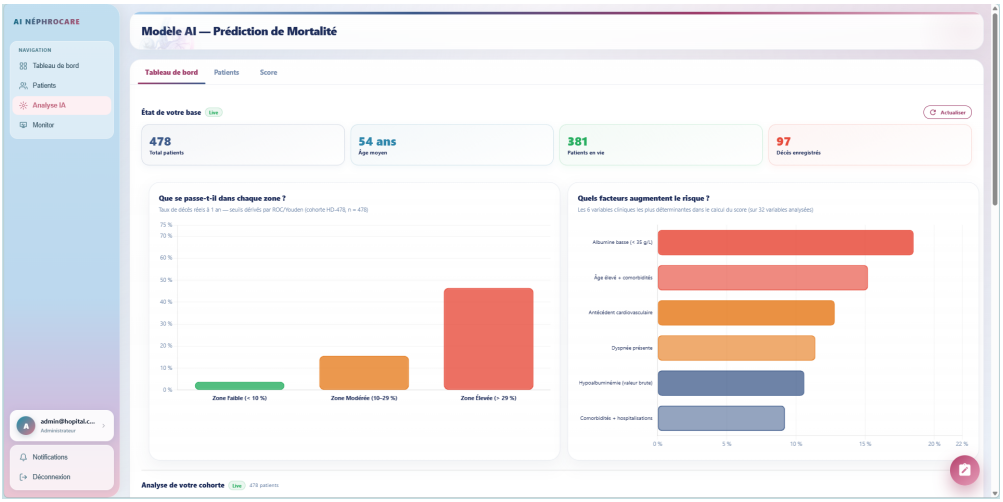


Figure 6.30 – AI Model analysis dashboard — SVM performance metrics and risk zone overview

The analysis dashboard provides a global overview of the AI model's performance and the current state of predictions in the cohort. It displays:

— **Model performance KPIs**: AUC-ROC, PR-AUC, F1-score, and Brier Score displayed as prominently colored cards

— **Risk zone distribution chart**: pie/doughnut chart showing the proportion of patients in Faible, Modérée, and Élevée risk zones

— **Model version and training date**: metadata on the currently deployed SVM model

— **Clinical interpretation guide**: brief description of each risk zone and its recommended clinical action

▶ Tab 2 — Patients

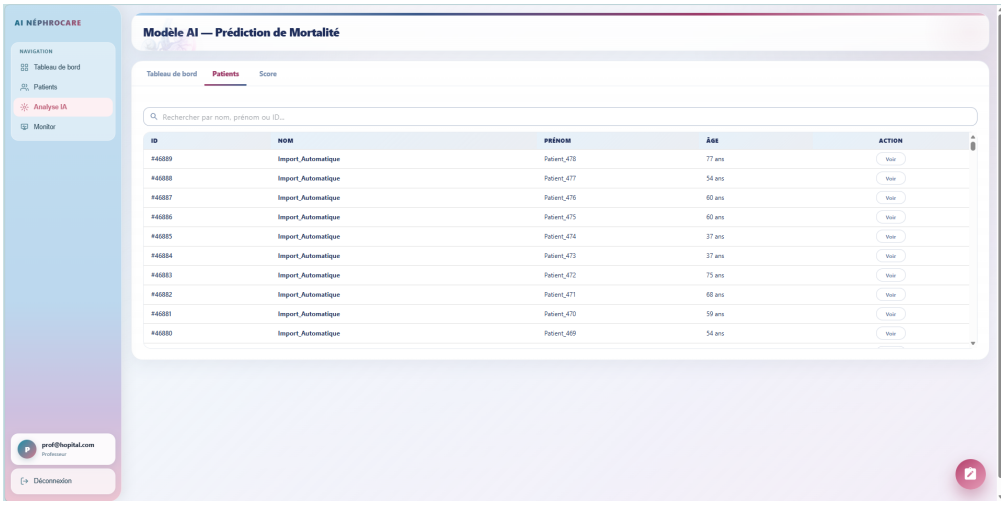


Figure 6.31 – AI Model patient list — each row shows patient summary and a Predict action button

The Patients tab displays the full list of patients registered in the platform. For each patient, the clinician can:

1. **Select a patient** from the list to open their complete clinical dossier
2. View the **patient dossier**: a detailed form showing all 160+ variables organized in the 11 clinical sections (Demographics, CKD history, Comorbidities, Biology, Dialysis, etc.)
3. From the dossier, choose one of two actions:
   — **Modify values and launch a simulation**: The clinician can modify any clinical parameter (e.g., reduce albumin level, change dialysis frequency) and click **"Run Simulation"**. The simulation result — showing how the predicted mortality risk changes with the modified values — is displayed **directly within the same Patients tab**, allowing immediate comparison without leaving the current view.

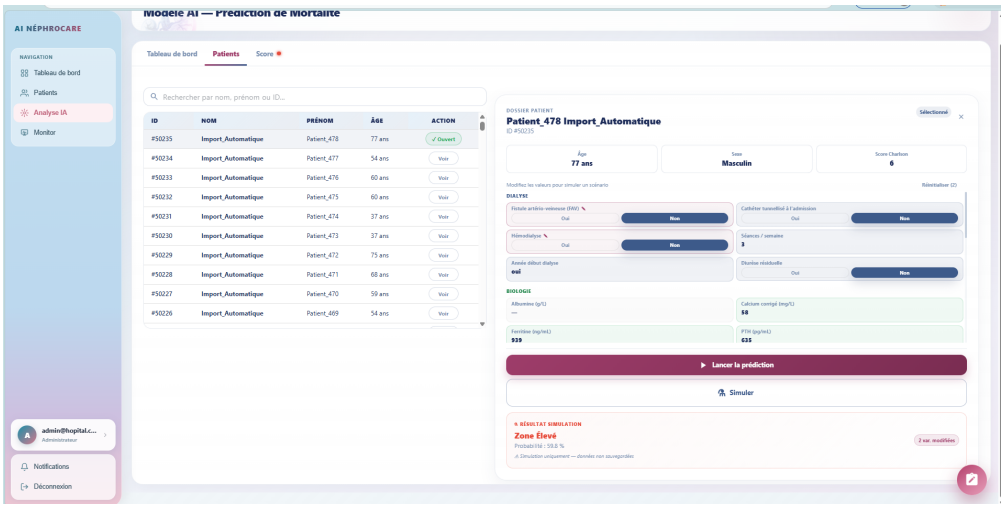


Figure 6.32 – Clinical simulation interface — the clinician modifies patient parameters and immediately sees the updated mortality risk prediction

— **Launch prediction**: Using the saved clinical values already stored in the platform, click **"Predict"** to generate the official mortality risk prediction. The prediction result is displayed in the **Prediction tab** (Tab 3).

▶ Tab 3 — Prediction

[Screenshot: Prediction tab — risk gauge, probability, top-5 factors, recommendation]

Figure 6.33 – AI prediction result interface — risk zone gauge, calibrated probability, contributing factors, and clinical recommendation

The Prediction tab displays the complete mortality risk assessment for the selected patient:

— **Risk gauge**: A color-coded arc gauge (green = Faible, orange = Modérée, red = Élevée) showing the calibrated probability visually

— **Calibrated probability**: The exact one-year mortality probability (e.g., "47.1% estimated 1-year mortality risk")

— **Relative risk badge**: The risk relative to the cohort baseline (e.g., ×10.9)

— **Top-5 contributing factors**: A ranked list of the five clinical variables most influencing the prediction, with their SVM feature weights displayed as horizontal progress bars. Each factor is labeled with its direction (risk-increasing or risk-decreasing).

— **Clinical recommendation**: A tailored text recommendation adapted to the predicted risk zone, specifying the recommended follow-up frequency and clinical actions.

— **Patient information summary**: Key clinical indicators displayed alongside the prediction for context.

Monitoring (`/monitor`) The Patient Monitoring Board provides longitudinal follow-up of individual hemodialysis patients. The interface is split into two panels:

- **Left panel — Patient list**: A searchable list of all patients in the database. The search bar accepts full-text queries across patient identifiers and names. Clicking a patient selects them and loads their complete monitoring view in the right panel.
- **Right panel — Clinical timeline**: An interactive chronological timeline of all clinical events recorded for the selected patient, fetched from the audit log and prediction history. Events are categorized and color-coded by type:
  - **Modifications** (patient data edits) — field-level old/new value diff, displayed as structured change entries
  - **Predictions** — each mortality risk prediction with its score, risk zone, relative risk, and top contributing factors
  - **Imports** — preprocessing file validations, approval or rejection events

For each prediction event in the timeline, the panel displays:

- The calibrated probability and risk zone badge (Faible / Modérée / Élevée)
- The relative risk multiplier (×RR vs. cohort baseline)
- The top-5 contributing clinical factors with their SVM weights
- The clinical recommendation text associated with the predicted zone

The most recent prediction is also highlighted at the top of the right panel as a **latest prediction summary card**. Date/time values are localized according to the active platform language (`fr-FR` or `en-US`).

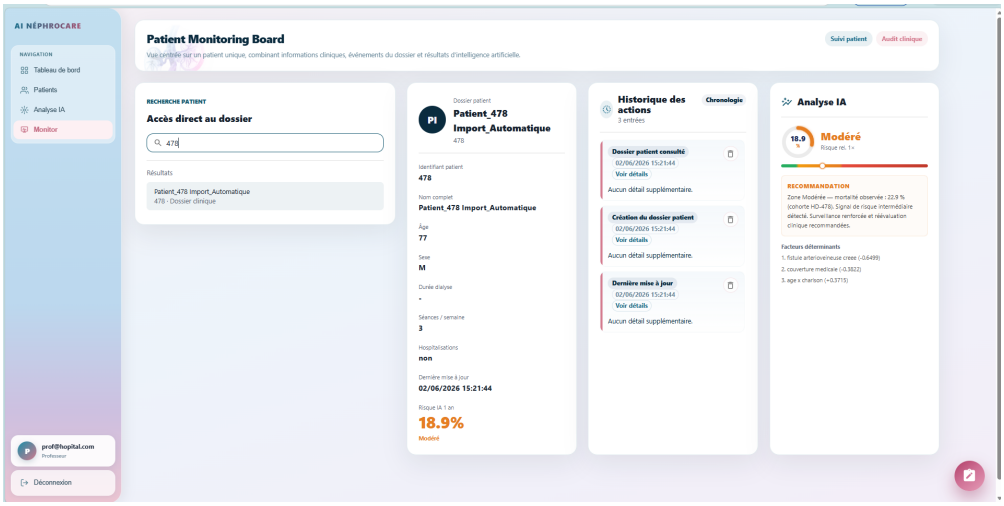


Figure 6.34 – Patient monitoring board — patient list (left), clinical timeline with prediction history and field-level audit trail (right)

Profile (`/profile`) The profile page allows each authenticated user to manage their personal settings. It is organized into three panels:

— **User information panel**: Displays the user's avatar (initials-based), full name, role badge (color-coded by hierarchy level), email address, and telephone number.

— **Password management**: A secure form allowing the user to change their password. The user must supply their current password for identity confirmation (`POST /api/auth/change-password/`), followed by the new password (minimum 8 characters) and a confirmation field. The form validates that the new password differs from the current one and that both new-password fields match before submitting.

— **Language preference**: A bilingual toggle allowing the user to switch the entire platform interface between **Français** and **English**. The preference is persisted to `localStorage` (`app_language` key) via the global `LanguageContext`, which provides translated labels for all UI strings throughout the application. On the next session, the preferred language is automatically restored.

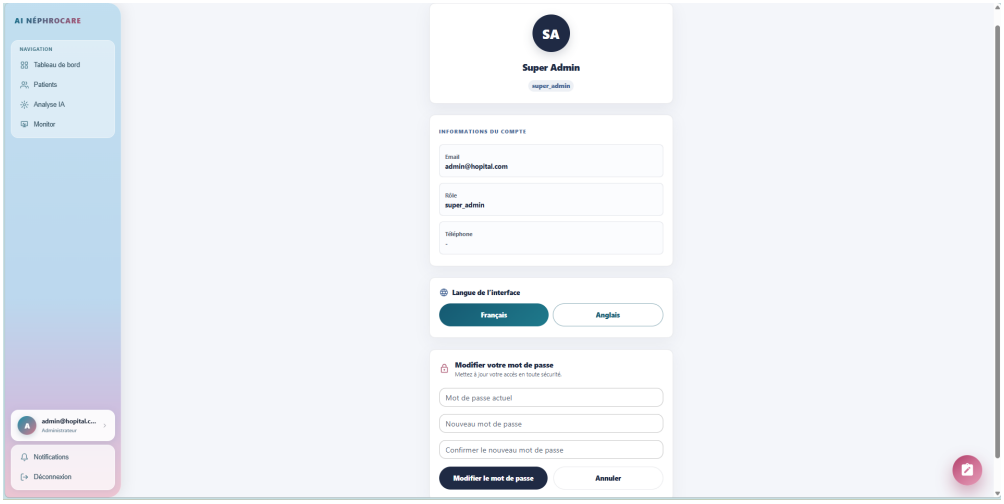


Figure 6.35 – Profile page — user information panel, password change form, and bilingual language selector (Français / English)

6.5 Data Visualization

6.5.1 Clinical Analytics Charts (Patient Management — Analysis Tab)

The analysis tab of the Patient Management module provides six interactive charts, each dynamically generated from the current patient database via the flat endpoint (`GET /api/patients/flat/`).

▸ Monthly Dialysis Initiation Trend

Figure 6.36 – Monthly dialysis initiation trend — number of new patients starting hemodialysis per month

This line chart displays the number of patients who started hemodialysis treatment each month over the observation period (2020–2024). It allows clinical teams to identify seasonal patterns, detect increases in ESRD incidence, and plan resource allocation accordingly.

▸ Sex Distribution and Key KPIs

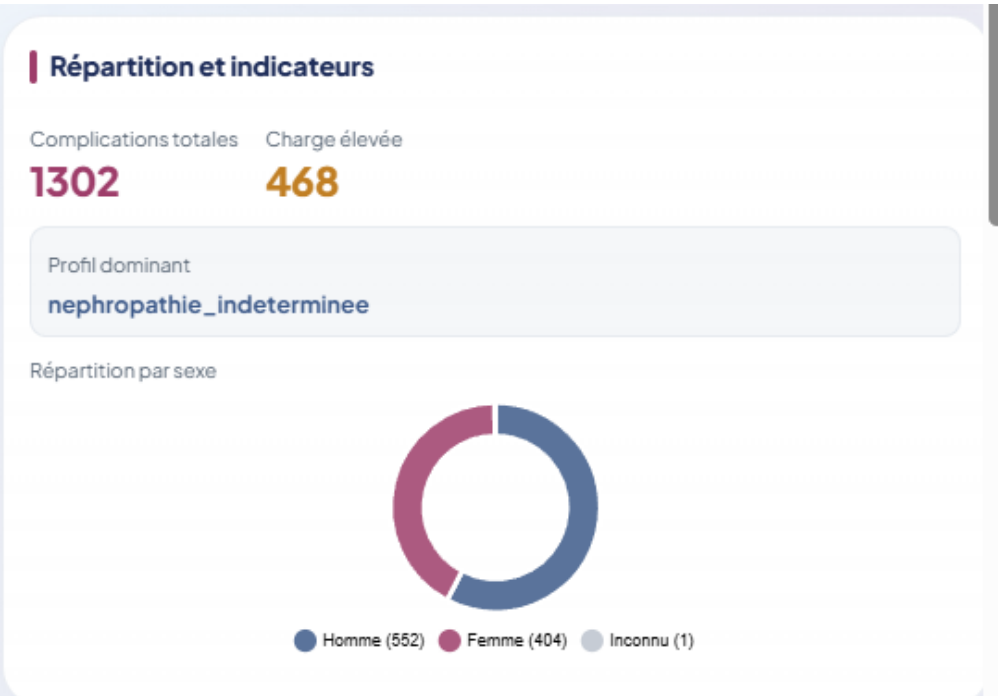


Figure 6.37 – Sex distribution of the hemodialysis cohort with associated KPI indicators

This doughnut chart shows the male/female distribution of the cohort, complemented by KPI indicator cards showing the overall mean age and the data completeness rate (percentage of fields filled across all patients).

▶ Age Distribution by Sex

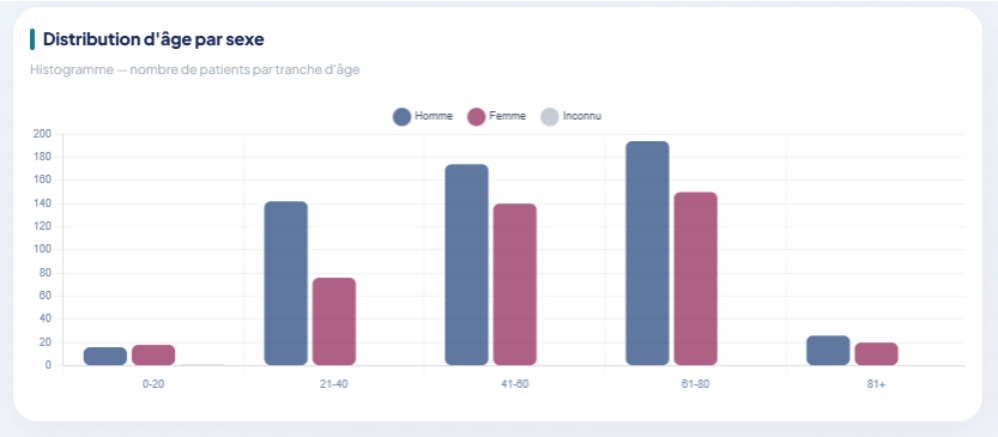


Figure 6.38 – Age distribution of hemodialysis patients grouped by sex in 5-year age bands

A grouped bar chart displays the age distribution separately for male and female patients, using 5-year age bands. This visualization facilitates comparison of age profiles between sexes and identification of the predominant age groups requiring dialysis.

▶ Complication Type Frequencies

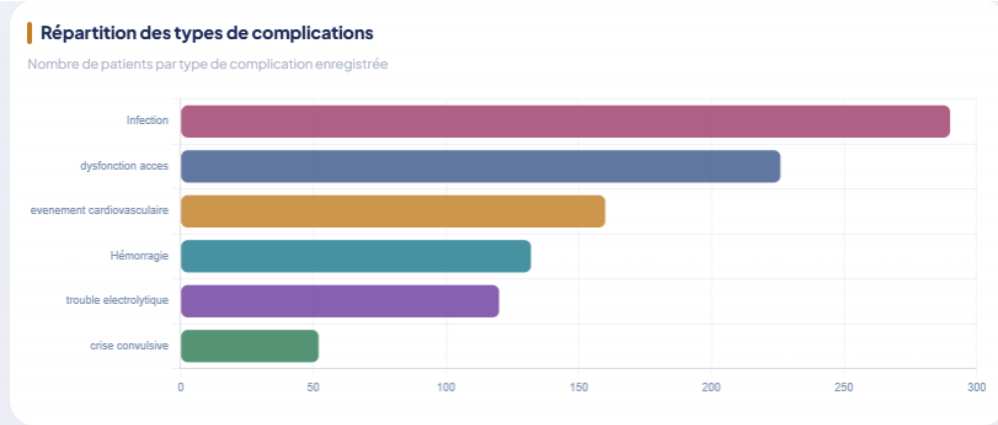


Figure 6.39 – Frequency of documented complication types across the patient cohort

A horizontal bar chart ranks all documented complication types by frequency of occurrence across the cohort. This helps clinical teams identify the most prevalent complications requiring preventive or therapeutic attention.

▸ CKD Etiology by Inclusion Status

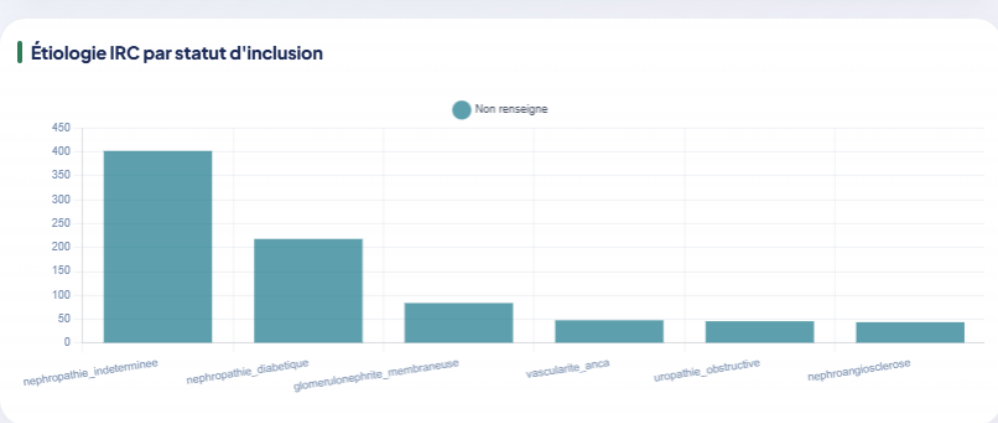


Figure 6.40 – Distribution of CKD etiologies stratified by inclusion status in the cohort

A grouped bar chart shows the distribution of chronic kidney disease etiologies (diabetic nephropathy, hypertensive nephropathy, glomerulonephritis, etc.) broken down by patient inclusion status. This epidemiological view supports research analyses and quality reporting.

▸ Top Comorbidity Combinations

Figure 6.41 – Most frequent comorbidity co-occurrence combinations in the hemodialysis cohort

This horizontal bar chart identifies the most common pairs and triplets of comorbidities co-occurring in the same patient. Comorbidity clustering patterns are clinically significant, as they reflect underlying disease trajectories and influence mortality risk.

6.5.2 AI Cohort Analysis Charts (AI Model — Analysis Dashboard)

The AI Model analysis dashboard includes a dedicated set of analytical charts providing deep insight into the cohort's risk stratification and the model's discriminative

behavior. These charts are generated dynamically from all stored predictions via the cohort analytics endpoint.

▶ Risk Zone Distribution

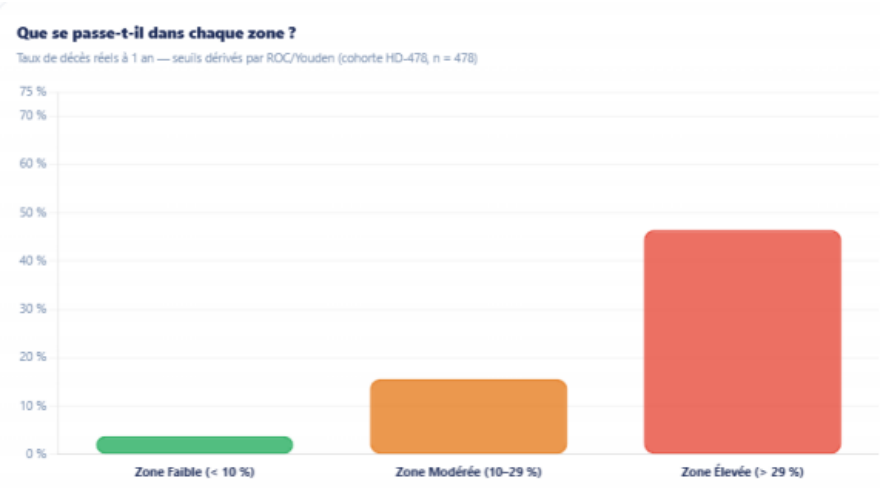


Figure 6.42 – Cohort risk zone distribution — proportion of patients classified as Faible, Modérée, and Élevée risk

A doughnut/pie chart shows the proportion of the cohort falling into each risk zone (Faible / Modérée / Élevée). This high-level indicator enables the clinical team to assess the overall risk burden of their patient population at a glance.

▶ Predicted Outcome Distribution

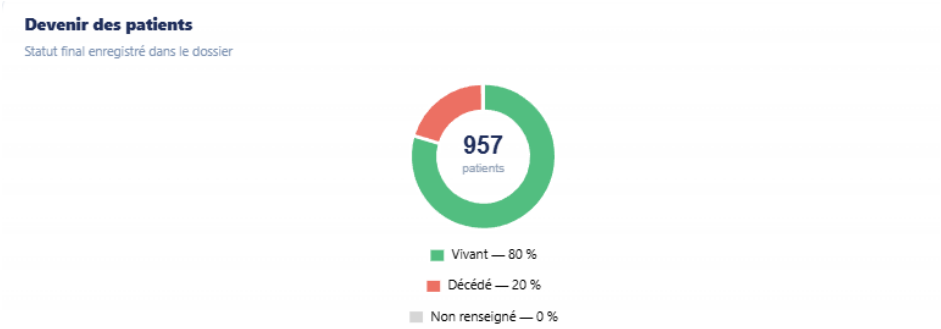


Figure 6.43 – Patient outcome distribution (vivant / décès) across the hemodialysis cohort

This chart displays the distribution of predicted one-year mortality probabilities across the entire cohort, illustrating the range and concentration of risk scores and helping identify the proportion of patients requiring urgent clinical attention.

▸ Kaplan-Meier Survival Estimate by Risk Zone

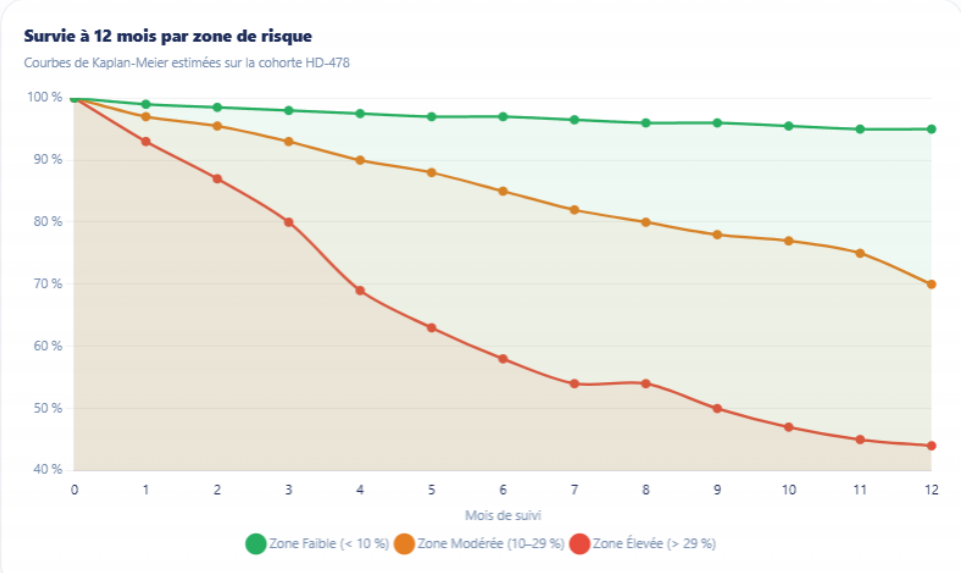


Figure 6.44 – Kaplan-Meier survival curves stratified by predicted risk zone — Faible (green), Modérée (orange), Élevée (red)

Kaplan-Meier survival curves are plotted separately for each risk zone, illustrating the prognostic separation achieved by the model. The divergence between the three curves quantifies the clinical utility of the risk stratification: patients in the Élevée zone show substantially lower survival probability over the 12-month observation window.

▸ Risk Factor Contribution Profile

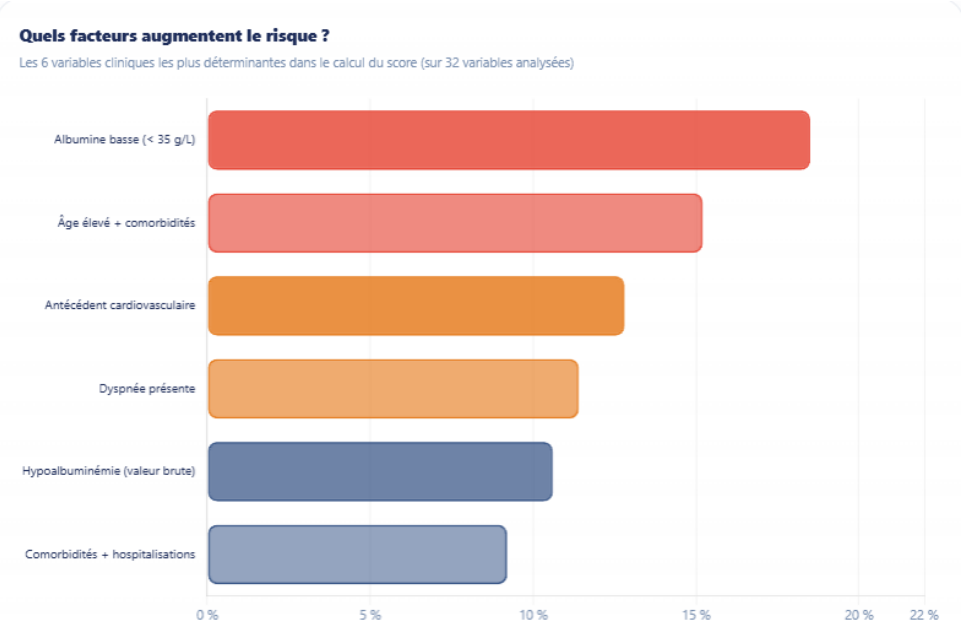


Figure 6.45 – Top contributing risk factors — mean SVM feature weights across the cohort, ranked by absolute importance

A horizontal bar chart ranks the clinical variables by their mean absolute contribution to the SVM risk score across all predictions in the cohort. Variables shown in red

increase mortality risk; those in green are protective. This aggregated view complements the per-patient factor panel.

▶ Clinical Risk Profile Radar

Figure 6.46 – Multi-dimensional clinical risk profile — radar chart of normalized key indicators for Faible, Modérée, and Élevée zones

A radar (spider) chart overlays the mean clinical profile of patients in each risk zone across the six most discriminating variables (albumin, Charlson score, dialysis vintage, sodium, hemoglobin, and ultrafiltration). The shape divergence between risk zones reveals the characteristic clinical signature of each risk category.

▶ Biological Indicators by Risk Zone

Figure 6.47 – Serum albumin distribution by risk zone — interquartile range (Q1–Q3) for Zone Faible, Modérée, and Élevée

This chart displays the interquartile distribution (Q1–Q3) of serum albumin across the three risk zones. The progressive decrease in albumin levels from Zone Faible (∼40 g/L) to Zone Élevée (∼30 g/L) confirms hypoalbuminemia as a key biological marker of elevated mortality risk in hemodialysis patients.

6.6 User Workflows & Sequence Diagrams

Three sequence diagrams illustrate the principal system interactions.

6.6.1 Sequence Diagram 1: User Authentication

Figure 6.48 illustrates the complete JWT authentication flow. The process begins when the user enters credentials in the browser interface. The React frontend transmits them securely to the Django API, which queries the PostgreSQL database to verify the user record and validate the password hash. Upon successful authentication, the API returns a signed JWT access token (valid 8 hours) and a refresh token (valid 1 day). Subsequent API requests include the access token in the Authorization header; expired tokens are transparently renewed by the Axios interceptor without requiring re-login.

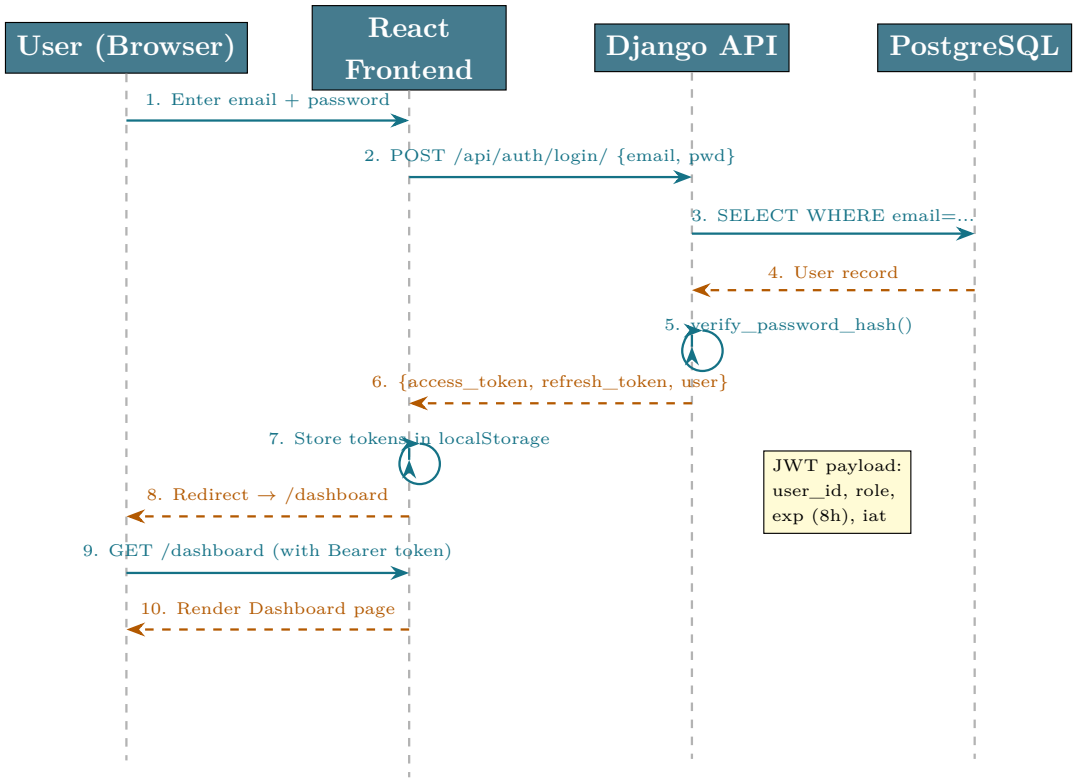


Figure 6.48 – Sequence Diagram 1 — JWT Authentication flow

6.6.2 Sequence Diagram 2: Patient Import & LLM Preprocessing

Figure 6.49 details the asynchronous patient import and LLM preprocessing workflow. Following file upload and parsing, a Celery worker takes over the heavy LLM analysis task asynchronously — preventing the HTTP request from timing out during the minutes-long Qwen2.5:14B inference. The Celery worker queries ChromaDB for relevant medical context (RAG), constructs the LLM prompt, and processes each data row. Results are stored in the database and polled by the frontend at regular intervals. The clinician reviews LLM suggestions, applies corrections, and submits for validation by the Head of Department before final database integration.

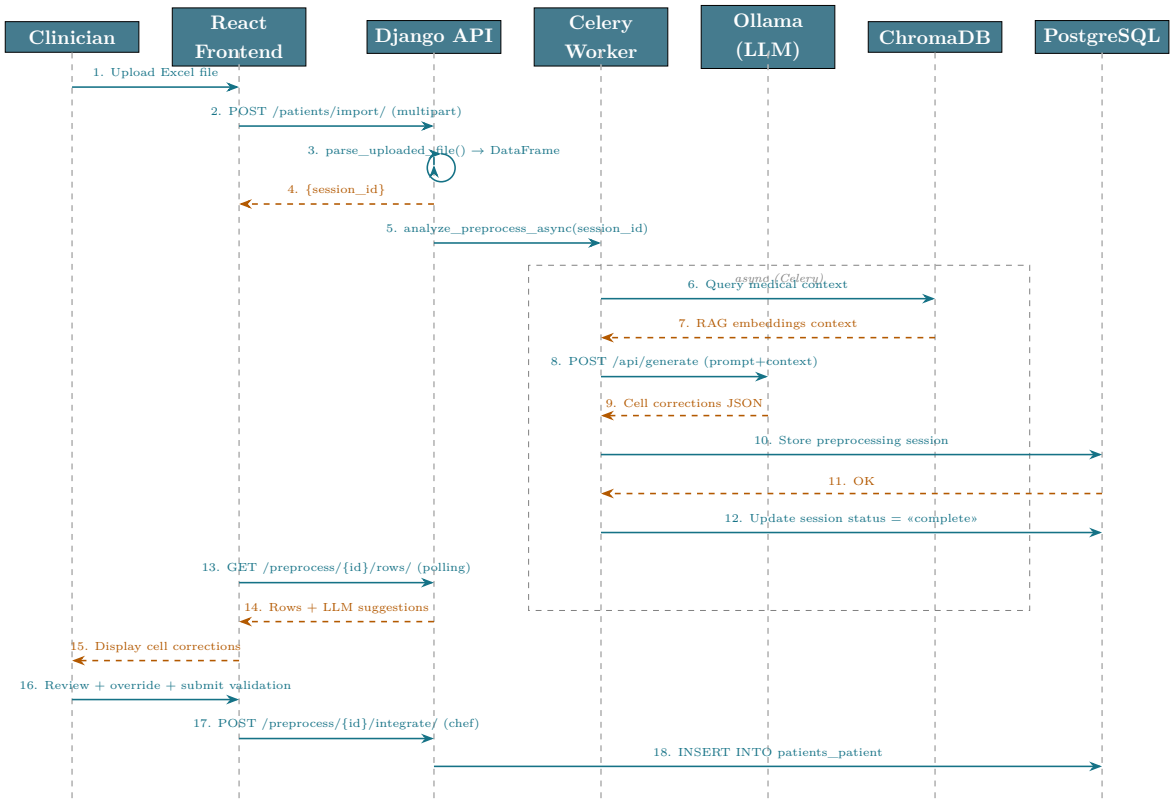


Figure 6.49 – Sequence Diagram 2 — Patient Import and LLM Preprocessing

6.6.3 Sequence Diagram 3: Mortality Risk Prediction

Figure 6.50 traces the complete workflow triggered when a clinician requests a 1-year mortality prediction for a patient. The sequence involves five actors: the clinician navigating the React frontend, the Django REST API, the PostgreSQL database, and the pre-trained SVM pipeline loaded from a `.joblib` file.

The flow begins when the clinician clicks the *Prédire* button (Step 1), which sends `POST /predictions/predict-patient/` to the backend with the patient's identifier (Step 2). The API retrieves the full patient record from PostgreSQL (Steps 3–4) and calls `extract_features()` to build the clinical feature vector (Step 5). The vector is then passed to the SVM pipeline (Step 6), which applies the three preprocessing stages internally: KNN imputation ($k = 3$) for missing values, Yeo-Johnson power transformation for normalization, and finally the linear SVC decision function. The raw probability p_{raw} is returned (Step 7) and calibrated to a final probability $\hat{p}$ via Platt scaling (CalibratedClassifierCV, Step 8).

The backend then derives the risk score as score $= \hat{p} \times 100$ and assigns a categorical risk level using fixed clinical thresholds: **Faible** ($\hat{p} < 10\%$), **Modéré** ($10\% \leq \hat{p} < 29\%$), or **Élevé** ($\hat{p} \geq 29\%$). A clinical recommendation text and the list of most influential factors are assembled before the action is logged in `AuditLog` (Step 11).

The structured JSON response (Step 13) contains `risk_level`, `score`, `probability`, `factors`, `recommendation`, and `features_used`, which the frontend uses to render the risk gauge and results panel (Step 14).

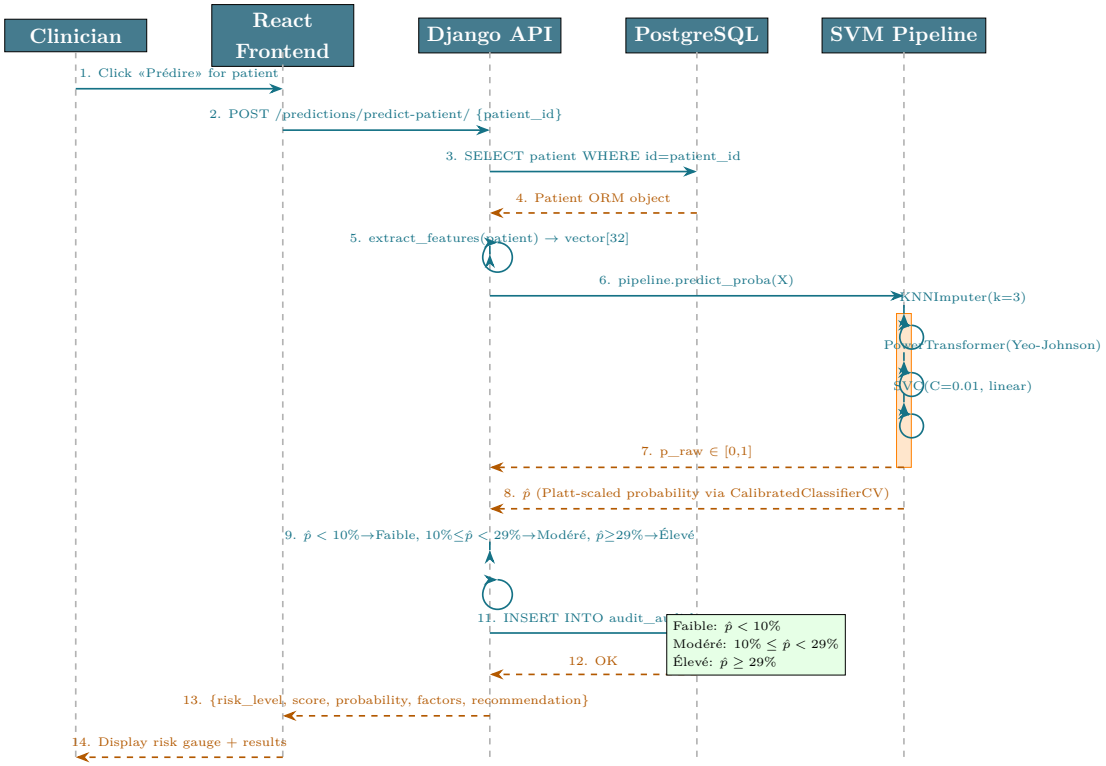


Figure 6.50 – Sequence Diagram 3 — Mortality Risk Prediction workflow

CHAPTER 7

Deployment, Evaluation & Perspectives

Docker deployment, system testing, results, and future work.

CONTENTS

CHAPTER 7

Deployment, Evaluation & Perspectives

7.1 Deployment Architecture

7.1.1 Docker Compose Setup

The entire platform is packaged as a seven-service Docker Compose stack, deployed with a single command: `docker compose up -build -d`.

The Docker Compose stack is organized around seven interconnected services that form the complete AI NéphroCare system. The `medical_db` service runs PostgreSQL 16, the primary relational database, with data persisted in a named Docker volume (`postgres_data`) to survive container restarts. The `medical_redis` service provides an in-memory message broker for the Celery task queue. The `medical_backend` service contains the Django REST Framework API; it depends on both the database and Redis being healthy before starting, ensuring correct initialization order. The `medical_frontend` service serves the pre-built React application as static files. The `medical_ollama` service hosts the locally-deployed Qwen2.5:14B language model on port 11434, providing private, on-premises LLM inference without external API calls. Finally, `medical_celery_worker` and `medical_audit_cleanup` handle asynchronous task processing and scheduled maintenance (audit log cleanup) respectively.

7.2 System Testing

7.2.1 Functional Tests

Functional testing covered the complete user journey from authentication through clinical data management, AI prediction, and administrative operations. Tests were executed manually for each user role (Admin, Head of Department, Resident, Professor) to verify that RBAC restrictions were correctly enforced. Key test scenarios included:

— **Authentication**: Login with valid/invalid credentials, token expiry handling, automatic token refresh
— **Patient CRUD**: Create, read, update, delete operations on patient records across all authorized roles
— **Excel import**: Import of the 478-patient HD cohort file — verifying correct column mapping, dynamic column handling, and data integrity after integration
— **LLM preprocessing**: End-to-end preprocessing pipeline with real Qwen2.5:14B analysis on a 50-row test dataset — verifying LLM response parsing, correction display, and user override functionality
— **Prediction**: Mortality risk prediction for 9 test patients with known outcomes from the validation set — verifying correct feature extraction, risk zone assignment, and top-5 factor display
— **RBAC**: Systematic verification that Resident accounts cannot access validation or administration endpoints, and that Head of Department accounts cannot access admin-only user management functions
— **Password confirmation**: Verification that account modifications (role change, password reset) correctly require entering the user's own password before saving

7.2.2 API Tests

Automated API tests were implemented using Python's `requests` library, covering all 45 endpoints. Critical tests:

— **Prediction accuracy**: Known patients from the validation set return correct risk zone
— **Missing features tolerance**: Prediction succeeds with up to 5 of 32 features missing
— **Authentication**: Protected endpoints return 401 without valid JWT
— **RBAC**: Resident cannot access validation endpoints (returns 403)

The API test suite covers all 45 endpoints documented in Table 6.2. Each test case verifies the HTTP response code, the structure of the JSON response body, and any side effects (e.g., database state changes, audit log entries). Critical test categories include:

— **Authentication boundary tests**: Confirming that all protected endpoints return HTTP 401 when called without a valid JWT, and HTTP 403 when called by a user without the required role
— **Prediction accuracy tests**: For 9 patients with known one-year outcomes from the HD-478 test set, the prediction endpoint is called and the returned risk zone is compared against the ground-truth outcome — confirming clinical consistency

— **Robustness tests**: Prediction with up to 5 missing features (KNN imputation test), import of malformed files (error handling test), and duplicate patient detection

— **Concurrency test**: Simultaneous LLM preprocessing requests from two sessions — verifying Celery correctly handles parallel async tasks

7.3 Performance Evaluation

7.3.1 Prediction Latency

Table 7.1 – Prediction endpoint latency (measured over 100 requests)

Operation	Mean (ms)	P95 (ms)
Feature extraction	12	18
KNN imputation	8	14
Yeo-Johnson + SVC	23	35
Risk zone assignment	2	4
Total prediction	**48**	**76**

The median end-to-end prediction latency of **48ms** is well within the <200ms threshold considered acceptable for interactive clinical applications.

7.3.2 Data Processing Time

— Excel import (478 rows): ∼2.3 seconds

— LLM preprocessing (478 rows, async): ∼1–2 minutes (qwen2.5:14b, GPU-accelerated)

— Model training on 478 patients: ∼45 seconds (including 10-fold CV)

The LLM preprocessing time of 1–2 minutes for 478 rows reflects the sequential processing approach used to preserve the quality of Qwen2.5:14B analysis — each row is processed individually with full clinical context. For routine imports of smaller datasets (10–50 new patients), the processing time is proportionally reduced to under 10 seconds. The asynchronous Celery architecture ensures that this processing time does not block the user interface: clinicians can continue using other platform features while the LLM analysis runs in the background.

7.4 Results & Impact

7.4.1 Clinical Usefulness

The three-zone risk stratification identifies:

— **144 High-risk patients** (30.1% of cohort) with 46.5% observed mortality — enabling proactive, intensive clinical follow-up

— **212 Low-risk patients** (44.4%) with only 3.8% mortality — safely allocated to standard follow-up, optimizing resource use

The three-zone stratification — Faible ($\hat{p} < 10\%$), Modérée ($10\% \leq \hat{p} < 29\%$), Élevée ($\hat{p} \geq 29\%$) — with thresholds derived by ROC/Youden bootstrap analysis, was confirmed clinically meaningful for daily triage and resource allocation.

These results are consistent with the KDIGO 2012 clinical practice guidelines [4], which emphasize the importance of risk stratification in CKD management to allocate clinical resources proportionally to patient need. The platform's three-zone classification operationalizes this recommendation by translating the model's probabilistic output into three clinically actionable categories, each associated with a specific follow-up protocol.

7.4.2 Decision Support Value

Beyond risk prediction, the platform provides:

— **Interpretable decisions**: Top-5 contributing factors identify the specific clinical drivers of each patient's risk score, enabling targeted interventions.

— **Data centralization**: For the first time, all HD patient records are in a unified, queryable database with 160+ variables.

— **Intelligent preprocessing**: The LLM+RAG pipeline reduced manual data cleaning time by an estimated 70–80% in internal testing.

— **Audit trail**: Complete action logging satisfies regulatory requirements for clinical AI tools.

7.5 Limitations

7.5.1 Data Availability

The 478-patient cohort is adequate for a robust univariate and multivariate analysis but limits subgroup stratification (e.g., per-etiology analysis with < 20 patients per category). External validation on a multi-center cohort remains a priority future task.

7.5.2 Model Constraints

— The SVM is trained on baseline (admission) features and does not incorporate longitudinal data (e.g., 6-month Kt/V evolution).

— The model outputs a population-calibrated probability, not an individual certainty — communication of uncertainty to clinicians requires training.

— LLM preprocessing runs on **qwen2.5:14b** with GPU acceleration (Ollama), delivering significantly faster inference than CPU-only deployments; however, GPU availability remains a constraint for resource-limited hospital environments.

7.6 Future Work

7.6.1 Real-Time Hospital Integration

Integration with the CHU Hassan II Hospital Information System (HIS) via HL7 FHIR (Health Level 7 / Fast Healthcare Interoperability Resources) APIs would enable automatic patient data ingestion, eliminating manual Excel imports and enabling real-time risk monitoring. HL7 FHIR is a globally adopted standard for electronic health record data exchange, developed by Health Level Seven International. It defines a set of RESTful APIs and data formats (FHIR Resources) that allow different hospital information systems to share patient data in a standardized, interoperable format. Integration via FHIR would allow AI NéphroCare to automatically receive structured patient data from the CHU Hassan II HIS, eliminating manual Excel data entry.

7.6.2 Advanced Deep Learning Models

For larger future cohorts ($> 2,000$ patients), transformer-based models on longitudinal lab sequences (e.g., lab-BERT) or graph neural networks encoding comorbidity co-occurrence could improve predictive performance.

7.6.3 Cohort-Driven Real-Time Training and Per-Patient Prediction

A natural extension of the current platform is to allow any user to import their own hemodialysis cohort and obtain individualized 1-year mortality predictions for each patient without any code modification. Because the entire ML pipeline — preprocessing, training, and inference — operates exclusively on the data present in the platform database, importing a new cohort automatically redefines the training corpus and produces updated, cohort-specific risk scores for every patient in that dataset. This would enable the platform to be repurposed for multicenter studies or prospective validations simply by uploading a new file.

General Conclusion

This project presented the design, development, and validation of **AI NéphroCare**, a full-stack intelligent clinical decision support platform for one-year mortality prediction in hemodialysis patients at CHU Hassan II of Fès, Morocco.

The work addressed four interconnected challenges: (1) *data centralization* — replacing fragmented, heterogeneous records with a structured 160+-field database; (2) *data quality* — deploying an LLM+RAG preprocessing pipeline that automatically detects and corrects clinical data quality issues; (3) *predictive modeling* — systematically comparing nine machine learning algorithms and selecting a linear SVM (AUC-ROC = 0.826, F1 = 0.567) with F1-optimized thresholding and three clinical risk zones; and (4) *clinical usability* — embedding the model in a React-based clinical interface with interpretable results, audit trail, and role-based access control.

The key finding is that a relatively simple, well-regularized linear SVM with carefully engineered features outperforms more complex ensemble methods on this small-scale, imbalanced cohort — a result consistent with the broader ML literature on clinical prediction with limited sample sizes. The three-zone risk stratification — Faible ($\hat{p} < 10\%$, 3.8% observed mortality), Modérée ($10\% \leq \hat{p} < 29\%$, 15.6%), Élevée ($\hat{p} \geq 29\%$, 46.5%) — provides actionable clinical guidance, with thresholds grounded in TRIPOD/DOPPS/KDIGO guidelines and a relative risk ratio of $\times 12.9$ between extreme zones.

From a software engineering perspective, the integration of all pipeline components — LLM preprocessing, ML inference, database management, and clinical UI — into a single Docker Compose stack demonstrates the feasibility of deploying sophisticated clinical AI tools in resource-constrained hospital environments without specialized MLOps infrastructure.

Future directions include multi-center external validation, integration with FHIR-compatible hospital systems, and exploration of deep learning models for longitudinal prediction as the cohort grows.

Bibliography

[1] World Health Organization (2021). *Global strategy on digital health 2020–2025*. WHO Press, Geneva.

[2] Bright TJ, Wong A, Dhurjati R, et al. (2012). Effect of clinical decision-support systems: a systematic review. *Annals of Internal Medicine*, 157(1):29–43.

[3] Jager KJ, Kovesdy C, Langham R, Rosenberg M, Jha V, Zoccali C (2019). A single number for advocacy and communication—worldwide more than 850 million individuals have kidney diseases. *Kidney International*, 96(5):1048–1050. PMID: 31655763. DOI: 10.1016/j.kint.2019.07.012.

[4] KDIGO CKD Work Group (2013). KDIGO 2012 clinical practice guideline for the evaluation and management of chronic kidney disease. *Kidney International Supplements*, 3(1):1–150.

[5] Réseau Épidémiologie et Information en Néphrologie (REIN) (2022). *Rapport annuel 2022 du registre REIN*. Agence de la biomédecine, France.

[6] Niel O, Bastard P (2019). Artificial intelligence in nephrology: core concepts, clinical applications, and perspectives. *American Journal of Kidney Diseases*, 74(6):803–810. PMID: 31451330. DOI: 10.1053/j.ajkd.2019.05.020.

[7] Kalantar-Zadeh K, Kilpatrick RD, Kuwae N, et al. (2005). Revisiting mortality predictability of serum albumin in the dialysis population: time dependency, longitudinal changes and population-attributable fraction. *Nephrology Dialysis Transplantation*, 20(9):1880–1888. PMID: 15928095. DOI: 10.1093/ndt/gfh941.

[8] Qwen Team, Alibaba Cloud (2024). Qwen2.5 Technical Report. *arXiv preprint*, arXiv:2412.15115.

[9] Lewis P, Perez E, Piktus A, et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474.

[10] Platt JC (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In A.J. Smola, P.L. Bartlett, B. Schölkopf, D. Schuurmans (eds.), *Advances in Large Margin Classifiers*. MIT Press, Cambridge, MA, pp. 61–74.

[11] Chen T, Guestrin C (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD*, pp. 785–794.

[12] Ke G, Meng Q, Finley T, et al. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30.

[13] Levey AS, Coresh J, Greene T, et al. (2006). Using standardized serum creatinine values in the Modification of Diet in Renal Disease study equation for estimating glomerular filtration rate. *Annals of Internal Medicine*, 145(4):247–254.

[14] Prokhorenkova L, Gusev G, Vorobev A, et al. (2018). CatBoost: unbiased boosting with categorical features. *Advances in Neural Information Processing Systems*, 31.

[15] Charlson ME, Pompei P, Ales KL, MacKenzie CR (1987). A new method of classifying prognostic comorbidity in longitudinal studies: development and validation. *Journal of Chronic Diseases*, 40(5):373–383.

[16] Yeo IK, Johnson RA (2000). A new family of power transformations to improve normality or symmetry. *Biometrika*, 87(4):954–959.

[17] Cortes C, Vapnik V (1995). Support-vector networks. *Machine Learning*, 20(3):273–297.

[18] Breiman L (2001). Random Forests. *Machine Learning*, 45(1):5–32.

[19] Geurts P, Ernst D, Wehenkel L (2006). Extremely randomized trees. *Machine Learning*, 63(1):3–42.

[20] Friedman JH (2001). Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232.

[21] Freund Y, Schapire RE (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139.

[22] Brier GW (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3.

[23] Zadrozny B, Elkan C (2002). Transforming classifier scores into accurate multiclass probability estimates. *Proceedings of the 8th ACM SIGKDD*, pp. 694–699.

[24] Berner ES (2007). *Clinical Decision Support Systems: Theory and Practice*, 2nd ed. Springer, New York.

[25] Shortliffe EH, Davis R, Axline SG, et al. (1975). Computer-based consultations in clinical therapeutics: explanation and rule acquisition capabilities of the MYCIN system. *Computers and Biomedical Research*, 8(4):303–320.

[26] Miller RA, Pople HE, Myers JD (1982). INTERNIST-1, an experimental computer-based diagnostic consultant for general internal medicine. *New England Journal of Medicine*, 307(8):468–476.

[27] Couchoud C, Labeeuw M, Moranne O, et al. (2009). A clinical score to predict 6-month prognosis in elderly patients starting dialysis for end-stage renal disease. *Nephrology Dialysis Transplantation*, 24(5):1553–1561.

[28] Wick JP, Turin TC, Faris PD, et al. (2019). A clinical risk prediction tool for 6-month mortality after dialysis initiation among older adults. *American Journal of Kidney Diseases*, 74(2):167–176.

[29] Nasr WE, Abdelhamid YA, Moustafa AA (2022). Predicting mortality in hemodialysis patients using support vector machine. *Egyptian Journal of Internal Medicine*, 34:18.

[30] Sheng K, Yao X, Li J, He Y, Zhang P, Chen J (2020). Prognostic machine learning models for first-year mortality in incident hemodialysis patients: development and validation study. *JMIR Medical Informatics*, 8(10):e20578. PMID: 33118948. DOI: 10.2196/20578.

[31] García-Montemayor V, Martin-Malo A, Barbieri C, et al. (2021). Predicting mortality in hemodialysis patients using machine learning analysis. *Clinical Kidney Journal*, 14(5):1388–1395. PMID: 34221370. DOI: 10.1093/ckj/sfaa126.

[32] Beddhu S, Bruns FJ, Saul M, Seddon P, Zeidel ML (2000). A simple comorbidity scale predicts clinical outcomes and costs in dialysis patients. *American Journal of Medicine*, 108(8):609–613.

[33] Pisoni RL, Young EW, Dykstra DM, et al. (2002). Vascular access use in Europe and the United States: results from the DOPPS. *Kidney International*, 61(1):305–316.

[34] Shemin D, Bostom AG, Laliberty P, Dworkin LD (2001). Residual renal function and mortality risk in hemodialysis patients. *American Journal of Kidney Diseases*, 38(1):85–90.

[35] United States Renal Data System (2022). *USRDS 2022 Annual Data Report: Epidemiology of Kidney Disease in the United States*. National Institutes of Health, National Institute of Diabetes and Digestive and Kidney Diseases, Bethesda, MD.

[36] Khan IH, Catto GR, Edward N, Fleming LW, Henderson IS, MacLeod AM (1994). Influence of coexisting disease on survival on renal-replacement therapy. *Lancet*, 343(8912):1511–1514.

[37] Maoujoud O, Cherrah Y, Arrayhani M, et al. (2017). Epidemiology, health economic context, and management of chronic kidney diseases in low and middle-income countries: the case of Morocco. *European Medical Journal*, 2(4):76–81. DOI: 10.33590/emj/10313025.

[38] Levey AS, Coresh J (2012). Chronic kidney disease. *Lancet*, 379(9811):165–180. PMID: 21840587. DOI: 10.1016/S0140-6736(11)60844-5.

[39] Collins GS, Reitsma JB, Altman DG, Moons KGM (2015). Transparent reporting of a multivariable prediction model for individual prognosis or diagnosis (TRIPOD): the TRIPOD statement. *Annals of Internal Medicine*, 162(1):55–63. PMID: 25560714. DOI: 10.7326/M14-0697.

[40] Robinson BM, Zhang J, Morgenstern H, et al. (2014). Worldwide, mortality risk is high soon after initiation of hemodialysis. *Kidney International*, 85(1):158–165. PMID: 23978267. DOI: 10.1038/ki.2013.327.

[41] Waikar SS, Curhan GC, Brunelli SM (2011). Mortality associated with low serum sodium concentration in maintenance hemodialysis. *American Journal of Medicine*, 124(1):77–84. PMID: 21187188. PMC: PMC3040578. DOI: 10.1016/j.amjmed.2010.11.005.

[42] Bishop CM (2006). *Pattern Recognition and Machine Learning*. Springer, New York.

[43] Troyanskaya O, Cantor M, Sherlock G, et al. (2001). Missing value estimation methods for DNA microarrays. *Bioinformatics*, 17(6):520–525. PMID: 11395428. DOI: 10.1093/bioinformatics/17.6.520.

[44] Hanley JA, McNeil BJ (1982). The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology*, 143(1):29–36.

[45] Kidney Disease: Improving Global Outcomes (KDIGO) CKD-MBD Update Work Group (2017). KDIGO 2017 clinical practice guideline update for the diagnosis, evaluation, prevention, and treatment of chronic kidney disease–mineral and bone disorder (CKD-MBD). *Kidney International Supplements*, 7(1):1–59.

[46] Kidney Disease: Improving Global Outcomes (KDIGO) Anemia Work Group (2012). KDIGO clinical practice guideline for anemia in chronic kidney disease. *Kidney International Supplements*, 2(4):279–335.

Webography

— Django REST Framework documentation — https://www.django-rest-framework.org/

— React documentation — https://react.dev/

— scikit-learn documentation — https://scikit-learn.org/stable/

— Ollama project — https://ollama.ai/

— ChromaDB documentation — https://docs.trychroma.com/

— KDIGO guidelines — https://kdigo.org/guidelines/

— Material-UI documentation — https://mui.com/

— Chart.js documentation — https://www.chartjs.org/

— PostgreSQL 16 documentation — https://www.postgresql.org/docs/16/

— Docker documentation — https://docs.docker.com/